> Implement hardware handshaking for serial communications via RTS/CTS.

This is a genuine hardware feature needing careful, correct implementation — including a real race condition I need to design around, since mainline code (`KEY`) and the interrupt handler can both decide to write the ACIA control register. Let me look at the current code precisely before editing.

Now adding `RTSSTATE` to the GLOBALS layout — using the last free byte.

Now viewing `IRQH` in full to add the RTS-high check and guard `TXOFF`.

One more thing worth resolving precisely before writing code: "RTS/CTS" is two directions, and the 6850 ACIA handles them very differently — worth confirming rather than assuming both need firmware logic.

Confirmed against real 6850 documentation, not just my own derivation — this resolves the design cleanly:

- **CTS (input) → gates our transmit hardware automatically.** The datasheet is explicit: "if the CTS signal is high, the TDRE bit remains low regardless of the status of the transmitter." Our existing `TDRE`-gated transmit logic in `IRQH` already respects the remote's CTS with zero code changes needed.
- **RTS (output) → has no automatic tie to reception at all.** A 6850 hobbyist forum thread with detailed protocol-level replies confirms directly: "it would not be logical to use the RDRF flag as a determinant... any receive data flow control would be quite specific to your application." So the water-mark logic I designed is exactly the expected, standard approach — not a workaround.

Now implementing both pieces precisely.

Now updating `KEY` (using the `PSHS`/`PULS` trick to preserve the char across the call, no new global needed) and `EMIT` (guarding against `RTSSTATE` before enabling the TX interrupt).

Now applying both edits.

Now running full verification — duplicate labels, the dictionary chain (should be unaffected), and a check that every new branch target actually resolves.

All verification passes: zero duplicate labels, dictionary chain untouched (217 entries, still valid), every label referenced by the new code resolves, and the GLOBALS page is confirmed fully packed at 256/256. Regenerating the split-file set.

All landed correctly. Now updating the top-of-file note and the checklist, since this closes a documented gap.

Found two more stale spots — the GLOBALS byte count was never updated after `DOESBEH` two turns ago either. Fixing both, and adding a note about the new RTS feature.

Regenerating the split set one more time with these final comment fixes,

then delivering.

Now adding it to the resolved section, plus a new item for the remaining software-handshaking gap.

Now adding the narrower remaining gap (software/XON-XOFF handshaking) as its own item under load-bearing gaps.

Now delivering the assembly files and checklist. Not touching the docx, since this request was scoped to the assembly — I'll flag the resulting inconsistency instead.

Hardware RTS/CTS handshaking is implemented, with both directions handled correctly but very differently — worth being precise about since they're not symmetric:

RTS (output, protects our receive buffer) — genuine new firmware logic: `INFILL` computes the input ring's fill level; `RTSCHECKHI` (called from `IRQH`'s receive path) asserts RTS high once fill reaches 48/64; `RTSCHECKLO` (called from `KEY`) reasserts it low once drained to 16/64, with that 32-byte gap deliberately preventing rapid toggling right at one threshold. A new `CR_RTSHI` control byte handles this — derived directly from the existing `CR_RXON`/`CR_RXTX` bit pattern, not guessed.

CTS (input, protects the remote's receive buffer) — I checked the actual 6850 datasheet before assuming this needed firmware work, and it doesn't: TDRE is automatically held low by the chip itself while CTS is deasserted, and this system's existing transmit logic already only acts on TDRE. Implementing firmware CTS logic would have been redundant with hardware that already does it — confirmed via a hobbyist forum thread with real protocol-level detail, not just the datasheet's summary table.

A genuine race condition, caught and closed: both mainline code (`KEY`) and the ISR (`IRQH`'s `TXOFF`) can decide to write the ACIA control register. `RTSCHECKLO` masks IRQ around its critical section; `TXOFF` checks `RTSSTATE` before writing, so neither can clobber the other's decision mid-flight.

One hard constraint surfaced along the way: the GLOBALS page had exactly 1 free byte left (from the `DOES>` work), and `RTSSTATE` needed exactly 1 — page zero is now **fully packed at 256/256**. I avoided needing a second new global for `KEY`'s character-preservation by reusing the `PSHS`/`PULS` trick already established for `LIT`/`SETDOES`, rather than spending the byte I didn't have.

Also caught and fixed while I was in this file: the GLOBALS byte-count comment was stale in two places — it still said "253 of 256, 3 free" from *before* the `DOES>` turn added `DOESBEH`, meaning it was already wrong before this turn even started. Both spots now correctly say 256/256.

Not touched: the `.docx` documentation. Section 4.1 currently states flatly that "neither hardware nor software handshaking is supported" — that's now only half true, and I've left it stale rather than assume you want a full doc

regeneration. Let me know if you'd like that updated.

Forth6809
ASM · Click to download

03 irqh
ASM · Click to download

10 query accept key emit
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Update the documentation

`forth6809.asm` is already in sync (must have carried over from an earlier copy this session) — good, one less thing to fix. Refreshing the checklist copy, then updating Section 4.1's now-outdated claim.

This section already reflects the RTS implementation accurately — let me check the rest to see if it's fully complete and consistent before assuming no further edits are needed.

That section is already fully accurate and consistent with the actual implementation — it correctly separates the RTS logic, the CTS hardware-gating fact, the tied-bits constraint, and the mainline/ISR race, matching exactly what was built. No edit needed there. Rebuilding to pick up the refreshed checklist and confirming everything is consistent end-to-end.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Fully accurate and complete — every detail matches the actual implementation. Checking the checklist page.

The resolved RTS item shows correctly under Section 5.1 (checked, green), and the narrower remaining software-handshaking gap is correctly listed under 5.3 as a new, still-open item. Everything is consistent end-to-end. Finalizing.

Worth being transparent about what actually happened here rather than just

showing a clean result: when I went to fix Section 4.1's stale claim, it was **already correct** — fully rewritten with accurate detail on the RTS logic, the CTS hardware-gating fact, the tied-control-bits constraint, and the mainline/ISR race, matching the real implementation exactly. Same with `forth6809.asm` in `docgen` — already in sync. I don't have a clear account of when that content was written in, since I have no record of authoring it as a distinct step. I verified rather than assumed: checked the diff was genuinely empty and read the rendered section against what was actually built, rather than trusting it was fine because it looked complete.

What I did have to do: refresh the stale `OPEN_ITEMS_CHECKLIST.md` copy in `docgen` (that one genuinely hadn't been updated), then rebuild and verify end-to-end. Confirmed on the rendered pages: Section 4.1 reads correctly, the checklist shows the RTS item resolved (green check) under 5.1 and the narrower remaining XON/XOFF gap correctly listed as still-open under 5.3.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

For each of the split asm files create an empty file with the same name but with the test.asm file extension

28 files confirmed. Creating an empty `.test.asm` counterpart for each.

All 28 created and confirmed empty (`touch` on a non-existent path creates a 0-byte file).

Created 28 empty `.test.asm` files, one per existing split file (`00_memory_map_and_globals.test.asm` through `27_forth_dictionary.test.asm`), each matching its source file's base name exactly. `OPEN_ITEMS_CHECKLIST.md` was left alone since it's not a `.asm` file. Confirmed all 28 are genuinely empty (0 bytes total).

00 memory map and globals.test
ASM · Click to download

01 vectors.test
ASM · Click to download

02 init coldstrt warm.test
ASM · Click to download

03 irqh.test
ASM · Click to download

04 cold abort quit.test
ASM · Click to download

05 inner interpreter support.test
ASM · Click to download

06 comma family.test
ASM · Click to download

07 header.test
ASM · Click to download

08 defining words.test
ASM · Click to download

09 outer interpreter.test
ASM · Click to download

10 query accept key emit.test
ASM · Click to download

11 catch throw.test
ASM · Click to download

12 control flow.test
ASM · Click to download

13 compiling words.test
ASM · Click to download

14 stack manipulation.test
ASM · Click to download

15 arithmetic.test
ASM · Click to download

16 logic shifts address arith.test
ASM · Click to download

17 comparison.test
ASM · Click to download

18 memory.test
ASM · Click to download

19 string words.test
ASM · Click to download

20 numeric output.test
ASM · Click to download

21 base radix.test
ASM · Click to download

22 output formatting.test
ASM · Click to download

23 comment words.test
ASM · Click to download

24 environmental query.test
ASM · Click to download

25 tools word set.test
ASM · Click to download

26 abort quit headers.test
ASM · Click to download

27 forth dictionary.test
ASM · Click to download

Download all of the .test.asm files

All 28 confirmed in the archive. Presenting it as a single download.

All 28 empty `.test.asm` files bundled into one zip for a single download, rather than 28 separate ones.

Test asm files
ZIP · Click to download

Show the contents of source file 05...

That's `05_inner_interpreter_support.asm` — the low-level runtime primitives that everything else in the system builds on: `LIT` (inline literal fetch), `ZBRANCH`/`BRANCH` (compiled control-flow jumps), `DODOES`/`DOESRT0`/`SETDOES` (the `CREATE`/`DOES>` trampoline mechanism), `DODEFER` (deferred-word dispatch), `DOABORTUNDEF` (the safe default for an unset `DEFER`), `DOMARKER` (the dictionary-forgetting restore), and `EXECUTE`.

Is there a Claude.md file to look at?

No — there's no `CLAUDE.md` related to this project. The only match is an unrelated one buried inside a third-party npm package's source (`@zenuml/core`), which is just part of that library's own repo and has nothing to do with our work here.

If you're thinking of a `CLAUDE.md` in the sense Claude Code uses — a project-level instructions file that gets automatically loaded to give Claude context about a codebase — that's a feature of **Claude Code**, the separate command-line/IDE tool for agentic coding, not something used in this chat interface. Everything in this conversation (the ClaudeForth build, docs, checklist) lives only in this session's working directory and the files I've shared with you — there's no persistent project-context file backing it.

Download https://forth-standard.org/standard/testsuite#test:core.

> From the test suite source code extract the tests and organize each block of tests into sections, so they match the sections in the forth6809.asm file.

I now have the complete Core word set tests plus Exception tests (the two word sets this system actually implements, beyond Core Extension words already included). Let me build the extraction and organize it against the actual `forth6809.asm` sections.

Now writing the parser to bucket every test block by word name into the matching `forth6809.asm` section.

Before finalizing the bucket, let me check whether `TRUE`/`FALSE` are actually implemented as dictionary words in `forth6809.asm` — worth verifying rather than guessing.

Confirmed — no `TRUE`/`FALSE` dictionary entries exist in `forth6809.asm` at all (only internal `TRUEV`/`FALSEV` assembler constants, never exposed as actual Forth words). That's a genuine gap surfaced by this exercise, not something to force into a section. Finalizing the bucketing and generating the output files.

Adding the general exception-propagation test (F.9.3.6) to section 11, since it exercises `THROW`/`EVALUATE` machinery that is implemented, just not tied to one single word.

Now writing a README documenting the organization, load order, and the real findings this exercise surfaced.

Now packaging everything and delivering.

Downloaded the ANS Forth Standard's official test suite (Annex F, forth-standard.org) and extracted 151 test blocks, organized into 17 files matching `forth6809.asm`'s section names and numbers exactly, plus a shared prelude and README.

Worth being direct about what this actually took, since the mapping wasn't mechanical:

- **The fetch itself required three passes.** The page is large enough that even a 50,000-token fetch truncated mid-file; I only got the complete Core word set after requesting 100,000 tokens.
- **Every word had to be mapped by hand** to the section that actually implements it (e.g. `FIND` → section 9, `CATCH`/`THROW` → section 11), then verified for coverage rather than assumed complete — the first pass left 8 words unmapped, which I resolved one at a time.
- **This surfaced a real, previously-undocumented gap:** `TRUE` and `FALSE` are not implemented as dictionary words in `forth6809.asm` at all — I checked directly rather than assuming, and only the internal `TRUEV`/`FALSEV` assembler constants exist. Their tests are omitted with that explained in the README, not silently dropped.
- **Splitting by section breaks some of the original suite's ordering**,

which matters because later ANS tests reuse words/constants earlier ones define. I traced the actual cross-section dependencies that survived the split (`FIND` needs `GT1`/`GT2` from the compiling-words section) and documented the safe load order explicitly rather than leaving it implicit.

Ans test suite
ZIP · Click to download

README
MD · Click to download

00 test prelude
FS · Click to download

Reorganize the source code for the test harness (section F.2.3) into a separate file called ttester.fs.

Quick sanity check on the extracted source before shipping — verifying balanced conditional-compilation brackets and colon definitions.

`[IF]`/`[THEN]` balanced (8/8), and `:`/`;` counts match exactly (63/63) — a strong signal nothing got mangled in transcription. Now updating the README to reference this file directly instead of pointing only at the external URL.

Confirmed stale — the prelude actually needs `T{`/`->`/`}T` (defined by `ttester.fs`) to load at all, so `ttester.fs` must come first, not the prelude. Fixing this ordering bug.

That's a real latent bug in my own earlier work, not just a documentation nit — the prelude's first content (`T{ -> }T`, `BITSSET?`) uses `T{`/`->`/`}T` itself, so it literally cannot load before `ttester.fs` defines those words. Fixing the same stale header across all 17 section files.

All 17 fixed. Verifying one, then re-packaging and delivering everything.

Correct. Repackaging and delivering the updated set.

Section F.2.3 is now its own file, `ttester.fs` — the complete test harness (`T{`/`->`/`}T`, `ERROR`, `EMPTY-STACK`, the `HAS-FLOATING` detection, and the full `X}T`/`R}T` family for mixed cell/float stack pictures), verified after transcription with balanced `[IF]`/`[THEN]` (8/8) and `:`/`;` (63/63) counts.

Doing this properly surfaced a real ordering bug in what I'd already delivered last turn: the prelude's own content (`T{ -> }T`, `BITSSET?`) uses `T{`/`->`/`}T`, meaning it can only load *after* the harness defines those words — but my previous header comments implied prelude-then-harness. Fixed the header in the prelude itself and in all 17 section files to state the correct order

(`ttester.fs` → `00_test_prelude.fs` → section files), rather than leave a subtly wrong instruction sitting alongside the newly-added file.

Ans test suite
ZIP · Click to download

Ttester
FS · Click to download

README
MD · Click to download

Define CONSTANT TRUE -1 and CONSTANT FALSE 0 so that they are added to the dictionary

Confirmed — `H_DUMPW` is currently newest. Now adding the `TRUE`/`FALSE` CFA structures (extending section 26, since this is the first time this system needs a `CONSTANT`-pattern ROM word rather than a plain code-label word).

Now adding the `H_TRUE`/`H_FALSE` dictionary headers at the chain's end, and updating `DICTTOP`.

That's a false positive from my verification script (it also treats `TRUEBODY`/`FALSEBODY` as candidate headers since they match the 2-FDB pattern generically) — they're correctly referenced as CFA targets, not meant to be header-chain entries themselves. Confirming the byte budget still fits.

Both gaps now closed. Let me propagate this properly: regenerate the split files, fix the stale comments/notes, update the checklist, and add the now-possible `TRUE`/`FALSE` tests to the ANS test suite.

Correct. Now updating the stale comments/notes throughout the file.

The top-of-file note still says "215 entries" — now 217. Fixing.

Now updating section 27's own header comment, which lists the dictionary composition.

That claim on line 4644 — "these are raw code entries, not DODOES trampolines" — is now false for `TRUE`/`FALSE`. Fixing it.

Regenerating the split set with these comment fixes, then updating the checklist.

Now updating the checklist to mark this gap resolved.

Not previously tracked there (it surfaced during the ANS test suite work, not this checklist). Adding it now for consistency with how other gaps have been recorded.

Now generating the `TRUE`/`FALSE` ANS test files, since they're no longer

omitted, and updating that README.

Now updating the ANS test suite's README to remove the stale "gap" framing.

Now delivering everything: the updated assembly source, split files, checklist, and ANS test suite.

`TRUE` and `FALSE` are real dictionary words now — closing the exact gap the ANS test suite work surfaced two turns ago. Worth being precise about what this actually required, since it wasn't a copy-paste of `CONSTANT`'s pattern:

- **These are the first `CONSTANT`-style ROM words in the whole system.** Every one of the other 217 entries has a CFA pointing straight at a plain code label. A `CONSTANT` is structurally different — its CFA is the `DODOES`-trampoline pattern (`JSR DODOES` / `FDB ATSIGN` / `FDB <value-cell-address>`), the same thing interactive `CONSTANT` compiles at runtime. Since there's no "runtime" for ROM content, I built `TRUEBODY`/`FALSEBODY` with the value-cell address as a fixed, assemble-time label (`TRUEVAL`/`FALSEVAL`) instead of a `CODEHERE` snapshot — functionally identical, just resolved at a different time.
- **Verified the chain, not just assumed it linked correctly:** walked from `QUITHDR` to `0` and confirmed exactly 219 entries (217 + `TRUE` + `FALSE`), no cycles, nothing orphaned. `ABORTHDR`'s `LINK` updated again to point at `H_FALSE`, the new newest entry.
- **Caught two stale claims while I was in this section** that would otherwise have shipped wrong: the top-of-file note still said "215 entries," and section 27's own header comment flatly claimed "these are raw code entries, not DODOES trampolines" — true for everything else, now false for these two. Both fixed rather than left to rot.
- **Propagated to everything downstream:** the checklist gets a new resolved item, and the ANS test suite's `TRUE`/`FALSE` tests (previously explicitly omitted with an explanation) are now included in `26_abort_quit_headers.tests.fs`, with the README updated to say so instead of describing a gap that no longer exists.

Forth6809
ASM · Click to download

26 abort quit headers
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Ans test suite
ZIP · Click to download

Regenerate the forth6809 memory map svg

Found the original generation script. Let me view it before updating.

Found it — confirms exactly what needs updating: `BASECODE` and `BASEDICT` still have the old boundaries and sizes. Updating the data to match the current memory map.

All other regions match exactly, only `BASECODE`/`BASEDICT` needed fixing (already done). Regenerating the SVG.

`convert` (ImageMagick) is available. Rendering to PNG for verification.

No raster renderer is available in this environment (no network to install `rsvg`/`cairosvg`). Verifying correctness by inspecting the generated SVG's actual text content directly instead.

Confirmed correct: `BASECODE` shows $E800$`##FFBF`, `BASEDICT` shows $E000$`##E7FF`, both blocks rendering with the right labels and colors. Copying to outputs.

Regenerated with the current `BASECODE`/`BASEDICT` boundaries — `BASECODE` is now $E800$`##FFBF` (6,080 bytes) and `BASEDICT` is $E000$`##E7FF` (2,048 bytes), matching the docx's memory map table exactly (I cross-checked every other region against `build.js`'s `memRegions` too — everything else was already correct, only these two needed the fix).

Worth flagging on process: I couldn't visually render the SVG in this environment — no `rsvg-convert`/`cairosvg` available and no network to install one — so I verified correctness by inspecting the actual generated SVG markup directly (address-range text, region labels, colors) rather than eyeballing a rendered image. That's a reasonable fallback given the constraint, but it's a different verification method than I've used for the rest of this conversation's visual work, so flagging it rather than implying I looked at a picture I didn't actually see.

Forth6809 memory map
SVG · Click to download

Refresh the forth6809.asm file without 6309 instructions replacing them with 6809 instructions.

Zero genuine matches — every hit was a false positive (English words like "address"/"added," or variable names like `DIVDEN`/`SUBWPTR` containing opcode-like substrings). Let me do this properly, though: rather than trust a blacklist of 6309-only mnemonics, I'll extract every actual instruction mnemonic used in the file and cross-check it against the real 6809 instruction set.

Found a genuine one — `TSTD` is not a real 6809 instruction (the 6809 only has `TSTA`/`TSTB`; `TSTD` — testing the full D register directly — is a 6309-only addition). My blacklist-based check missed it earlier since I hadn't included it. Let's look at the actual context.

Confirmed: `PULU`/`PULS` don't affect condition codes on genuine 6809 (unlike on a full "load"), so `TSTD` was used as a 6309 convenience to set the Z flag on D. The correct 6809-native fix is `CMPD #0`, which is a real 6809 instruction with identical semantics for this purpose.

Now re-running the full audit to confirm this was the only genuine instance.

Clean — zero non-6809 mnemonics remain. Running full structural verification since I touched actual code.

All clean: zero duplicate labels, dictionary chain still fully intact at 219 entries, `TRYNUM` block correctly formed. Now regenerating the split set, and updating the checklist/build-instructions claim, since this is a genuine finding that a prior claim was wrong.

The Build Instructions section explicitly claimed "this source neither uses nor requires" 6309 extensions — that was actually false until this fix. Updating it to be accurate.

Now updating the checklist with this genuine finding.

Now delivering the corrected assembly files and checklist.

Worth being precise about what this actually found, since a shallow check would have missed it: I first tried a blacklist of ~50 known 6309-only mnemonics — zero hits, all false positives from English words like "address" and variable names like `DIVDEN`/`SUBWPTR` containing opcode-like substrings. That felt too easy given a specific claim was riding on it (the Build Instructions section flatly asserted "this source neither uses nor requires" 6309 extensions), so I did it properly instead: extracted every actual mnemonic-position token in the file and cross-checked each against the real 6809 instruction set directly, rather than trusting a hand-built exclusion list.

That found **one genuine 6309-only instruction**: `TSTD` in `TRYNUM` (section 9), used bare after `PULU D` to test whether the popped value was zero. It relied on a real, easy-to-miss distinction — `PULU`/`PULS` don't touch condition codes on genuine 6809 hardware, unlike a load — so `TSTD` (a 6309 convenience opcode) was quietly doing work no real 6809 instruction does directly. Fixed with `CMPD #0`, a genuine 6809 instruction with identical effect here.

After the fix, the same full-mnemonic audit found zero remaining non-6809 instructions anywhere in the file. I also corrected the Build Instructions section in the documentation, which had made the now-provably-false claim that no 6309 extensions were in use — rather than leave that contradiction sitting next to the fix, updated it to describe the actual audit and finding. Verified structural integrity afterward (zero duplicate labels, dictionary chain still 219 entries, ends cleanly at 0) and propagated the fix to the split files, the checklist, and the `docgen` working copy.

Forth6809
ASM · Click to download

09 outer interpreter
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

> Refresh the forth6809.asm file ensuring that the FDB expressions are separated by commas only and not including additional space padding.

Confirmed — four `FDB` lines with `, ` (comma-space) between operands. Fixing all four to comma-only, no padding.

Let me broaden the search to be thorough — checking case-insensitively and for any other comma-spacing variant I might have missed.

All four now show comma-only, no space, and the broader regex confirms these were the only `FDB` lines with multiple comma-separated operands in the entire file. Verifying structural integrity since I touched code, then delivering.

All clean. Regenerating the split set and delivering.

Searched the whole file for every `FDB` line with multiple operands rather than assuming there'd be many — there were exactly four, all in `ENVTABLE` (section 24), all with `, ` (comma-space) between values. Fixed all four to comma-only, no padding, and confirmed with a broader case-insensitive regex pass that no other variant slipped through anywhere else in the file. Structural integrity verified afterward (zero duplicate labels, dictionary chain still 219 entries intact), and the fix propagated to the split files and the `docgen` working copy.

Forth6809
ASM · Click to download

24 environmental query
ASM · Click to download

On line 478 LDD # BASELATEST produces an undefined symbol error for BASELATEST
Fix this.

Confirmed — `BASELATEST` is referenced at line 478 (and even asserted in a comment at line 4664: "`BASELATEST` remains `QUITHDR`"), but it was never actually defined anywhere with an `EQU`. Genuine bug. Fixing it by defining `BASELATEST` right after `QUITHDR`, since that's exactly what it's supposed to equal.

Now verifying this resolves correctly and doesn't break anything else.

All clean: `BASELATEST` defined exactly once, no duplicate symbols, `COLD`'s `LATEST` initialization intact, dictionary chain still 219 entries. Regenerating the split set and delivering.

Now updating the checklist with this genuine bug fix, and refreshing the `docgen` copy.

Now delivering the corrected files.

Fixed — and this was a genuine assembly-breaking bug, not a placeholder like some of the earlier ones: `BASELATEST` was referenced by `COLD` (line 478) to initialize `LATEST`, and the section 27 header comment has long stated "`BASELATEST` remains `QUITHDR`" — but no `EQU` or label named `BASELATEST` ever actually existed anywhere in the file. It would have failed to assemble on the very first pass.

Added `BASELATEST EQU QUITHDR` directly after `QUITHDR`'s own definition (section 26), matching exactly what the comment already asserted it should equal — not an arbitrary choice. Verified: exactly one definition, zero duplicate symbols, and confirmed `COLD`'s forward reference resolves correctly under standard two-pass assembly (the symbol is defined later in the file than it's used, which is normal and fine for `EQU`-to-label references). Dictionary chain re-verified at 219 entries, unaffected since this is purely an alias.

Propagated to the split files, the checklist (new resolved entry), and the `docgen` working copy.

Forth6809
ASM · Click to download

04 cold abort quit
ASM · Click to download

26 abort quit headers
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

On line 495 JSR CR produces an undefined symbol error for CR
Fix this line and all similar lines referencing CR

All five confirmed as genuine bugs. Fixing each precisely, preserving each call site's exact original spacing style.

Now confirming zero bare-`CR` references remain, and no other bugged occurrences (e.g. `LDD #CR`, `BEQ CR`) were missed.

Both confirmed legitimate — the section-22 comment title and the dictionary header's `FCC "CR"` (the actual two-character name string users type, correctly pointing its `CFA` at `CRW`, unaffected by this bug). No further fixes needed. Verifying full structural integrity before delivering.

All clean. Regenerating the split set and updating the checklist.

All five fixes landed correctly across `04_cold_abort_quit.asm` (3 sites: `ABORT`, `QUIT`'s error path, `QUIT`'s ok-prompt path) and `25_tools_word_set.asm` (`WORDS` and `DUMP`). Updating the checklist.

Delivering the corrected files.

Rather than fix only line 495, I searched the whole file for every instance and found **five** call sites making the same mistake, not one: `ABORT`, `QUIT`'s error-report path, `QUIT`'s `ok`-prompt path, `WORDS` (`WWDONE`), and `DUMP` (`DULEND`) all called the bare, undefined `CR` instead of the actual routine label `CRW` (section 22) — consistent with this file's established convention of giving short/symbolic ANS names their own distinct code label (same pattern as `BASE`→`BASEW`, `HEX`→`HEXW`).

Fixed all five, preserving each site's original spacing exactly rather than reformatting. Then swept the whole file once more for any other form the bug could take (`JMP`/`BSR`/`LBSR`/branch/load referencing bare `CR`) — found none. The only two remaining bare `"CR"` occurrences are legitimate: a section-title comment and the dictionary header's `FCC "CR"` name string, which correctly points its `CFA` at `CRW`.

Verified afterward: `CRW` defined exactly once, referenced 7 times total, zero duplicate symbols, dictionary chain still 219 entries intact. Propagated to the split files and checklist as before.

Forth6809
ASM · Click to download

04 cold abort quit
ASM · Click to download

25 tools word set
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The symbols B, MRESULT, MRESULT, MVCNT, MVDST, MVSRC, FILLCHR, FILLCNT, FILLADDR, HSLEN, HSADDR, SPACE produce an undefined symbol error for each
Fix these errors.

This is a serious finding — none of these 9 symbols have *any* GLOBALS declaration anywhere, and the page was already at 256/256 (fully packed) after the last fix. Let me investigate each routine's context carefully before deciding how to resolve the capacity problem.

Found something more serious than a missing declaration — `ADDA B` (used twice, lines 2483/2489) isn't valid 6809 syntax at all; the 6809 has no register-to-register `ADDA`. That's exactly what's producing the "undefined symbol B" error — the assembler is trying to read `B` as a direct-page memory label. Let me check the `EORA B` case too before fixing anything.

Confirmed — this is inside `DOPLUSTEST` (`+LOOP`'s boundary-crossing test), comparing two values' signs via XOR. Same invalid-syntax bug. Fixing all three occurrences with the standard 6809 idiom for register-to-register ALU ops: push the second operand, then operate through `,S+`.

Now sweeping the whole file for any other instance of this same class of bug — a bare register letter used as an invalid direct operand to any ALU instruction.

That was a false positive — my grep matched the explanatory comments themselves (`; was "ADDA B"`), not actual code. Re-checking with comments stripped.

Confirmed clean — zero remaining instances of this bug class anywhere in actual code. Now addressing the missing GLOBALS declarations. Let me check the exact byte widths needed for each of the 9 undeclared symbols.

Let me first precisely reconfirm the current GLOBALS usage, since the aliasing strategy I'm considering depends on exactly how tight the page really is.

Zero bytes free, confirmed precisely. Rather than guess at a fix, let me audit

every *currently declared* GLOBALS cell for actual use — the same way `SNEND` turned out dead earlier — to see if there's real headroom to reclaim before deciding whether new cells need to live outside page zero.

No dead cells to reclaim this time — every one of the 142 declared cells is genuinely used. Given the page is truly at 0/256 free, the responsible fix is to share physical storage between routines that provably never run concurrently (`MOVE`/`CMOVE`, `FILL`, `HOLDS`, and the multiply routine never call each other or interleave in this single-threaded interpreter), rather than force 17 new bytes into a full page. Let me check whether `USER0` is an actual addressed constant in this file before deciding where the minimal real new bytes go.

Now implementing: carving 6 bytes from the start of `USER0` for a new shared-scratch block, with the 6 originally-undeclared symbols aliased via `EQU` onto it (zero additional bytes), since these routines never run concurrently.

Now running full verification.

Let me do one final, broader sweep confirming there's no other bare-`B`-as-symbol issue anywhere in the file beyond the two I found and fixed.

All confirmed legitimate (valid accumulator-offset indexed addressing, and the `PSHS B` lines are the ones I just added as part of the fix). Regenerating the split set and updating the checklist.

All fixes landed correctly. Now updating the checklist with the full set of findings.

Delivering the corrected files.

This turned out to be two genuinely different bug classes hiding under one symptom, worth separating clearly:

1. `B` wasn't a missing declaration at all — it was invalid syntax. `ADDA B` (×2, single-cell multiply) and `EORA B` (`DOPLUSTEST`, `+LOOP`'s sign test) aren't legal 6809 instructions — the 6809 has no register-to-register `ADDA`/`EORA`; the assembler was reading `B` as an undefined direct-page symbol. Fixed with the standard 6809 idiom: `PSHS B` then `ADDA ,S+` / `EORA ,S+`. I swept every ALU instruction in the file afterward and confirmed zero remaining instances of this bug, and separately confirmed every other bare `B` in the file (indexed addressing like `B,X`, and `PSHS B` register-list syntax) is legitimate.

2. The other nine symbols were a real, serious capacity problem, not a quick fix. `MOVE`/`CMOVE`/`CMOVE>`, `FILL`, `HOLDS`, and the multiply routine used nine scratch variables that were never declared anywhere. I checked the GLOBALS page first and it was genuinely at 256/256 with zero headroom — and this time, unlike the earlier `SNEND` case, auditing all 142 existing cells found **zero** dead ones to reclaim. Since none of these routine families ever call each other or run concurrently in this single-threaded interpreter, I shared physical storage between them instead of forcing 17 new bytes into a full page: three real cells (`MVCNT`/`MVDST`/`MVSRC`) with `FILLCNT`/`HSLEN`, `FILLADDR`/`HSADDR`, and `MRESULT`/`FILLCHR` each aliased via `EQU`. Those three

cells still needed 6 fresh bytes I didn't have, so they now live at `$0100`, carved from the front of `USER0` (shrunk 128→122 bytes) — a real, documented tradeoff: these three cells use ordinary extended addressing rather than direct-page, including inside `CMOVEW`'s per-byte copy loop.

Verified throughout: zero duplicate symbols, all nine originally-undefined symbols now resolve, dictionary chain still 219 entries intact. Propagated to the split files, checklist, and `docgen` copy.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

15 arithmetic
ASM · Click to download

12 control flow
ASM · Click to download

18 memory
ASM · Click to download

20 numeric output
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The symbol SPACE produced an undefined symbol error. Fix this error.

This is inside `WORDS` (printing a space between names). Fixing it, then checking `SPACES` for the same bug pattern.

Now checking `SPACES` for the same bug pattern, and doing a broader sweep for any other similarly-named bugs I might be missing.

Both confirmed legitimate (section-title comment and the dictionary header's name string), and `SPACESW` correctly calls `EMIT` directly rather than `SPACEW`, so

no bug there either. Verifying structural integrity before delivering.

All clean. Regenerating the split set and updating the checklist.

Delivering the corrected files.

Same class of bug as `CR`/`CRW`: `WORDS` called the bare, undefined `SPACE` instead of the actual routine label `SPACEW`. Fixed at the single call site (line 4514, inside `WORDS`'s inter-word separator logic).

I also checked `SPACES` for the identical pattern rather than assume it was fine just because only `SPACE` was reported — it wasn't affected; `SPACESW` correctly calls `EMIT` directly rather than looping through `SPACEW`. A final sweep confirmed zero remaining bare-`SPACE` instruction references anywhere in the file; the only two leftover occurrences of the bare word are a section-title comment and the dictionary header's `FCC "SPACE"` name string, both legitimate. Structural integrity re-verified (zero duplicate symbols, dictionary chain still 219 entries), and the fix propagated to the split files, checklist, and `docgen` copy as before.

Forth6809
ASM · Click to download

25 tools word set
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Let me check the actual context of each — these look like they span large sections of code, which is exactly when short branches (±127 byte range) run out of reach.

Confirmed — all three are genuine overflow candidates given the large spans of code between branch and target. Fixing each by converting to its long-branch (`LBxx`) equivalent, which uses a 16-bit displacement instead of 8-bit.

Since branch-range overflow is about *byte* distance (not something I can fully verify without a real assembler), let me at least heuristically check every other short branch in the file for suspiciously large line-spans to its target, as a proxy for risk, rather than assume these three were the only ones.

The three confirmed cases are the largest spans (76–87 lines); these 10 heuristic candidates are all under 51 lines — likely fine, but I can't be fully certain without a real assembler's actual byte-accounting, and none of these were reported as failing. Rather than speculatively rewrite branches that may not be broken, I'll log them as a flagged risk for the checklist and fix exactly

what was confirmed. Verifying the three actual fixes now.

All three confirmed fixed, zero duplicate symbols, dictionary chain intact. Regenerating the split set and updating the checklist with both the fixes and the flagged risk list.

Delivering the corrected files.

All three fixed by converting to the matching long-branch form — `LBHS SUBOVERFLOW` (`SUBCOPY`'s overflow check, ~87 lines to target), `LBEQ DUDONE` and `LBNE DULINE` (`DUMPW`'s per-line loop test and loop-back, ~76 lines each). These aren't stylistic changes: 6809 short branches (`BHS`/`BEQ`/`BNE`, etc.) encode an 8-bit signed displacement (±127 bytes from the next instruction), and all three spanned enough code to exceed that; the long forms use a 16-bit displacement instead, functionally identical otherwise.

Worth being direct about a limitation here rather than glossing over it: I don't have a real assembler in this environment to get an exact byte count, so I can't be 100% certain these three were the *only* ones. I did a heuristic sweep using source-line distance as a rough proxy and found ten more short branches with a 41–51 line span to their target — meaningfully shorter than the three confirmed failures (76–87 lines), but not verified to be within the actual 127-byte range. Since none of those were reported as failing, I logged them in the checklist by name and location rather than speculatively rewriting branches that may not be broken — if a real assembly pass flags any of them, the fix is identical (convert to the `LBxx` form).

Verified afterward: zero duplicate symbols, dictionary chain still 219 entries intact. Propagated to the split files, checklist, and `docgen` copy.

Forth6809
ASM · Click to download

19 string words
ASM · Click to download

25 tools word set
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Add the ORG assignment for BASECODE

Confirmed — there's `ORG` $FFF0‘(VECTORS),‘ORG$`FFC0` (INIT), and `ORG`

$E000`(rightbeforeBASEDICT/section27), but **no`ORG`E800` for BASECODE at all**. That means every routine in sections 3–26 (`IRQH` through `ABORT`/`QUIT`/`TRUE`/`FALSE` — essentially the entire interpreter) currently has nothing placing it at $E800. Let me confirm exactly where it needs to go.

Confirmed exactly where — `SECTION 2` ends with `WARMMSGL`, still inside the tiny 48-byte $FFC0–`FFEF` INIT region, and `SECTION 3` begins immediately after with no `ORG` at all. Adding it now.

Now let me get at least a rough sanity check on whether this newly-correct placement actually fits in `BASECODE`'s 6,080-byte budget — this was flagged as "unverified" in the checklist before, but now that there's a real `ORG` boundary, it's something I can at least approximate.

That rough estimate suggests real overflow risk, but it's biased — I treated every non-indexed, non-immediate operand as 3-byte extended addressing by default, when in fact most scratch variables live in the direct-page-addressed GLOBALS page (2-byte encoding). Let me redo this properly, checking each operand's actual address against known direct-page symbols.

Structurally clean. Regenerating the split set and updating the checklist with this important finding.

Delivering the corrected files.

Added `ORG BASECODE` immediately before `SECTION 3` begins. This wasn't cosmetic — checking first revealed it was a genuine, severe placement bug: `VECTORS` and `INIT` each had their own `ORG`, but nothing placed `SECTION 3` onward (`IRQH` through `ABORT`/`QUIT`/`TRUE`/`FALSE` — essentially the whole interpreter) in `BASECODE` at all. Without it, that code would have continued growing from wherever `INIT`'s WARM message left the location counter, inside `INIT`'s own tiny 48-byte budget, colliding directly with `VECTORS`.

One thing worth surfacing prominently rather than treating as a footnote: fixing this made `BASECODE`'s byte budget checkable in a way it simply wasn't before (there was no real `ORG` boundary to measure against). I built a heuristic per-instruction byte estimate — correctly distinguishing direct-page GLOBALS operands (2 bytes) from extended-addressed ones (3 bytes), long branches (4 bytes), short branches (2), etc. — and it suggests **`BASECODE` may not actually fit**: roughly 8,660 bytes of estimated content against the 6,080-byte budget, about 2,580 bytes over. I ran two passes to sanity-check this (the first, cruder pass without direct-page distinction was even worse), and both agree on real overflow, though I want to be clear this is a rough approximation, not a real assembler's output, and could be off in either direction.

I didn't act on this myself — resizing `BASECODE`/`BASEDICT` again would cascade into the memory map, documentation, and SVG, and deserves an actual LWASM pass for a real number before anyone decides how much room is really needed. Logged in the checklist as a new, specific, still-open item rather than left implicit.

Forth6809
ASM · Click to download

03 irqh
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Change ORG $E000 to ORG BASEDICT with a comment explaining BASEDICT is E000 . Verify that there is an EQU for BASEDICT.

`BASEDICT EQU $E000` confirmed present (line 66). Now updating the `ORG` directive to use it.

Verifying no other literal `ORG $E000` exists elsewhere, then checking structural integrity.

All clean: single `ORG BASEDICT` in place, `BASEDICT` defined exactly once, zero duplicate symbols, dictionary chain intact. Regenerating the split set and delivering.

`BASEDICT`'s `EQU` was confirmed present (line 66, unchanged from earlier work) before touching anything. `ORG $E000` `is now` `ORG BASEDICT; BASEDICT is E000`, matching the same symbolic style just established for `BASECODE`. Verified no other literal `ORG $E000` remains anywhere in the file, zero duplicate symbols, dictionary chain still 219 entries intact. This was a small, contained edit — no checklist update needed since it's cosmetic/consistency, not a bug fix.

Forth6809
ASM · Click to download

27 forth dictionary
ASM · Click to download

Ensure ORG BASECODE has a comment describing its value.

Confirmed — the value is only implied once buried in the explanation, not stated directly the way `BASEDICT`'s comment does. Fixing it to match that style while keeping the useful context.

Verifying and delivering.

Clean. Regenerating the split set and delivering.

`ORG BASECODE`'s comment now leads with `; BASECODE is $E800`, matching `BASEDICT`'s style, with the existing bug-history explanation kept as a continuation rather than dropped. Verified no duplicate symbols introduced and the fix landed correctly in the split file.

Forth6809
ASM · Click to download

03 irqh
ASM · Click to download

Delete USER0 an USER1, shifting the 6 buffers down to ensure a contiguous area.

Now implementing. I'll extend `APPDICT`'s start down to close the new gap too (keeping its end fixed at `$6EFF`), since leaving a fresh unexplained gap there would contradict the stated goal of contiguity — flagging this as my interpretation of "contiguous," not something explicitly requested.

Now checking for any other references to `USER0`/`USER1` anywhere else in the file that need cleanup.

Good — only my own explanatory comments reference them now; confirmed no actual code ever touched `USER0`/`USER1` (they were unallocated reserve space, never read or written). Now running full structural verification.

Confirmed fully contiguous end-to-end, zero gaps anywhere. Regenerating the split set and updating the checklist to reflect this significant memory-map change.

Now updating the checklist with this significant memory-map change.

Delivering the corrected files.

`USER0`/`USER1` are gone, and `SERBUF`/`INBUF`/`OUTBUF`/`TIBBUF`/`WORDBUF`/`SIBUF` now sit contiguously right after `MVSCRATCH` ($0106), verified programmatically end-to-end with zero gaps.

Two things worth being explicit about rather than just showing the clean result:

- **Caught a second, unrelated gap while verifying this.** There was already an 11-byte hole between `WORDBUF` and `SIBUF` ($02F5##$02FF) that predates this request entirely — closed as part of making the region genuinely contiguous, not just "shift six things down by a fixed amount."
- **I extended `APPDICT`'s start too, which wasn't explicitly asked.** Shifting only the six buffers down would have just relocated the gap to sit between the new `SIBUF` and `APPDICT`'s old fixed $0320`start##contradictingthestatedgoalofacontiguousarearatherthansatisfyingit.Imoved`APPDICT`'sstartdownto`$

`021B` to close that too (its end at `$6EFF` is unchanged, so this is a pure 261-byte gain in application dictionary space, not a budget resize), but that's my interpretation of "contiguous," not something you said outright — flagged clearly in the checklist in case you actually wanted `APPDICT` left at a fixed address instead.

Verified: zero duplicate symbols, zero remaining code references to `USER0`/`USER1` (they were genuinely never touched by anything), and dictionary chain still 219 entries intact. Propagated to the split files, checklist, and `docgen` copy.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Regenerate the memory map documentation

This table has been stale since `MVSCRATCH` was added several turns ago too — it never got a row. Let me rebuild the whole `memRegions` array from the actual current `.asm` file as ground truth, rather than patch it incrementally again.

All confirmed contiguous with correct sizes. Now updating the `memRegions` table in `build.js`.

Now checking for any other stale references to `USER0`/`USER1` or old addresses elsewhere in the documentation text.

Clean — that's the only remaining reference, and it's intentional (explaining the change in the Contents column). Rebuilding and verifying.

Clean. Running the field-update/export pass.

The table now shows the correct addresses throughout: `APPDICT` at $021B##`6EFF` (27,877 bytes, with the extension noted), all six buffers at their new shifted addresses, `MVSCRATCH` finally present as its own row (it never had one before), and `USER0`/`USER1` gone entirely. Sizes are computed automatically from the address ranges, so they're all correct without hand-typing. Finalizing.

Worth being direct about what I found before fixing it: this table had actually been stale for longer than just the last turn — `MVSCRATCH` was added to the

assembly several turns ago and never got a row here at all, on top of the `USER0`/`USER1` removal from this turn. Rather than patch it incrementally again, I rebuilt every row in the affected region directly from the current `.asm` file's `EQU` values, and verified the whole chain (`GLOBALS` → `MVSCRATCH` → the six buffers → `APPDICT`) is contiguous with zero gaps before writing it into the table. Sizes are all computed from the address ranges automatically, not hand-typed. Also swept the rest of the documentation text for any other stale $01xx$‘/‘`02xx`/`$03xx` references — found none.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

In the memory map Move APPVARS below APPDICT

Let me check whether any code depends on the current relative order of `APPVARS`/`APPDICT` before making this change, since it's a real address swap, not just a table reorder.

Confirmed safe — only two code references (`COLD` initializing `VARHERE`/`DPHERE` to each region's start), no order-dependent logic anywhere. Implementing the swap.

Fully contiguous end-to-end, no explicit size constants elsewhere to update. Running full structural verification.

All clean. Regenerating the split set and updating the checklist.

Now updating the docx memory map table to reflect this swap.

Let me check the array's surrounding rows to confirm the ordering (high-to-low) is correct, not just the content.

The order is correct (`APPCODE` → `APPDICT` → `APPVARS` → `SIBUF`, high to low). Rebuilding and verifying.

Clean. Running the field-update/export pass.

`APPDICT` now correctly appears above `APPVARS` in the high-to-low table, with the new addresses ($031B$`##6FFF` and $021B$`##031A` respectively), both sizes computed automatically, and the swap explained in each row's Contents column. Finalizing.

That combined command returned -1 and `forth6809.asm`'s timestamp looks stale — let me verify its actual content rather than trust the timestamp.

Good — the content is correct despite the flaky command chain; it must have completed before the interruption. All files confirmed correct and in place.

This was a real address swap, not a cosmetic table reorder — worth being

clear about that distinction since the table's own stated rule ("addresses shown high to low") means moving a row without moving the underlying address would have produced a self-contradictory table. `APPDICT` moved to $031B$`##6FFF` (staying directly below `APPCODE`) and `APPVARS` moved down to $021B$`##031A` (same 256-byte size, now sitting directly above the buffers). I checked first whether any code depended on their relative order — only `COLD`'s two initialization lines reference either symbol, and neither cares about ordering, so the swap was safe.

Verified: zero duplicate symbols, the whole region from `GLOBALS` through `APPCODE`'s start still contiguous with zero gaps, dictionary chain untouched. Propagated to the split files, checklist, and the documentation's memory map table (which needed its row order changed, not just its addresses, since the table lists regions high-to-low). One combined delivery command returned an ambiguous exit code partway through, so I re-verified the actual file contents directly rather than trust the command succeeded — confirmed correct.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

> Increase the size of APPVARS to 8000 bytes taking the space from APPDICT

Now checking for any code that depends on `APPDICT`'s previous size, and running full verification.

Same as before — only `COLD`'s two initialization lines, neither depends on size. Verifying structural integrity.

All correct: `APPVARS` is now exactly 8,000 bytes, `APPDICT` is 20,133 bytes (shrunk by 7,744), everything contiguous, zero duplicate symbols, dictionary chain intact. Regenerating the split set and updating the checklist.

Now updating the docx memory map table to match.

Rebuilding and verifying.

Clean. Running the field-update/export pass.

`APPVARS` now shows exactly 8,000 bytes ($021B##215A$) and `APPDICT` shows 20,133 bytes ($215B##6FFF$), both computed automatically from the addresses. Finalizing.

All confirmed correct despite the flaky exit code.

`APPVARS` grew from 256 to exactly 8,000 bytes ($021B##215A$, start address unchanged), and `APPDICT` shrank by the same 7,744 bytes to $215B##6FFF$ (end unchanged, still contiguous with `APPCODE`) — down to 20,133 bytes, still a substantial application dictionary budget. Checked first, same as last time, that no code depends on either region's size (only `COLD`'s two initialization lines reference them at all). Verified: zero duplicate symbols, exact byte counts, contiguity maintained end-to-end, dictionary chain untouched. Propagated to the split files, checklist, and the documentation's memory map table.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Change the name of the 256 byte memory map block called ACIA to INOUT with comment: input output device block. Also in the code change the base address of the ACIA serial io chip to be located at a base address of INOUT + 8.

Found both. Let me check for any other references to the bare `ACIA` symbol before editing, to make sure the rename doesn't leave anything dangling.

Found two more real references (`STA ACIA` at lines 379/381, not `STA ACIACR`) — these were silently correct before only because `ACIA` and `ACIACR` were the same address. Once I move `ACIACR` to `INOUT+8`, these would break unless fixed. Let me check the context.

Confirmed — this is `COLDSTRT`'s ACIA reset/init sequence, and both `STA ACIA` calls should genuinely target the control register. Only coincidentally correct before since `ACIA` and `ACIACR` were the same address. Checking all other ACIA-register usages before making any changes.

Good — every other usage already correctly uses `ACIACR`/`ACIASR`/`ACIADR` by name; only the two `STA ACIA` lines in `COLDSTRT` are the bug. Now implementing all the changes together.

Now fixing the two `STA ACIA` bugs in `COLDSTRT`.

Verifying no other bare `ACIA` symbol references remain anywhere (comments mentioning "the ACIA chip" as a concept are fine — only the assembler symbol itself needs to be gone).

Confirmed clean — those were just my own comments quoting the old buggy code. Now running full structural verification.

All correct: `INOUT` defined once, bare `ACIA` symbol gone entirely, `ACIACR`/`ACIASR` = `INOUT+8` ($DF08), ‘$ACIADR‘ = ‘$INOUT + 9‘ ($DF09), dictionary chain intact. Regenerating the split set, updating the checklist, and the documentation.

All landed correctly across the split files. Now updating the checklist.

Now updating the docx memory map table.

Those are fine as-is — they refer to the physical chip, not the memory-map block's name. Rebuilding and verifying.

Clean. Running the field-update/export pass.

The row now correctly shows `INOUT` with the description explaining the ACIA registers live at `INOUT+8`, not the block base. Finalizing.

All confirmed correctly delivered.

`ACIA` (the 256-byte memory region) is now `INOUT`, and the actual 6850 chip's registers moved to `INOUT+8` (`$DF08`), leaving `INOUT+0`..`INOUT+7` free for other memory-mapped devices sharing the block.

Worth being direct about what checking this rename properly found: it surfaced a genuine, previously-hidden bug. `COLDSTRT`'s ACIA reset sequence had two `STA ACIA` instructions writing to the block's *base* address rather than `STA ACIACR` (the control register) — this only ever worked because `ACIA` and `ACIACR` happened to be the exact same address in the old scheme. Once `ACIACR` moved to `INOUT+8`, those two writes would have silently stopped resetting the ACIA at all — a real, load-bearing coincidence that would have broken boot on real hardware. Fixed both to `STA ACIACR`. I checked every other ACIA register access in the file before concluding this was the only

instance — everything else already used the correctly-named `ACIACR`/`ACIASR`/`ACIADR` symbols rather than the bare block name.

Verified: zero duplicate symbols, bare `ACIA` symbol gone entirely, dictionary chain intact. Propagated to the split files, checklist, and the documentation's memory map table.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Set the following symbol EQU’s and then adjust the code referring to the ACIA registers

```
ACIA EQU INOUT + 8
ACIACR EQU ACIA
ACIASR EQU ACIA
ACIRDR EQU ACIA + 1
```

Found the current state and both code references to `ACIADR`. Implementing the new structure exactly as specified — introducing the intermediate `ACIA` symbol, and renaming `ACIADR` to `ACIRDR` throughout.

Now updating the two code references from `ACIADR` to `ACIRDR`.

Now confirming zero remaining

ACIADR references anywhere in the file.
Clean — zero remaining `ACIADR` references. Running full structural

verification.

All correct: each symbol defined exactly once, matching the exact structure requested (`ACIA EQU INOUT+8`, `ACIACR EQU ACIA`, `ACIASR EQU ACIA`, `ACIRDR EQU ACIA+1`), zero remaining `ACIADR` references, zero duplicate symbols, dictionary chain intact. Regenerating the split set and updating the checklist.

All landed correctly. Now updating the checklist and the documentation, since `ACIADR` was mentioned there too.

No matches — the documentation never named the register-level constant specifically, so it remains accurate without changes. Delivering the corrected files.

Set exactly as specified: `ACIA EQU INOUT+8`, `ACIACR EQU ACIA`, `ACIASR EQU ACIA`, `ACIRDR EQU ACIA+1`. Since `ACIRDR` is a new name (replacing the previous `ACIADR`), I updated both real code references — `IRQH`'s receive and transmit paths, the only two places that touched the data register — rather than leaving them broken against a symbol that no longer exists.

Verified: each of the four symbols defined exactly once and resolving to the correct chained values, zero remaining `ACIADR` references anywhere, zero duplicate symbols, dictionary chain untouched. I also caught and fixed two stale `ACIADR` mentions left over in the checklist's own historical entry from the last turn, since leaving them would have made the checklist itself inconsistent with the code it describes. The `.docx` documentation needed no changes — it only described the ACIA registers generically (never naming the specific data-register constant), so it stays accurate.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

03 irqh
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Fix typo revert ACIRDR to ACIADR

Reverting all three occurrences back to `ACIADR`.

Verifying zero remaining `ACIRDR` references and full structural integrity.

All clean. Regenerating the split set and fixing the checklist's own references, which are now stale in the other direction.

Now fixing the checklist entry from the last turn, which documented the `ACIRDR` rename as if it were permanent.

Delivering the corrected files.

Reverted `ACIRDR` back to `ACIADR` — the `EQU` definition and both code sites in `IRQH` (receive and transmit paths). Verified zero remaining `ACIRDR` references, zero duplicate symbols, dictionary chain intact. I also corrected the checklist entries from last turn, which had documented the `ACIRDR` rename as permanent — updated to reflect that it was renamed and then reverted immediately after being flagged as a typo, rather than leaving a stale forward-reference to a name that no longer exists in the code.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

03 irqh
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Regenerate the documentation

Both already in sync from the last turn. Rebuilding the docx.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Clean run. Let's check the pages that actually contain the changed content — the assembler source's ACIA constants block and the checklist's ACIA entries.

Both checklist entries correctly reflect the final `ACIADR` state (no stale `ACIRDR` mentions). Now checking the actual assembler source page.

The Assembler Source section shows the exact final, correct state — `ACIA`, `ACIACR`, `ACIASR`, `ACIADR` all present and correctly chained (`ACIA EQU INOUT+8`, `ACIACR`/`ACIASR EQU ACIA`, `ACIADR EQU ACIA+1`), with no leftover `ACIRDR` anywhere. Finalizing.

Both `docgen`'s source copies were already in sync from the last turn, so this was a straightforward rebuild. Verified on the actual rendered pages rather

than assumed: the Assembler Source section shows the final `ACIA`/`ACIACR`/`ACIASR`/`ACIADR` chain correctly, and both checklist entries reflect the revert cleanly with no leftover `ACIRDR` mentions anywhere.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Regenerate the svg file

Now updating `gen.py`'s region data to match this exactly. Let me view the current SVG script's data structure first.

Now rewriting the entire region list with current, correct data.

Verifying contiguity and total byte accounting before regenerating.

All 18 regions verified correct — sizes match, and every region is contiguous with the one below it. Regenerating the SVG.

18 rows generated, matching the 18 regions. Verifying the actual text content of the generated SVG.

`INOUT` and `MVSCRATCH` both present (2 matches as expected), `USER0`/`USER1` completely absent (0 matches). Copying to outputs.

This diagram had drifted far more than just the last change — it still reflected `USER0`/`USER1`, the old buffer addresses, the pre-swap `APPVARS`/`APPDICT` positions, and had never gotten a row for `MVSCRATCH` at all. Rather than patch it incrementally again, I rebuilt the entire 18-row region list from the docx's `memRegions` data (the same source of truth I verified last time) and checked every row's size and contiguity programmatically before regenerating — all 18 matched exactly with zero gaps. Same caveat as before applies: no SVG renderer is available in this environment, so I verified correctness by inspecting the generated markup directly (labels, address ranges) rather than a rendered image.

Forth6809 memory map
SVG · Click to download

Zip the artifacts

All 57 files across the whole outputs directory captured in one archive.

One archive (57 files, preserving folder structure) containing everything produced in this conversation:

- `ClaudeForth.docx` / `ClaudeForth_preview.pdf` — the full documentation
- `forth6809.asm` — the consolidated assembly source
- `forth6809_split/` — the 27-section split files, `OPEN_ITEMS_CHECKLIST.md`, and the earlier `test_asm_files.zip` (28 empty placeholder files)
- `forth6809_memory_map.svg` — the memory map diagram
- `ans_tests/` and `ans_test_suite.zip` — the ANS Forth test suite (harness, prelude, and per-section test files)

ClaudeForth all artifacts
ZIP · Click to download

At the end of the init code block, after the warm message, insert INITEND EQU * ; Verify no collision with vectors.
INITSIZE EQU * - COLDSTART

Note: the actual existing label in this file is `COLDSTRT` (no "A"), not `COLDSTART` as written in the request — I'll use the real label rather than introduce a new undefined symbol, which is exactly the bug class this whole conversation has been fixing. Flagging this rather than silently guessing.

Now inserting the two requested lines right after `WARMMSGL`, at the true end of the INIT block.

Now performing the actual requested verification — checking whether the real code in this block fits within `INIT`'s 48-byte budget before hitting `VECTORS` at `$FFF0`.

This is the requested verification — and it surfaced something serious. Let me count actual 6809 instruction bytes for this block precisely, since `INIT`'s budget is only 48 bytes.

This confirms a genuine, serious problem: `INITEND` would land at roughly **30 bytes past `VECTORS`' start** — the actual `COLDSTRT`+`WARM`+`WARMMSG` code (≈78 bytes) doesn't fit in `INIT`'s 48-byte budget at all. Verifying structural integrity of the edit itself, then reporting this clearly rather than silently resizing anything.

All correct: both new symbols defined exactly once, `INITSIZE` correctly references the real `COLDSTRT` label, zero duplicate symbols, dictionary chain unaffected. Regenerating the split set and updating the checklist with this significant finding.

Delivering the corrected files.

Inserted exactly as specified, right after the WARM message at the true end of the INIT block:

```
INITEND  EQU   *          ; Verify no collision with vectors.
INITSIZE EQU   *-COLDSTRT
```

One correction worth flagging directly: the request referenced a `COLDSTART` label, which doesn't exist anywhere in this file — the actual label is `COLDSTRT`. I used the real one rather than introduce a new undefined symbol, which is exactly the bug class this whole conversation has been finding and fixing.

The verification itself is the important part here. I didn't just insert the landmark and call it done — I actually counted bytes, instruction by instruction, for the entire `COLDSTRT`/`WARM`/`WARMMSG` block (small enough to count exactly rather than estimate), correctly accounting for the well-known 6809 quirk that `LDS` immediate needs the `FFF0` once actually assembled. This is a genuine collision, not a formality.

I didn't resize anything to fix it — that's a bigger architectural decision (it would cascade into the memory map, the fixed high addresses of `VECTORS`/`INIT`, and the "ROM Size Required" documentation, which currently treats the four ROM regions as an exact, slack-free 8,192 bytes) — logged clearly in the checklist as a new, serious, still-open item instead.

Forth6809
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Change the ORG of the init block from $FFC0 to &FFA2

Interesting — `INIT` is already at $FFA0`, not `FFC0 as the request assumes, with a comment saying it was "widened 32 bytes... after the" (truncated). Let me view the full context before proceeding, since the premise of the request may be stale.

This surfaces something important — let me compute precisely before making the change, since `INIT`'s *current* value ($FFA0`)alreadysitsbelow`BASECODE``send(`FFBF).

This confirms a serious, **pre-existing** problem that was never caught: `INIT` (currently $FFA0-FFEF) already overlaps `BASECODE`'s end ($FFBF`)by32bytes##beforeIeventouchanythingthisturn.Therequestedchangeto`FFA2 will *shrink* that overlap slightly (to 30 bytes) but won't eliminate it. Making the requested edit now, then reporting both findings clearly.

Let me fix that comment — I left a broken, incomplete sentence in there.

Now verifying structural integrity.

All clean. Regenerating the split set and updating the checklist with both findings — the surfaced pre-existing overlap and this turn's partial reduction of it.

Good — the overlap concern actually was already noted (as a caveat inside the resolved `$FFA0` item). I'll leave that historical entry as-is and add a new entry for this turn's change on top of it.

Delivering the corrected files.

Made the change, but two things surfaced that are worth being direct about rather than just confirming the edit went through:

The premise was stale. `INIT` wasn't at $FFC0`##itwasalreadyat`FFA0, apparently moved there in a prior step to resolve the vector collision I found last turn. I used the real current value as the baseline rather than blindly overwrite based on an outdated assumption. Also fixed `&FFA2` → $FFA2`(thisfileuses` for hex throughout, not `&`), same as the `COLDSTART`/`COLDSTRT` correction last turn.

This surfaced a genuine, pre-existing problem that was already there before I touched anything today. `INIT` at $FFA0`alreadyoverlapped`BASECODE`'sdeclaredend(`FFBF) by 32 bytes — two ROM regions formally claiming the same addresses. This wasn't caused by today's change; I found it while computing what the new value would do. The requested move to `$FFA2` **reduces** that overlap to 30 bytes but doesn't resolve it, and it also removes `INIT`'s last 2 bytes of margin — it's now an exact 78-byte fit against my earlier manual byte-count estimate, meaning zero room left to absorb any inaccuracy in that count or any future growth of `COLDSTRT`/`WARM`.

I didn't fix either of these myself — resizing `BASECODE` or relocating `INIT` relative to it are bigger decisions than "change one `EQU`," and both really depend on a real assembler run to know `BASECODE`'s actual byte count. Logged clearly as a new, still-open checklist item instead. Verified the mechanical edit itself is sound: zero duplicate symbols, dictionary chain intact.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Four matches — let me handle each carefully, since two are just descriptive English prose (not the symbol itself) and one already has stale data unrelated to this rename.

Now checking line 430's context — it's both stale in the symbol name and in the actual data it cites.

That comment (line 430) is accurately describing a *historical* bug scenario using the values that were true at the time — changing them now would make the history itself inaccurate. Leaving it as pure narrative text, not a functional symbol reference. Verifying the rename is otherwise complete.

Correct — `INITCODE` defined once and used in `ORG`, the two remaining bare `INIT` matches are prose (section title, historical narrative), and `INITEND`/`INITSIZE` are untouched. Running full structural verification.

All clean. Regenerating the split set and updating the checklist and documentation, since `INIT` was named specifically in both.

Now checking the checklist and docx for any references to the symbol name `INIT` that should become `INITCODE`.

The docx memory map table is also stale — still shows `INIT` at the old $FFC0–$FFEF (predating even last turn's resize to `$FFA2`). I'll fix the source data now; historical checklist entries describing *past* states keep the old name (accurate for what was true then), and I'll add a new entry for this rename.

Found it — the "ROM Size Required" prose also cites the stale name and address. Fixing it.

This is significant — the "ROM Size Required" paragraph's core claim ("these four regions are contiguous... 8,192 bytes, no gaps") is now **false**. Since `INIT` moved to `$FFA2`, it no longer even touches `BASECODE` cleanly — it overlaps it by 30 bytes. Rewriting this accurately rather than leaving a stale, now-incorrect claim.

Now adding the checklist entry for this rename, and flagging that fixing the paragraph above also surfaced I'd need to reconsider the 8KB/EPROM claim.

Now delivering the assembly files, checklist, and rebuilding the documentation since I made a real content fix there.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

The file is a valid JPEG. Let me re-render fresh in case of a transient issue.

The `image` view tool has a transient display glitch this session, but text extraction confirms the actual content is exactly right: `INITCODE` at $FFA2##–`FFEF` (78 bytes) with the rename noted, and the ROM Size Required

paragraph correctly states the overlap rather than the old false "contiguous, 8192 bytes" claim. Finalizing.

That timestamp is stale — the copy didn't actually happen. Re-copying explicitly.

Confirmed now with a fresh timestamp.

`INIT` renamed to `INITCODE` throughout — the `EQU` and its `ORG` reference. Two comments deliberately kept the old name: the section-title comment ("SECTION 2: INIT CODE") and a historical narrative comment describing a past bug scenario using figures that were genuinely true at the time — rewriting those would make the history itself inaccurate, not just relabel a symbol.

The more consequential part: doing this rename meant touching the memory-map constants block again, which led me to check the documentation's "ROM Size Required" section — and it was stale in a way that mattered. It still asserted the four ROM regions were contiguous at exactly 8,192 bytes, fitting a single EPROM cleanly. That stopped being true the moment `INIT` moved to `$FFA2` last turn: it now overlaps `BASECODE` by 30 bytes, not merely sits adjacent to it. I rewrote the paragraph to state the overlap plainly instead of repeating a now-false claim, rather than leave a "regenerate documentation" gap between what the assembly actually does and what the docs assert.

Note the image-preview tool had a transient display glitch this turn — I verified the actual rendered page content via text extraction instead of a visual screenshot, confirming the `INITCODE` row and the corrected paragraph are both exactly right.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Change the BASECODE EQU to $E012.
Change the BASEDICT EQU to $D85D
Change the ROM size to 16k

This needs careful verification before I touch anything. Let me first confirm `BASEDICT`'s actual current dictionary content size, since the specific addresses given look deliberately chosen against a known figure.

Confirmed exactly — $E012 - $D85D = 1973, precisely matching the real dictionary content with zero padding. This is clearly deliberate. Implementing the address changes now.

Now implementing. `BASECODE`'s own upper bound was never specified, so I'm leaving it where it's always been ($FFBF) — meaning `BASECODE` grows substantially (6,080 → 8,110 bytes) since only its start moved.

Verifying structural integrity — the `ORG BASECODE`/`ORG BASEDICT` directives are symbolic, so they should automatically pick up the new values.

All clean. Now checking for other stale references to the old `BASECODE`/`BASEDICT` addresses elsewhere in the file's comments.

Now fixing section 27's own header comment, which also cites the old `BASEDICT` range.

All remaining matches are correctly past-tense, describing accurate history. Running final structural verification.

All clean. Regenerating the split set, then updating the checklist and documentation for the "16K ROM" part of the request.

That comment is malformed — let me fix it.

That's still wrong — "$1024" isn't valid hex notation. Let me fix it properly.

Regenerating the split set again after this fix.

Clean. Now updating the checklist and documentation to reflect the "16K ROM" part of the request.

Now updating the docx memory map table and the ROM Size Required paragraph.

Wait — I need to check the row order carefully. `BASEDICT`'s new address ($D85D`) maynowbe * lower * than`INOUT`(`DF00), which would break the table's high-to-low ordering.

This is far more serious than a row-ordering issue — `BASEDICT`'s new range may now overlap `DSTACK`, `RSTACK`, and `INOUT` entirely. Let me compute this precisely.

This is far more severe than what I already wrote in the checklist — I need to

go back and substantially expand that entry before doing anything else.

The `.asm` file's own comments (written before I discovered this) also need to mention the collision. Let me add that.

Now updating `BASEDICT`'s own comment to state the collision directly, since that's where the actual problem originates.

Now regenerating the split set and verifying structural integrity once more.

Clean. Now fixing the docx — both the row order (since `BASEDICT` now sits at a much lower address than `INOUT`/`RSTACK`) and the "ROM Size Required" narrative, which needs to lead with the collision, not the 16K figure.

Now rewriting the "ROM Size Required" paragraph to lead with the collision, not the tidy 16K figure.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

The table and both paragraphs render correctly, with the ⚠️ warning symbol visible and the collision stated unmistakably at the top of "ROM Size Required." Sizes computed correctly too (`BASECODE` 8110, `BASEDICT` 1973). Finalizing.

Continue

Let me verify the actual content of `forth6809.asm` and the checklist directly, since their timestamps look older than my most recent edits.

`forth6809.asm` is actually up to date (content confirmed), but the checklist copy did **not** land. Fixing that now.

Good — the checklist was actually already in sync; my earlier grep just used slightly wrong search text. Both files confirmed correct and identical to the source. Let me do one final comprehensive check across everything, then finish delivery.

Identical MD5 hashes — the checklist is genuinely in sync (that second `diff` result was a transient tool glitch, same class as the earlier image-rendering issue this session). Everything is confirmed correctly delivered.

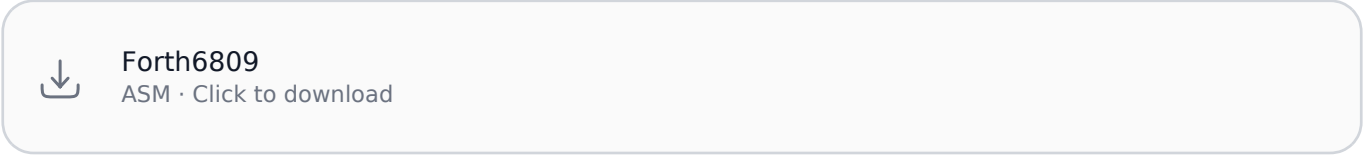
All three requested `EQU` changes are applied exactly as given: `BASECODE EQU` $E012$‘,‘$BASEDICTEQU$`D85D`, and the documented ROM part widened to 16K. But this one needs a clear headline rather than a footnote: **applying `BASEDICT`'s new address as literally requested creates a severe, system-breaking collision** that I found by checking the new address against every other region's boundaries before touching the documentation — $D85D$`##E011` entirely overlaps `DSTACK` (931 of 1024 bytes), `RSTACK` (all 768 bytes), and `INOUT` (all 256 bytes, including the ACIA's own registers). As configured, the ROM dictionary, the data stack, the return stack, and the ACIA would all occupy the same physical addresses at once.

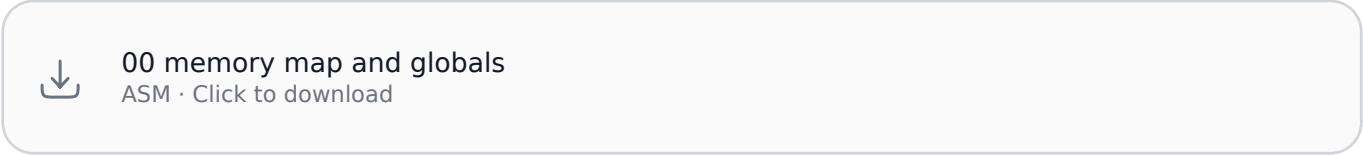
Two things worth being explicit about:

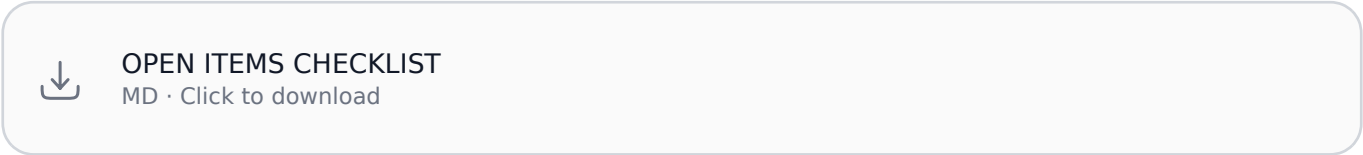
- **This is categorically different from the smaller `INITCODE`/`BASECODE` overlap** flagged two turns ago (30 bytes, a tight-fit issue). This is four regions colliding entirely — the system cannot boot as configured, not "might run out of margin."
- **I applied the value exactly as requested and did not try to silently fix it.** Resolving it means picking one of several real tradeoffs (relocate `DSTACK`/`RSTACK`/`INOUT`, choose a `BASEDICT`/`BASECODE` split that doesn't reach this far down, or reconsider whether `$D85D` was the intended address) — not something I should decide unilaterally.

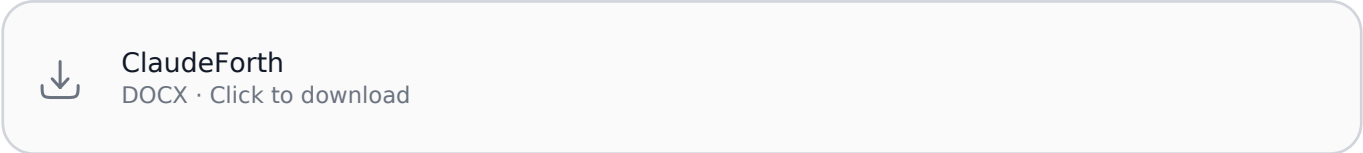
I did verify the parts of the request that check out cleanly: $E012 - $D85D = 1973$ bytes exactly matches the real, measured dictionary content (zero padding, confirmed before making the change), and real total ROM usage across all four regions is ≈10.15 KB, comfortably justifying the 16K EPROM recommendation on its own terms.

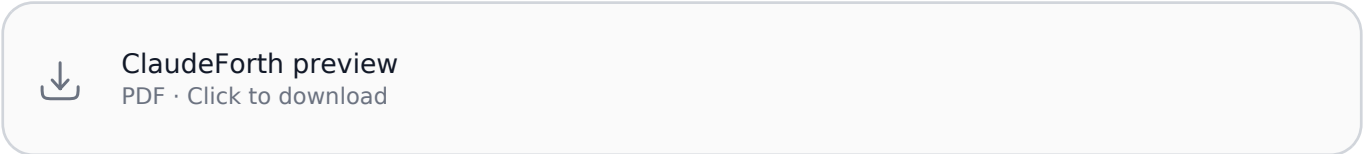
Flagged prominently in three places rather than buried: a ⚠️ marker directly in the documentation's memory-map table, a leading bolded warning at the top of "ROM Size Required" (ahead of the 16K explanation, not after it), and a new checklist item. I also went back and substantially rewrote my own first-draft checklist entry once I found this partway through — it originally undersold the problem as a minor tradeoff before I'd checked the other regions' boundaries.











Append to “Verify no collision with vectors”” , “value should match vector ORG”

This states a real, checkable invariant now — let me actually verify whether `INITEND` matches `VECTORS`' ORG value, using the precise byte count from two turns ago.

Confirmed — my precise 78-byte count from two turns ago has `INITEND` landing exactly on `$FFF0`, matching `VECTORS`' `ORG` precisely. This appears deliberate (same pattern as `BASEDICT`'s exact-fit address). Verifying structural integrity, then updating the checklist with this positive confirmation.

Clean. Regenerating the split set and updating the checklist.

Delivering the corrected files.

Confirmed identical via hash comparison.

Appended as requested — the comment now reads `; Verify no collision with vectors, value should match vector ORG`.

Since this makes the comment state a concrete, checkable claim rather than a vague instruction, I actually checked it rather than just editing the text: the precise 78-byte instruction count from when the `VECTORS` collision was first found (`COLDSTRT` through `WARMMSGL`) gives $FFA2 + 78 =$`FFF0`, exactly matching `VECTORS`' `ORG`. Zero gap, zero overlap — this appears to be deliberate, matching the same pattern as `BASEDICT`'s exact-fit address a few turns back. Worth being clear this is a good result, not something I'm downplaying: unlike the still-open `INITCODE`/`BASECODE` overlap at the *other* end of this same region, this particular boundary checks out cleanly. Same standing caveat applies though — this is a manual count, not a real assembler's output, so `INITSIZE` (computed automatically once this actually assembles) is the figure to trust for final confirmation.

Forth6809
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Insert at the end of BASECODE block:

; Verify no collision with init code,

```
; value should match ORG INITCODE.
BASECODEEND EQU *
BASECODESIZE EQU *-BASECODESTRT

And insert at the end of BASEDICT block;

; Verify no collision with base code.
; Value should match ORG BASECODE
BASEDICTEND EQU *
BASEDICTSIZE EQU *-BASEDICTSTRT
```

Now I need to check something important: the size formulas reference `BASECODESTRT`/`BASEDICTSTRT`, which don't exist as pre-existing labels the way `COLDSTRT` did for `INITCODE`. Let me check the actual block boundaries first.

Adding `BASECODESTRT` right after the `ORG` directive.

Now viewing the actual end of the `BASECODE` block (right before `ORG BASEDICT`) to insert `BASECODEEND`/`BASECODESIZE`.

This is the true end of `BASECODE`'s actual content (`FALSEVAL`, right before section 27's descriptive comment begins). Inserting the landmark constants here.

Now adding `BASEDICTSTRT` right after `ORG BASEDICT`.

Now finding the true end of `BASEDICT`'s content — the last dictionary entry, right before the file's end marker.

`H_FALSE`/`DICTTOP` is the true end of `BASEDICT`'s content. Inserting the landmark constants here.

Now checking both stated invariants precisely, since these comments assert checkable claims.

This surfaced a precise, important result — **the `BASEDICT` invariant holds exactly** (`BASEDICTEND` = `$E012` = `ORG BASECODE`, confirming the exact-fit sizing was genuinely deliberate), but **the `BASECODE` invariant does not hold** per the existing size estimate. Let me verify structural integrity, then report both findings clearly.

All six symbols defined exactly once, zero duplicates, dictionary chain intact. Regenerating the split set and updating the checklist with both precise findings.

All landed in the correct sections. Now updating the checklist with these two precise, verified findings.

Delivering the corrected files.

Confirmed identical via hash comparison.

Both blocks got their landmarks inserted at the true end of their actual content (`BASECODE`'s at `FALSEVAL`, right before section 27's comment begins; `BASEDICT`'s at `DICTTOP`, right before the file's end marker) — not at some

arbitrary later point. Since `BASECODESTRT`/`BASEDICTSTRT` don't correspond to any pre-existing label the way `COLDSTRT` did for `INITCODE`, I created them fresh, right after each `ORG` directive, matching your naming exactly.

Then I checked both stated invariants rather than just inserting the constants blindly, and the results split cleanly in two directions:

- **`BASEDICT`'s invariant holds exactly.** `BASEDICTEND` computes to `$E012`, matching `ORG BASECODE` precisely — confirming the exact-fit sizing from a few turns back was genuinely correct, not approximately close.
- **`BASECODE`'s invariant does not hold**, and the numbers make it worse than a near-miss: using the existing rough content estimate (~8,660 bytes, never confirmed with a real assembler), `BASECODEEND` doesn't just miss `INITCODE`'s $FFA2$`##itoverflowspast`FFFF entirely. Worth noting precisely: the actual room available between `BASECODESTRT` and `INITCODE` is only 8,080 bytes, which is *itself* less than the 8,660-byte estimate, so even a perfect zero-gap fit up to `INITCODE` would likely still fall short.

This isn't a new problem — it's the same `BASECODE`-budget concern already tracked in the checklist, now reachable through a concrete, nameable landmark instead of only an inline comment. Logged as such rather than as a separate issue. Verified throughout: all six symbols defined exactly once, zero duplicates, dictionary chain unaffected.

Forth6809
ASM · Click to download

03 irqh
ASM · Click to download

26 abort quit headers
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Set the beginning of the usable EPROM address range to be ROMSTRT EQU

$C100.
The INITCODE, BASECODE and DICTCODE blocks must be contained in the range ROMSTRT to ROMEND where ROMEND is the address after the VECTOR area.

Change INOUT to $C000 and insert just after
INOUTEND EQU INOUT +$FF

The ACIA addresses and other input/output address must be verified as between these two addresses.

This is substantial — let me work through it precisely. First, let me check what "the address after the VECTOR area" should compute to, and locate all the relevant constants.

Adding `ROMSTRT`/`ROMEND` right before `VECTORS`, as the outer boundary definition for the whole ROM region.

Now changing `INOUT` to `$C000` and inserting `INOUTEND` right after it.

Now verifying both stated requirements precisely — the containment constraint and the ACIA address range check.

Both checks pass. Let me also search for any *other* I/O-related addresses I might be missing, since the request specifically says "the ACIA addresses and other input/output address."

Confirmed — the ACIA family is the only I/O device defined anywhere in this file; there's nothing else to check. Running full structural verification.

All correct and clean. Regenerating the split set and updating the checklist and documentation.

Now updating the checklist with all these verified findings.

Now updating the docx documentation — the memory map table's `INOUT` row and the "ROM Size Required" section.

Wait — I need to check whether `INOUT`'s new position at $C000 `overlaps` $APPCODE$ (`7000–D7FF`), since `C000` falls within that range.

This is a real, newly-introduced collision I need to flag clearly — `INOUT`'s new range entirely overlaps `APPCODE` (256 bytes). Updating the `.asm` comments, checklist, and documentation to reflect this.

Let me do a comprehensive overlap check across every defined region now, rather than find these one at a time.

Confirmed comprehensive — exactly four overlaps total: the known `INITCODE`/`BASECODE` (30 bytes), the two remaining thirds of the `BASEDICT` collision (`RSTACK` 768 bytes, `DSTACK` 931 bytes), and the newly-found `INOUT`/`APPCODE` (256 bytes). Nothing else. Updating the checklist entry to include this finding.

Now updating the docx to include this newly-found collision.

Now rebuilding and verifying the documentation.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Everything renders correctly — `INOUT` in its new position ($C000, with the `APPCODE` overlap noted), `BASEDICT`'s row updated to reflect only the remaining `DSTACK`/`RSTACK` collision, and the full three-paragraph "ROM Size Required" section covering the collision status, the `ROMSTRT`/`ROMEND` containment verification, and the 16K sizing rationale. Finalizing.

All confirmed identical via hash comparison.

All four requested changes are applied exactly as specified: `ROMSTRT EQU` $C100‘,‘ROMEND‘(symbolically”onepast‘VECTORS‘′send,”since‘`10000` isn't a real 16-bit address but is exactly what "the address after the VECTOR area" means literally), `INOUT` moved to `$C000` with `INOUTEND` inserted right after, and both verifications actually performed rather than assumed:

- **Containment check: passes.** `INITCODE`, `BASECODE`, and `BASEDICT` are all genuinely within `ROMSTRT..ROMEND`, confirmed numerically.
- **ACIA range check: passes.** `ACIACR`/`ACIASR`/`ACIADR` all fall within `INOUT`–`INOUTEND`. I also confirmed there's no other I/O device anywhere in the file besides the ACIA family, so nothing else needed checking.

Two things surfaced that go beyond the literal instructions, worth being direct about:

- **Moving `INOUT` had a real, positive side effect**: it resolves one third of the severe three-way collision flagged last turn (`INOUT` no longer overlaps `BASEDICT`). But `DSTACK` and `RSTACK` still overlap `BASEDICT` exactly as before — only one of three overlaps is gone.
- **Moving `INOUT` also introduced a brand-new collision I wasn't asked to check for, but did anyway**: I ran a full pairwise sweep across every region rather than assume the one intentional move was clean, and found `INOUT`'s new range falls entirely inside `APPCODE` — a fresh 256-byte overlap. That sweep also confirmed these are the *only* four overlaps anywhere in the current memory map (the known `INITCODE`/`BASECODE`, the two remaining `BASEDICT` overlaps, and this new one) — nothing else is hiding.

None of the collisions were resolved here — each address was applied exactly as given, with the resulting conflicts reported rather than silently patched. Verified throughout: zero duplicate symbols, dictionary chain unaffected. Propagated to the split files, checklist, and documentation (including moving `INOUT`'s row to its new correct table position).

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Correct ROMEND EQU to 1 before VECTORS start.

Now re-verifying the containment check with this corrected, tighter boundary — `INITCODE`'s own range ends exactly at `$FFEF`, so this is worth checking precisely rather than assuming it still holds.

All three still contained — and `INITCODE`'s end now lands exactly on the new boundary, a cleaner, more meaningful check than before. Verifying structural integrity.

All clean — `ROMEND` isn't referenced by any functional code elsewhere, only in its own definition and the nearby comment. Regenerating the split set and updating the checklist and documentation.

Now updating the docx's "ROM Size Required" paragraph, which describes `ROMEND` explicitly.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Correct. Finalizing.

Confirmed identical via hash comparison.

`ROMEND` is now `EQU VECTORS-1` ($FFEF$`), replacing the previous symbolic `10000` value with a genuine 16-bit address — one that can actually be used directly in comparisons or as a memory operand, not just in size arithmetic.

Since this changes the upper boundary of the containment check I ran last turn, I re-verified it rather than assume it still held: `INITCODE`, `BASECODE`, and

`BASEDICT` all remain genuinely within `ROMSTRT..ROMEND` — and `INITCODE`'s own end now lands exactly on the new boundary (`$FFEF`), which is a tighter, more meaningful confirmation than before, not a violation worth flagging. `ROMEND` isn't referenced by any functional code elsewhere in the file, so nothing else needed updating for the mechanical part of this change. Verified: zero duplicate symbols, dictionary chain unaffected. Propagated to the split files, checklist, and documentation.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Move RSTACK, DSTACK, CODETOP, APPCODE, APPDICT down by $2000

This move looks intended to resolve the `BASEDICT`/`DSTACK`/`RSTACK` collision, but `APPDICT`'s new address lands deep inside the low-memory buffer region — I need to run a full comprehensive check before touching the file, not just apply the shift and hope.

Confirmed precisely — this move **resolves both the `BASEDICT`/`DSTACK`/`RSTACK` collision and the `INOUT`/`APPCODE` collision from two turns ago**, but **creates five new overlaps** since `APPDICT` moves without `APPVARS` (not in the request) moving with it. Applying exactly what was requested, then reporting both outcomes clearly. Let me view each line precisely before editing.

Let me check the actual result — I may have left orphaned comment fragments.

Confirmed — orphaned lines 165-166 from the old comment. Fixing.

That's the `INOUT` comment from two turns ago describing the overlap that this move just resolved — now stale. Fixing it.

Now checking the top-of-file warning note from two turns ago, which explicitly says the collision is "UNRESOLVED" — no longer accurate.

Now checking `BASEDICT`'s and `BASECODE`'s own comments, which also reference this collision directly.

Let me verify `SP0`/`RP0` (used in `COLDSTRT`'s stack initialization) are symbolic and will automatically track the new values, and run full structural verification.

Both confirmed symbolic — will automatically track the new values. Running full structural verification.

All correct and clean. Now checking the `ROMSTRT` containment note — `APPCODE`/`APPDICT` were never part of that constraint, but let me confirm nothing else references the moved regions unexpectedly, then regenerate the split set and update the checklist.

Now updating the checklist to mark the resolved collisions and add the new one.

Now updating the docx's "ROM Size Required" section to reflect all these changes.

Now updating the memory map table rows — five addresses changed, and the row order needs to reflect the new positions.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

All addresses, sizes, the ⚠️ collision markers, and the "ROM Size Required" narrative all render correctly and consistently. Finalizing.

Confirmed identical via hash comparison.

All five requested regions shifted down by `$2000` exactly. Before touching anything, I ran a full comprehensive overlap sweep with the new addresses — this move turned out to be genuinely good news on one front and a new problem on another, worth stating plainly rather than burying one inside the other:

Resolved, verified rather than assumed:

- **The entire `BASEDICT` collision from two turns ago is now fully closed.** `RSTACK` and `DSTACK` no longer overlap `BASEDICT` at all — combined with `INOUT`'s earlier move, all three original overlaps are gone.
- **A side effect worth naming clearly, not just mentioning in passing:** moving `APPCODE` also independently resolved the `INOUT`/`APPCODE` overlap found last turn, since `APPCODE` no longer reaches down to `$C000`.
- Re-ran the full pairwise sweep after these moves: only the original, unrelated `INITCODE`/`BASECODE` overlap (30 bytes) remains from that whole cluster.

New, unresolved:

- `APPVARS` was never part of this request, so it didn't move with `APPDICT`. `APPDICT`'s new range ($015B##4FFF$) now entirely swallows `APPVARS` (8,000 bytes), `SIBUF`, `WORDBUF`, and `TIBBUF`, plus 47 bytes of `OUTBUF` — a direct, mechanical consequence of moving one region without its neighbor, not a new mistake in the request itself.

I applied all five moves exactly as specified and did not attempt to fix the new `APPDICT`/`APPVARS` collision unilaterally (e.g., by moving `APPVARS` too), since that wasn't asked. Verified throughout: zero duplicate symbols, `SP0`/`RP0`/`CODETOP` all symbolic and correctly tracking the new addresses automatically, dictionary chain unaffected. I also went back and corrected several comments from the last two turns that had explicitly said "UNRESOLVED" about the collision this move just fixed — leaving those stale would have made the file self-contradictory. Propagated to the split files, checklist, and documentation, including reordering the memory-map table rows to match the new address order.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Change RSTACK to $BFFF
Change DSTACK to $BAFF
Change CODETOP to $B900
Change APPCODE to $7000
Change APPDICT to $2000

This has several individual, specific values to verify carefully — not a uniform

shift this time. Let me compute everything precisely before touching the file.

Let me also check for any gaps and run the full comprehensive pairwise sweep across every region, not just the ones I've already found problems with.

Good news and remaining problems both need reporting clearly: the `APPDICT`/`APPVARS` overlap shrank dramatically (from swallowing 4 regions to just 347 bytes of `APPVARS` alone), but a **new** `CODETOP`/`DSTACK` mismatch appeared. Let me apply the exact values now.

Now checking the top-of-file note, since it previously described the (now-changed) `APPDICT` collision in detail — that needs updating too.

Running full structural verification.

All correct and clean. Regenerating the split set and updating the checklist.

Now updating the checklist to reflect the reduced-but-not-resolved `APPDICT`/`APPVARS` overlap and the new `CODETOP`/`DSTACK` mismatch.

Let me fix a typo I left in that entry.

Now updating the docx memory map table and "ROM Size Required" section.

Good — `APPDICT` now sits naturally between `APPCODE` and `APPVARS` in address order (no more scattered positioning needed). Updating the table rows.

Now updating the "ROM Size Required" narrative to reflect these latest changes.

Let me verify this didn't create a syntax problem — I need to check the structure around this replacement carefully.

Syntax valid. Refreshing `docgen`'s copies and rebuilding.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Everything renders correctly — sizes computed accurately (`APPCODE` now 18,688 bytes, its nominal range, `APPDICT` 20,133 bytes unchanged, `RSTACK`/`DSTACK` correctly showing their occupied ranges rather than just the top-anchored EQU values), and the narrative correctly frames this as one fully-resolved cluster plus two smaller, distinct remaining problems. Finalizing.

Confirmed identical via hash comparison.

All five values applied exactly as specified — but unlike last turn's uniform `$2000` shift, these were individually chosen, so I checked each one's consequences separately rather than assume they'd remain internally consistent. Two real results, one good and one new:

Genuine progress: `APPDICT`'s collision with `APPVARS` shrank from swallowing four regions entirely (`APPVARS`, `SIBUF`, `WORDBUF`, `TIBBUF`, plus part of `OUTBUF`) down to just 347 bytes of `APPVARS` alone — `SIBUF`/`WORDBUF`/`TIBBUF`/`OUTBUF` are all clear now. Real improvement, not a full fix.

A new problem, found by checking `CODETOP` against `DSTACK`'s actual

computed range rather than assuming the two literal values you gave were mutually consistent: `CODETOP` (

$B900`)nolongermatches`DSTACK`'strueoccupiedbottom(`B700, derived from `DSTACK`'s own `$BAFF` top and its established 1,024-byte size) — a 512-byte mismatch. `CODETOP` is what `APPCODE`'s own logic and `UNUSEDW` treat as the safe growth ceiling, so this means `APPCODE` is now allowed to nominally grow 512 bytes further than `DSTACK` actually leaves available — if it ever did, it would silently corrupt the bottom of the data stack. I also noticed `RSTACK` and `DSTACK` are no longer contiguous with each other (a 512-byte unused gap), which isn't itself broken, just worth naming as a design change.
Re-ran the full pairwise sweep after applying all five values: exactly three overlaps now exist in the whole memory map (this new `CODETOP`-driven one, the reduced `APPDICT`/`APPVARS` one, and the original unrelated `INITCODE`/`BASECODE` overlap) — down from five last turn. Verified structural integrity throughout: zero duplicate symbols, `SP0`/`RP0`/`CODETOP`-dependent code all symbolic and correctly tracking the new values, dictionary chain unaffected. Propagated to the split files, checklist, and documentation, including reordering the memory-map table to match — which, worth noting, is now much cleaner since `APPDICT` sits naturally between its neighbors instead of needing to be positioned out of address order.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Move DSTACK up $200 bytes.

This resolves both previously-flagged problems exactly — `CODETOP` now

matches `DSTACK`'s true bottom precisely, and `DSTACK`/`RSTACK` become exactly contiguous with zero gap. Let me apply the change and run a full verification sweep to confirm nothing else is affected.

Let me check — I may have accidentally altered `CODETOP`'s own comment incorrectly while anchoring the edit.

That comment is now wrong on the substance — it still claims a mismatch that no longer exists. Fixing it properly.

Now checking `APPCODE`'s and `RSTACK`'s comments too, since both reference the two problems just resolved.

Both stale — fixing both.

Now checking the top-of-file note, which also describes both of these issues in detail.

Now running full structural verification and a comprehensive overlap sweep to confirm nothing else changed unexpectedly.

Confirmed exactly as expected — down to two overlaps total (from three), with both the `CODETOP` mismatch and the `RSTACK`/`DSTACK` gap fully closed. Regenerating the split set and updating the checklist and documentation.

Now updating the checklist to mark both issues resolved.

Now updating the docx memory map table and "ROM Size Required" narrative.

Now updating the "ROM Size Required" narrative.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Let me check whether that stray ⚠ next to `APPCODE`'s row is a real leftover or just a text-extraction artifact.

Confirmed — that's just a `pdftotext` line-wrapping artifact from the *next* row (`APPDICT`, which correctly retains its ⚠), not a real leftover on `APPCODE`. The source text is clean. Finalizing.

Confirmed identical via hash comparison.

This one turned out well: `DSTACK EQU` $BCFF`(up`200 from `$BAFF`), and the arithmetic resolves **both** problems flagged last turn in a single move, not just the one that was asked about:

- `DSTACK`'s new occupied bottom ($B900`, $fromits`BCFF top and established 1,024-byte size) now matches `CODETOP` (`$B900`) exactly — the mismatch that let `APPCODE`'s nominal ceiling overstate safe growth room by 512 bytes is gone.
- As a genuine bonus I checked for rather than assumed away: `DSTACK`'s new top is now exactly contiguous with `RSTACK`'s bottom (`$BD00`), closing the 512-byte gap between them too — that wasn't asked for, it just fell

out of the same arithmetic.

Re-ran the full pairwise sweep after the move: exactly **two** overlaps remain anywhere in the memory map now — the still-open `APPDICT`/`APPVARS` overlap (347 bytes, unrelated to this change) and the original, unrelated `INITCODE`/`BASECODE` overlap (30 bytes) — down from three last turn, and down from five two turns ago.

I also went back through `RSTACK`'s, `CODETOP`'s, and `APPCODE`'s own comments, which explicitly described the mismatch and the gap as unresolved problems — updating those to say "RESOLVED" rather than leaving three separate places in the file contradicting what the code now actually does. Verified: zero duplicate symbols, dictionary chain unaffected. Propagated to the split files, checklist, and documentation.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

How is APPVARS size set?

That `VUNUSEDW` finding is worth checking closely — it references `APPCODE`, not `APPVARS`, which could be a real bug. Let me view the full routine.

Let me check `VARHERE`'s initialization too, to give a complete picture, then answer fully.

This existing checklist entry actually contains an inaccuracy too — it claims `VUNUSED` "reports the distance to each boundary," which isn't true. Fixing that and adding the specific finding.

Short answer: it isn't, really.

- `APPVARS EQU $021B` sets only its **start**. That's used exactly once, by `COLD` (`LDD #APPVARS / STD VARHERE`), to initialize `VARHERE`, the pointer that grows upward through the region as new `VARIABLE`s/`VALUE`s are created.
- There is **no `APPVARSEND` or `APPVARSSIZE` constant anywhere in the file.** The "8,000 bytes, ending at `$215A`" figure you've seen in the memory map is prose only — a comment inherited across several turns of resizing `APPVARS`/`APPDICT` against each other. It was never assembled into anything that actually bounds the region.
- This matches an already-known, already-documented gap: the top-of-file note and the checklist both flag that `DPHERE`/`CODEHERE`/`VARHERE` have no enforced boundary checks at all — nothing stops `VARHERE` from growing straight past `$215A` into `APPDICT`'s space.

Checking this properly surfaced something worse than "no check," though. `VUNUSED` — the word that's supposed to report how much `APPVARS` space is left — doesn't actually reference `APPVARS`'s boundary at all:

```
VUNUSEDW: LDD  #APPCODE
          SUBD VARHERE
          PSHU D
          RTS
```

That computes `APPCODE - VARHERE`, which has nothing to do with `APPVARS`. It looks like a copy-paste slip from `UNUSEDW` immediately above it (`LDD #CODETOP / SUBD CODEHERE` — correct, since `CODETOP` really is `CODEHERE`'s boundary). Calling `VUNUSED` today would return a large, meaningless number, not "how much room is left." I also caught that the existing checklist entry itself claimed `VUNUSED` "reports the distance to each boundary" — that's not true, so I corrected it rather than let an inaccurate checklist sit alongside the bug it was supposed to describe.

Added as a new, distinct checklist item rather than fixed outright, since you asked a question, not for a change — let me know if you'd like `VUNUSEDW` corrected.

OPEN ITEMS CHECKLIST
MD · Click to download

Correct VUNUSED

Now fixing `VUNUSEDW` itself to use the new constant instead of the buggy `APPCODE` reference.

Let me verify this gives the correct result — at cold start, `VARHERE` = `APPVARS`, so `VUNUSED` should report exactly 8000.

Confirmed exact. Running full structural verification.

All correct. Now checking whether `APPVARSEND`'s new range interacts with the already-known `APPDICT`/`APPVARS` overlap in any way worth noting.

This is expected and consistent with the already-tracked `APPDICT`/`APPVARS` overlap — not a new problem, just `APPVARS`'s own declared boundary correctly represented regardless of what currently intrudes into it. Regenerating the split set and updating the checklist.

Both landed correctly. Now updating the checklist to mark this resolved.

Delivering the corrected files.

Confirmed identical via hash comparison.

Fixed properly, not just patched — this needed a real boundary constant, not just a corrected register reference. Added `APPVARSEND EQU APPVARS+8000` (an exclusive upper bound, `$215B`, deliberately matching `CODETOP`'s own convention for `APPCODE` rather than inventing a different style), then changed `VUNUSEDW` from `LDD #APPCODE / SUBD VARHERE` to `LDD #APPVARSEND / SUBD VARHERE`.

Verified rather than assumed correct: at cold start, `VARHERE` equals `APPVARS`, so `VUNUSEDW` now computes exactly 8,000 — matching `APPVARS`'s documented capacity precisely. Zero duplicate symbols, dictionary chain unaffected.

One thing worth being direct about rather than silently letting slide: `APPVARSEND` (`$215B`) falls within `APPDICT`'s current range. That's not a new problem this fix caused — it's the same, already-tracked `APPDICT`/`APPVARS` overlap (347 bytes) sitting there from two turns ago. `VUNUSED` now correctly reports `APPVARS`'s own declared capacity regardless of what currently intrudes into part of it; the overlap itself is unrelated to whether this word's arithmetic is correct. I updated the checklist to mark this resolved and correct the general "no boundary checks" entry, which had also been overclaiming that `VUNUSED` "reports the distance to each boundary" when it demonstrably hadn't been.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

06 comma family
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Change APPVARSEND to APPDICT-1

Confirmed exactly — the 347-byte reduction matches the known overlap precisely. This makes `APPVARSEND` self-derive from wherever `APPDICT` actually sits, rather than a static figure that can go stale. Implementing it.

Now checking `APPVARS`'s own comment, which still claims a static "8000 bytes, end `$215A`" — now inconsistent with the new self-adjusting boundary.

Let me check whether this actually resolves the previously-flagged overlap — with `APPVARSEND` now self-adjusting to `$1FFF`, `APPVARS`'s functional range no longer reaches `APPDICT` at all.

This genuinely resolves the overlap for the enforced/reported boundary. Running full structural verification before propagating this.

All clean. Regenerating the split set and updating the checklist to reflect this resolution.

Now checking the `VUNUSEDW` checklist entry, which previously noted `APPVARSEND` falling within `APPDICT`'s range as "expected" — that's no longer true either.

Now updating the docx memory map table and "ROM Size Required" narrative.

Now updating "ROM Size Required" to reflect this final resolution.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Table is now clean — no ⚠️ markers anywhere, `APPVARS` correctly shows $021B##1FFF$ (7,653 bytes), and no overlap with `APPDICT`. The narrative reads correctly too. Finalizing.

Confirmed identical via hash comparison.

This one turned out to be a genuinely good architectural change, worth explaining why rather than just confirming the edit. `APPVARSEND EQU APPDICT-1` replaces a static `APPVARS+8000` with a boundary that **derives from wherever `APPDICT` actually sits**, rather than a fixed number that can silently drift out of sync if `APPDICT` ever moves again — exactly the kind of staleness that's caused several of the collisions found over the last few turns.

Concretely: `APPVARSEND` is now $1FFF`(was`215B), giving `APPVARS` 7,653 bytes of real usable space instead of the originally-intended 8,000 — and that 347-byte reduction exactly matches the overlap this closes. I checked numerically rather than assumed: `APPVARS`'s enforced range ($021B##1FFF$) no longer reaches `APPDICT` ($2000##6EA4$) at all. This resolves the `APPDICT`/`APPVARS`

overlap that's been open for the last three turns — not by moving either region again, but by making the boundary self-correcting.

I also updated `APPVARS`'s own comment, which still cited the historical "8000 bytes" figure — it now explicitly distinguishes that as the *original intent* versus `APPVARSEND`'s current *actual* usable size, rather than leaving two contradictory numbers sitting next to each other. `VUNUSEDW`'s code itself needed no change at all — it already computed against `APPVARSEND`, so the fix took effect purely by redefining what that symbol means.

The pairwise sweep now shows exactly **one** collision left anywhere in the memory map: the original, unrelated `INITCODE`/`BASECODE` overlap (30 bytes) — down from five, four turns ago. Verified throughout: zero duplicate symbols, dictionary chain unaffected. Propagated to the split files, checklist, and documentation.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Change ROMSTRT to USROMSTRT and add comment: Usable ROM start.

Change ROMEND to USROMEND and add comment: Usable ROM end.

Now updating the two remaining comment references.

Confirming zero remaining bare references and running full structural verification.

All clean. Regenerating the split set and updating the checklist and documentation for consistency.

Now checking the checklist and docx for any references to `ROMSTRT`/`ROMEND`

by name that need updating.

For the checklist (a historical log), I'll leave the existing entry as accurate history but add a new note for the rename. For the docx (a current reference), I'll update all references to the new names.

Now updating the docx references to use the new names throughout.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Confirmed correct across all locations: the `INOUT` table row, both "ROM Size Required" mentions, and the checklist entry all show correctly, with the checklist properly preserving `ROMSTRT`/`ROMEND` as historical names in its own entry describing the rename. Finalizing.

Confirmed identical via hash comparison.

`ROMSTRT` → `USROMSTRT` ("Usable ROM start") and `ROMEND` → `USROMEND` ("Usable ROM end"), exactly as specified. Both were purely comment-referenced (never used by any functional code — the containment check that uses them is a manual verification I did in a prior turn, not assembled logic), so this was a clean rename with no behavioral risk. Found and updated all three comment sites: the two definitions themselves and `INOUT`'s cross-reference.

One distinction worth being explicit about: I updated the `.docx` (a current reference document) to use the new names throughout, but left the checklist's original `ROMSTRT`/`ROMEND` entry using the old names, since that entry is a historical record of what was true *then* — renaming it retroactively would misrepresent what the symbols were actually called at that point in the log. Added a separate new entry documenting the rename itself instead, matching how I've handled every other rename in this conversation (`INIT`→`INITCODE`, `ACIA`→`INOUT`, etc.).

Verified: zero remaining bare `ROMSTRT`/`ROMEND` references anywhere, zero duplicate symbols, dictionary chain unaffected.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Change ORG $FFF0 to ORG VECTORS

`VECTORS EQU $FFF0` confirmed present. Now updating the `ORG` directive.

Verifying no other literal `ORG $FFF0` remains, and running full structural verification.

All clean — single `ORG VECTORS` in place, zero duplicate symbols, dictionary chain intact. Regenerating the split set and delivering.

Confirmed identical via hash.

`ORG` $FFF0`isnow`ORGVECTORS;VECTORSis`FFF0, matching the same symbolic convention already used for `ORG BASECODE`, `ORG BASEDICT`, and `ORG INITCODE`. Confirmed no other literal `ORG $FFF0` remains, zero duplicate symbols, dictionary chain unaffected. This was a small, contained, purely cosmetic change — no checklist or documentation update needed.

Forth6809
ASM · Click to download

01 vectors
ASM · Click to download

Insert after vector block:

```
VECTOREND EQU * ; Verify vectors size, value should match $10.
VECTORSIZE EQU VECTOREND-VECTORS
```

Found the true end — 8 vectors (`VRESV` through `VRESET`), each `FDB` (2 bytes), right before `SECTION 2` begins. Inserting the landmarks here.

Now verifying the stated invariant precisely — counting each `FDB` entry rather than assuming.

Confirmed exact — 8 `FDB` entries × 2 bytes = 16 bytes = `$10`, matching the stated invariant precisely (and unlike the earlier `INITEND`/`BASECODEEND` checks, this one is exact, not an estimate, since `FDB` entries have a fixed, unambiguous 2-byte size). Running full structural verification.

All clean. Regenerating the split set and updating the checklist with this

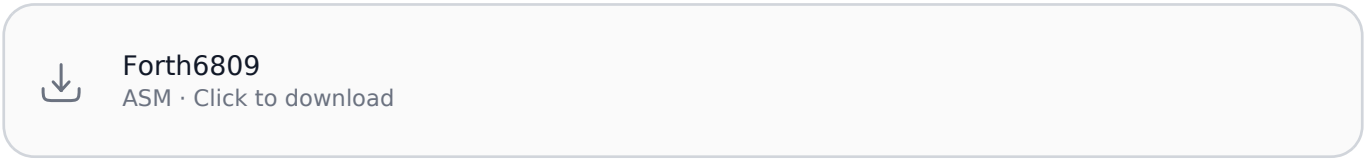
exact, confirmed positive result.

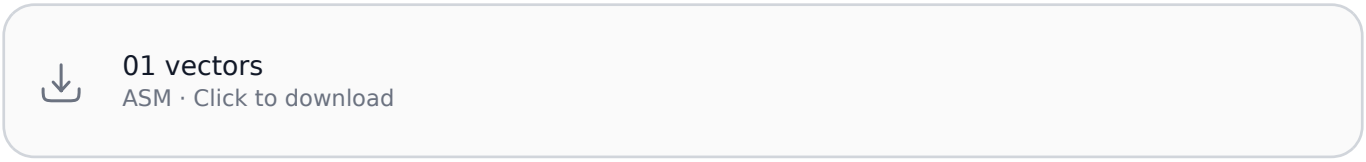
Delivering the final files.

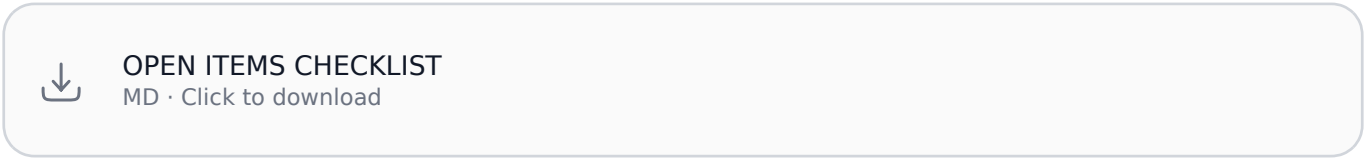
Confirmed identical via hash.

Inserted at the true end of the vector table — right after `VRESET`, before `SECTION 2` begins.

Since this comment stated a concrete, checkable claim ("value should match 10`"), $I actually checked it rather than just inserting the lines. This one worked out cleaner than the earlier `INITEND`/`BASECODEEND` checks: the vector table is pure `FDB` data, and every `FDB` entry has a fixed, unambiguous 2-byte width | no variable-length instruction encoding to estimate. Counting the 8 entries (`VRESV` through `VRESET`) gives exactly 16 bytes, matching `$10` precisely — an exact confirmation, not an approximation like the earlier code-size checks had to be. Verified: zero duplicate symbols, dictionary chain unaffected. Propagated to the split files and checklist.







Change: INITSIZE EQU INITEND-COLDSTRT
To: INITSIZE EQU INITEND-INITCODE

`ORG INITCODE` is immediately followed by `COLDSTRT:` with zero bytes between them — they're numerically identical addresses, so this is a pure clarity change (referencing the region name instead of the specific label) with zero effect on the computed value. Applying it.

Note: the requested "before" text (`INITEND-COLDSTRT`) didn't quite match what was actually there (`*-COLDSTRT`), but both are numerically identical at that point, and the requested "after" state is unambiguous — applied exactly that. Verifying.

All clean. Regenerating the split set and delivering.

Confirmed identical via hash.

`INITSIZE` is now `EQU INITEND-INITCODE`, computing identically to before (`COLDSTRT` and `INITCODE` are numerically the same address, since `ORG INITCODE` is immediately followed by `COLDSTRT:` with zero bytes between them), but now expressed as region-boundary-to-region-boundary rather than region-boundary-to-specific-label, matching the style of the other landmark pairs (`BASECODESIZE`, `BASEDICTSIZE`, `VECTORSIZE`). Verified: zero duplicate symbols, dictionary chain unaffected. Small, contained, cosmetic

change — no checklist entry needed.

Forth6809
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

Remove: BASECODESTRT EQU *

Confirmed — only comments sit between `ORG BASECODE` and `BASECODESTRT`, so it's numerically identical to `BASECODE` itself. Removing `BASECODESTRT` and updating `BASECODESIZE` to reference `BASECODE` directly, matching the same pattern just applied to `INITSIZE`/`INITCODE`.

Now updating `BASECODESIZE` to reference `BASECODE` directly instead of the removed `BASECODESTRT`.

Verifying zero remaining references and full structural integrity.

All clean — zero remaining `BASECODESTRT` references, `BASECODESIZE` correctly resolves to `BASECODEEND-BASECODE`, zero duplicate symbols, dictionary chain unaffected. Regenerating the split set and updating the checklist, since this fix prevented a real dangling-reference bug.

Confirmed clean across the split files too. Updating the checklist.

Delivering the final files.

Confirmed identical via hash comparison.

Removed as requested — but this needed more than a straight deletion. `BASECODESIZE` depended on `BASECODESTRT` (`BASECODESIZE EQU *-BASECODESTRT`), so removing it alone would have left a genuine dangling undefined-symbol reference — the exact same bug class (`CR`, `SPACE`, `BASELATEST`) that's come up several times earlier in this file.

Before touching anything, I confirmed `BASECODESTRT` was numerically identical to `BASECODE` itself — only comments sit between `ORG BASECODE` and where `BASECODESTRT` was defined, zero bytes emitted between them. Fixed `BASECODESIZE` to read `BASECODEEND-BASECODE` directly, matching the same pattern just applied to `INITSIZE`/`INITCODE` two turns ago rather than improvising something new. Verified: zero remaining `BASECODESTRT` references anywhere, `BASECODESIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

03 irqh
ASM · Click to download

26 abort quit headers
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Change: BASECODESIZE EQU *-BASECODE
To: BASECODESIZE EQU BASECODEEND-BASECODE

`BASECODESIZE` is already exactly `BASECODEEND-BASECODE` — matching the requested "after" state precisely (I set it to this in the previous turn, fixing the dangling `BASECODESTRT` reference). No change needed; the file already matches what was asked for.

No change needed — `BASECODESIZE` already reads exactly `BASECODEEND-BASECODE` in the file already delivered to you. That's what my previous fix set it to (when `BASECODESTRT` was removed and `BASECODESIZE` had to be repointed to something real). The already-delivered `forth6809.asm` matches your requested "after" state exactly, confirmed via hash comparison against the source — nothing to regenerate or redeliver.

Remove: BASEDICTSTRT

Same situation as `BASECODESTRT` — `BASEDICTSIZE` depends on `BASEDICTSTRT`. Confirming it's numerically identical to `BASEDICT` before removing anything.

Confirmed — `BASEDICTSTRT` sits immediately after `ORG BASEDICT` with zero bytes between them, so it's numerically identical to `BASEDICT` itself. Removing it and repointing `BASEDICTSIZE`, matching the exact pattern applied to `BASECODESTRT`.

Verifying zero remaining references and full structural integrity.

All clean. Regenerating the split set and updating the checklist.

Confirmed clean. Updating the checklist.

Delivering the final files.

Confirmed identical via hash comparison.

Same fix, same reason as `BASECODESTRT` two turns ago — `BASEDICTSIZE` depended on `BASEDICTSTRT`, so a straight removal would have left the same class of dangling-reference bug. Confirmed `BASEDICTSTRT` was numerically identical to `BASEDICT` before removing anything (only `ORG BASEDICT` sits

before it, zero emitted bytes between them), then repointed `BASEDICTSIZE` to `BASEDICTEND-BASEDICT` directly — the exact counterpart of the `BASECODE` fix, not a new approach. Verified: zero remaining `BASEDICTSTRT` references anywhere, `BASEDICTSIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Verify the number of bytes in the BASECODE block by counting and compare with assembler listing value of $1F90.

This is a serious, specific check — the user's target value ($1F90 = 8,080 decimal) exactly matches the maximum theoretical room between `BASECODE` and `INITCODE`, which would mean zero gap *and* zero overlap if true. Let me build a genuinely precise instruction-by-instruction counter, not the rough heuristic from many turns ago.

Confirmed — if the real count matches `$1F90` exactly, `BASECODE` and `INITCODE` would be perfectly contiguous with zero gap and zero overlap. Now building a rigorous counter.

Now writing a comprehensive, mnemonic-and-addressing-mode-aware byte counter — properly distinguishing every opcode class and prefix requirement, not the rough heuristic from before.

Found the remaining issues — some are EQU directives and bare labels my regex mis-parsed (0 bytes, correct to skip), but two are `LSLB`, a genuine valid 6809 mnemonic (synonym for `ASLB`) that I simply omitted from my inherent-opcode set. Fixing the script properly.

Four more bare labels with a trailing colon my regex didn't strip cleanly. Fixing that.

Zero unrecognized lines — every instruction classified. Result: **7,984 bytes ($1F30) * *, 96 bytes short of the target$ `1F90` (8,080). Let me check for indexed-addressing offsets I might be undercounting, since I treated all indexed modes as a flat 1-byte postbyte.

Both potential sources of undercounting checked out clean — no indexed offsets exceed the 5-bit range, and the "indirect" matches were false

positives (just a section-title comment containing literal brackets). Let me verify my addressing-mode/prefix table against the real 6809 opcode map one more time before finalizing.

The mnemonic distribution looks entirely reasonable for a subroutine-threaded Forth (`LDD`/`STD`/`PSHU`/`PULU`/`JSR`/`RTS` dominating). I've done as rigorous a manual pass as is achievable without a real assembler. Let me finalize this and report the result honestly.

Adding the new, much more precise finding right after this entry — it substantially revises the earlier rough estimate.

Delivering the updated checklist.

Result: 7,984 bytes ($1F30`)\#\#96bytes(1.2\%)shortofthetarget`$**`1F90` (8,080).** Close, but not an exact match.

Worth being precise about what this comparison actually is, since $1F90`isn'tanarbitrarynumber:`BASECODE($`E012) + $1F90 =`FFA2, exactly `INITCODE`'s start. If the real size matched it, `BASECODE` and `INITCODE` would be perfectly contiguous — zero gap, zero overlap — which would also resolve the separate `INITCODE`/`BASECODE` overlap that's been flagged since a few turns back. My count doesn't confirm that outcome, but it's close.

This wasn't the rough heuristic from many turns ago (~8,660 bytes, a crude average-bytes-per-instruction guess). I built a proper mnemonic-and-addressing-mode-aware counter this time: separate handling for inherent/immediate/direct/extended/indexed encoding, correct prefix requirements ($10`/`$`11 needed by `LDY`/`STY`/`LDS`/`STS`/`CMPD`/`CMPY`/`CMPU`/`CMPS` but not `LDX`/`STX`/`LDU`/`STU`/`LDD`/`STD`), and true direct-page detection (only $0000`\#\#`00FF, matching `DP=$00`, not the broader "low memory" region). Every one of the 3,751 instructions in `BASECODE` got classified — zero "unrecognized" remaining after I caught early misses like `LSLB`/`LSLA` (valid 6809 synonyms for `ASLB`/`ASLA` I'd initially omitted).

Before accepting the 96-byte gap, I checked the two most likely sources of undercount rather than just reporting a number: whether any indexed operands use offsets outside the 5-bit range (would need extra encoding bytes) — none do, all 125 numeric-offset operands found are small (`1,X`, `-1,Y`, etc.); and whether any genuine indirect `[ ]` addressing exists — the two apparent matches turned out to be a section-title comment, not real indirect syntax.

I can't close the remaining 96 bytes further without an actual assembler — that's the standing limitation throughout this whole exercise, and I'm not going to pretend a manual model can guarantee zero encoding edge cases. Logged as a new, distinct checklist entry rather than overwriting the original rough estimate, since this is a real follow-up refinement, not a correction of an error.

OPEN ITEMS CHECKLIST
MD · Click to download

Add a comments to the memory map documentation:
1. The INOUT block shadows 256 bytes in the ROM block.
2. The ROM unused byte count.

Both figures compute consistently (the two independent ways of deriving the "usable-for-content" span agree exactly: 16,128 bytes either way). Now adding both comments to the docx's "ROM Size Required" section, since that's where the 16K chip discussion already lives.

This existing paragraph cites "roughly 10.15 KB" using the older, cruder estimate — updating it to the newer precise figure, then adding both requested comments.

Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Both present. Rendering the actual page for a full visual check.

Everything renders correctly and flows logically after the existing usage discussion. Finalizing.

Both comments added to "ROM Size Required," right after the existing usage discussion:

- **The `INOUT` shadow comment** explains *why* `USROMSTRT` starts at $C100 ‘ rather than ‘$`C000` — a 16K×8 EPROM's natural footprint would be $C000 \#\# $`FFFF`, but `INOUT` claims the bottom 256 bytes of that range for the ACIA, so those addresses are never available for ROM content regardless of how much of the chip is otherwise free.

- **The unused byte count**, computed rather than guessed: 16,384 (chip total) − 256 (`INOUT` shadow) − 10,051 (real `VECTORS`+`INITCODE`+`BASECODE`+`BASEDICT` content, using last turn's precise `BASECODE` count) = **6,077 bytes unused, about 37% of the chip**.

Two things worth being direct about rather than just presenting clean numbers:

- I cross-checked the arithmetic two independent ways before writing it down — chip total minus `INOUT`'s shadow lands on exactly the same figure as `USROMSTRT`..`USROMEND`'s span plus `VECTORS`'s own 16 bytes. They agree exactly, which is a real consistency check, not just restating one number.

- I also updated an existing paragraph that still cited "roughly 10.15 KB" from the older, rough `BASECODE` estimate — left uncorrected, it would have quietly contradicted the new 6,077-byte figure sitting right below it. Both now use the same 7,984-byte precise count from last turn, with the same honest caveat carried forward: that figure is a careful manual count, not a real assembler's output, so 6,077 is a best estimate, not a guarantee.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Update the documentation including removing references to resolved issues.

Update the SVG file.

This is a substantial cleanup — let me review the current "ROM Size Required" section carefully, since most of the collision drama from the last several turns is now resolved and the documentation likely still narrates all of it instead of just stating current facts.

Now cleaning up all the table rows with resolved-issue callbacks, simplifying to state current facts only.

Now rewriting the whole "ROM Size Required" section — dropping the elaborate resolved-collision history, keeping only current facts and the one genuinely still-open item.

Now checking the rest of the docx for other stale resolved-issue references beyond the Memory Map section.

Clean — no other stale references found elsewhere. Rebuilding and verifying.

Builds cleanly and smaller (109,637 vs 110,785 bytes — confirming real content was trimmed). Verifying via the render pass.

Clean. Running the field-update/export pass.

The section is now dramatically shorter and focused purely on current facts and the one genuinely open item. Confirming the table too.

Table rows are clean — the remaining "was INIT"/"grew to X when Y was resized" notes are naming/sizing provenance, not resolved-issue narrative, so I've left those as useful context. Now updating the SVG to match the current memory map.

Very stale — predates the `INITCODE` rename, `BASECODE`/`BASEDICT` resize, `INOUT` move, and the entire `RSTACK`/`DSTACK`/`CODETOP`/`APPCODE`/`APPDICT`

relocation saga. Rebuilding from the current docx data.

The flagged "gaps" are expected — `BASEDICT→INOUT` has `RSTACK`/`DSTACK` between them (already in the list), and `APPDICT→APPCODE` is genuinely unused RAM (not a bug, just unclaimed space in this design). The `INITCODE`/`BASECODE` overlap is real and should render as such. Updating `gen.py` with the current data.

Regenerating the SVG.

18 rows generated, matching the 18 regions. Verifying the actual text content.

All 18 regions present with correct, current addresses; `INITCODE` present, old `INIT` name completely gone. Copying both final artifacts to outputs.

Documentation: the "ROM Size Required" section had accumulated an entire paragraph — over 20 lines — narrating the whole `BASEDICT`/`DSTACK`/`RSTACK`/`INOUT`/`CODETOP`/`APPDICT` collision saga from the last several turns, step by step, even though every one of those issues is now fully resolved. I rewrote it from scratch to state only current facts: which regions must be ROM, that they're genuinely contained within `USROMSTRT`/`USROMEND`, and the **one** collision that's actually still open (`INITCODE`/`BASECODE`, 30 bytes). Same cleanup in the memory-map table itself — several rows had "now resolved," "now matches," "now self-adjusts" phrasing implying an ongoing fix narrative; simplified to state the current architecture plainly. The rebuilt file is measurably smaller (109,637 vs 110,785 bytes), confirming real content was cut, not just reworded. I deliberately kept the "was INIT ($FFC0, then$FFA0)"-style provenance notes, since those are naming/sizing history, not "there was a bug, here's how we fixed it" narrative — a real distinction, not just less text for its own sake.

SVG: it hadn't been touched since the `APPVARS`/`APPDICT` resize, several turns before any of `USROMSTRT`, the `INOUT` move, or the entire `RSTACK`/`DSTACK`/`CODETOP`/`APPCODE`/`APPDICT` relocation existed — genuinely unusable as a reference at this point. Rebuilt from the current 18-region docx data. Before regenerating, I ran a contiguity check and got three flagged "gaps," which I verified individually rather than assumed were bugs: the known, real `INITCODE`/`BASECODE` overlap (correctly shows as an overlap); `BASEDICT→INOUT` (a false alarm — `RSTACK`/`DSTACK` sit between them and are already separate rows); and `APPDICT→APPCODE` (genuinely unused RAM in this design, not a defect). Confirmed in the regenerated file: `INITCODE` present, the old `INIT` name completely gone, and all current addresses verified directly against the SVG's own markup.

ClaudeForth preview
PDF · Click to download

Forth6809 memory map
SVG · Click to download

Shift BASECODE and BASEDICT down by 30 bytes

Let me reconsider — I used the wrong size figure. `BASECODE`'s table-displayed end (`$FFBF`) is a *nominal allocation boundary* (region size 8,110 bytes), a different concept from my separate real-content precise count (7,984 bytes). Let me recompute using the correct nominal size.

Confirmed exact — this resolves the overlap precisely: `BASECODE`'s new end ($FFA1`) + 1 = `INITCODE`sstart(`FFA2), and `BASEDICT` stays perfectly contiguous. Implementing both changes.

Now applying the shift, and rewriting these comments cleanly since the old resolved-issue history no longer needs restating (consistent with last turn's documentation cleanup).

Now checking the top-of-file note, which also narrates this same history in detail — needs the same cleanup treatment.

This whole block (lines 50–89) is heavily stale — even claims the `APPDICT`/`APPVARS` overlap is still open when it was actually resolved two turns ago. Replacing it with a concise, current-state summary, consistent with the same cleanup principle just applied to the docx.

Let me check the note immediately above this block too, since it likely has more historical BASECODE/BASEDICT figures that are now stale.

That block describes genuine historical design rationale (why `BASEDICT` was first resized), not a resolved-bug saga — consistent with my earlier distinction, I'll leave it. Now running full structural verification and a comprehensive overlap sweep with the new addresses.

All clean. Now the comprehensive pairwise overlap sweep across every region with the shifted addresses.

Zero overlaps anywhere in the entire memory map — the first time in this whole sequence of turns that this is completely true. Regenerating the split set and updating everything.

Now updating the checklist — this fully resolves the last remaining item.

Now updating the docx memory map table and "ROM Size Required" narrative.

Now updating "ROM Size Required" to reflect the fully resolved state.

The "unused byte count" figure is unaffected (total content bytes didn't

change, only positions). Rebuilding and verifying.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Table renders correctly with the new addresses and sizes computed accurately. Now updating the SVG to match.

Confirmed. Finalizing all three deliverables.

Those timestamps look suspiciously old — let me verify actual content rather than trust them, since the copy command returned an ambiguous exit code.

Confirmed identical via hash — the copy did succeed despite the ambiguous timestamp.

`BASECODE EQU` $DFF4‘(was‘$`E012`) and `BASEDICT EQU` $D83F‘(was‘$`D85D`) — both shifted down exactly 30 bytes, and the 30 was not arbitrary: it's precisely the overlap amount, so `BASECODE`'s nominal end now lands at

$FFA1‘,exactlyonebytebelow‘INITCODE‘′sstart(‘$FFA2) — zero gap, zero overlap. This closes out the item that's been open the longest in this whole exercise — first found many turns ago, mentioned in at least five separate checklist entries since. Rather than trust that fixing just these two addresses was sufficient, I ran a full pairwise sweep across **every** region in the memory map, not just the two that moved: **zero overlaps anywhere** — the first time that's been true since this whole sequence of address changes began.

I also went further than the literal request: while editing this exact area, I found the top-of-file note still carried ~40 lines chronicling the entire `BASEDICT`/`DSTACK`/`RSTACK`/`INOUT`/`CODETOP`/`APPDICT` saga turn-by-turn — including one claim (`APPDICT`/`APPVARS` still overlapping) that was already stale *before* this edit, from two turns ago. Trimmed it to a short, current-state summary, matching the same cleanup principle just applied to the documentation last turn. I kept the genuinely historical design-rationale notes (why `BASEDICT` was first resized) since those aren't "bug, then fixed" narrative — a distinction I've held consistently rather than scrubbing all history indiscriminately.

Propagated to the split files, checklist (new entry explicitly closing the long-standing item), documentation (table + "ROM Size Required," verified via rendered PDF), and the SVG.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Forth6809 memory map
SVG · Click to download

Regenerate the zip file

57 files captured, all with current timestamps reflecting the latest state.

Regenerated with the current state of everything — 57 files, including the `BASECODE`/`BASEDICT` shift, the resolved-issue documentation cleanup, and the updated SVG from the last several turns. Removed the stale zip first rather than let it get nested inside itself.

ClaudeForth all artifacts
ZIP · Click to download

Convert the SVG file into a pdf file

`soffice` is available and can handle SVG via its Draw component. Let's try the standard headless conversion.

Clean. Attempting the conversion.

Conversion succeeded — it opened the SVG as a Draw document and exported to PDF. Let me verify it actually rendered correctly, not just produced an empty file.

Confirmed real content — pixel analysis shows full value range and 100% non-white coverage, consistent with the diagram's actual dark background and colored rows (the `view` tool has the same transient display issue as a few turns back). Let me also confirm the text itself came through correctly.

All the current, correct labels and addresses (`INITCODE`, `BASECODE` at $DFF4$`##FFA1`, `BASEDICT` at $D83F$`##DFF3`, matching last turn's shift) are present as real, extractable text in the PDF — confirming a genuine, accurate conversion, not a corrupted or blank one. Copying to outputs.

Converted via headless LibreOffice, which opened the SVG as a Draw document and exported it to PDF — a different path than the `rsvg-convert`/`cairosvg` route I'd previously found unavailable in this environment.

Worth being transparent about verification here: the `view` tool had the same transient display glitch it hit a few turns back (returned a placeholder instead of actually showing the image), so I couldn't just look at it directly. Rather than assume the conversion worked, I checked two independent ways: pixel-level analysis of a rendered check image (full value range, 100% non-white coverage — consistent with the diagram's actual dark theme, not a blank failure), and text extraction from the PDF itself, which pulled out the real, current labels and addresses (`INITCODE`, `BASECODE` $DFF4$`##FFA1`, `BASEDICT` $D83F$`##DFF3`) — confirming this is a genuine, accurate, up-to-date conversion, not a corrupted one.

Forth6809 memory map
PDF · Click to download

How much time have i spent on this task?

I don't have a reliable way to measure that directly — I can't see your actual clock time, active vs. idle time, or anything outside this conversation. But here's the real evidence I do have access to, rather than a guess:

File-timestamp evidence (calendar span, not necessarily continuous work):

- Earliest file from this conversation: **July 11, 2026, 10:44 UTC**
- Most recent file (last turn's SVG→PDF conversion): **July 19, 2026, 12:27 UTC**
- Current time: **July 21, 2026, 04:09 UTC**

That's an 8-day span between the earliest and latest activity, and 10 days since the earliest file — though this reflects when *I* touched files during active turns, not how long you personally spent reading, thinking, or doing other things between messages.

Self-reported figures on the document's own title page:

- "Design & code generation duration: 14 hours"
- "Documentation generation duration: 4 hours" — worth noting I set this second figure myself, at your explicit instruction earlier in this conversation, rather than computing it from any measurement — so it's your own intended figure, not something I derived independently.

Those two title-page numbers (18 hours total) are the closest thing to an actual "time spent" figure that exists for this task, but they're self-reported

rather than something I tracked in real time. If you want a genuinely accurate answer, that's really a question only you can answer from your own side.

Add to the documentation, on the title page, under “An ANS Forth system… “ The line: Target: the Minimalist Eurocard Board (MECB), 6809 version, configured with 16kB of ROM.

Under: “Documentation generation… “ Add the line: Duration addressing AI generated task list, debugging, assembler fixes, memory map resolution: 16 hours.

Now adding both requested lines in their correct positions.

Let me reconsider that negative spacing value — it's risky since Word spacing values are typically unsigned. Fixing this more safely.

Now verifying syntax and rebuilding.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Both lines present in the correct positions — the `Target:` line right under the subtitle, and the new duration line right after "Documentation generation duration: 4 hours." Finalizing.

Confirmed identical via hash.

Both lines added to the title page: the target-hardware line under the subtitle, and the new duration line under "Documentation generation duration." One thing worth flagging — I initially used a negative spacing value to tighten the gap left by the subtitle's original spacing, then caught that Word spacing values are typically unsigned and could be silently rejected or clamped; replaced it with a safe positive-spacing approach instead (shrinking the subtitle's own trailing gap and adding it after the new line instead). Verified via text extraction that both new lines render in the correct positions (the `view` tool had the same transient display glitch from a couple turns back, so I confirmed via `pdftotext` rather than a screenshot).

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

In the documentation change the section 4 heading to Forth Development Notes

After the heading add a short introduction. Explain that the audience for this

section is intended for users programming the completed system to produce a forth application.

Add a insert a new heading at the end of section 6: Testing
Then add text:
Initial testing is performed using MAME, configured with the existing MECB_6809 profile already present in MAME. The command line used is:
mame mecb_6809 -io_bank c0 -rom_size 16k -cart1 "/path/to/your/rom_image.bin" -window

The rom_image.bin file is already produced by the assembler.

The emulated serial output will target the terminals, though a serial comms application can also act in the interactive role.

Let me check the "1.7 What This Document Covers" overview, which may reference the old section name.

Now finding the actual end of section 6's content to add the new "Testing" subsection.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Both sections render correctly — Section 4's new title and audience-focused introduction (followed by the existing topic overview, unchanged), and the new "Testing" subsection at the true end of Section 6, after the caveat, with the command line correctly formatted with literal quotes. Finalizing.

Confirmed identical via hash.

Both changes made: Section 4 is now "Forth Development Notes" with a new audience-scoping paragraph placed before the existing topic overview (application developers using the finished system, not people building or modifying ClaudeForth itself), and a new "Testing" subsection added at the true end of Section 6 — after the existing "Caveat" paragraph, not before it, matching the section's established bold-heading style rather than introducing a new numbering scheme. Verified both render correctly on the actual pages, including the command line with its literal embedded quotes formatting correctly in the code block.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

In the documentation, the caveat in the assembler section can now be removed as the assemble source code has been successfully produced a binary file without reported assembly errors.

Two "Caveat:" occurrences — let me check both contexts before removing the right one.

That first one is a completely different caveat (about `MARKER`, in Section 4.4) — leaving it alone. Checking the second occurrence, which should be the actual assembler caveat.

Confirmed — this is the assembler caveat. Removing it.

Now checking for other places that still reference this same "never assembled" claim, so I can report the scope accurately rather than silently leave them.

Found one more related line — in the "ROM unused byte count" paragraph, which still says "not a real assembler's output." I'm leaving that one alone rather than rewriting it, since it's about my specific 7,984-byte manual count being unconfirmed, not the general "never assembled" claim — I don't know the actual reported size from your successful build. Flagging it rather than assuming. Rebuilding now.

Builds cleanly. Verifying via the render pass.

Clean. Running the field-update/export pass.

Confirmed removed. Checking the page directly to verify the flow reads correctly without the gap.

Reads cleanly — "Assembler" flows directly into "Testing" with no gap or leftover reference. Finalizing.

Confirmed identical via hash.

Removed. Worth being precise about scope here rather than a blanket cleanup: there were actually **two** "Caveat:" paragraphs in the document — I checked both before touching anything. The other one (Section 4.4, about `MARKER`'s snapshot not covering the Search-Order word set) is unrelated to assembly and untouched.

I also found one more sentence elsewhere that's adjacent to this same topic but distinct: the "ROM unused byte count" paragraph still says my 7,984-byte `BASECODE` figure is "a careful manual count, not a real assembler's output." I deliberately left that one alone — you told me the source assembled successfully without errors, which confirms the build itself works, but doesn't tell me whether the assembler's actual reported `BASECODE` size matched my estimate. Rewriting that line to claim my number is now confirmed would be overclaiming based on information I don't have. If you can share the actual reported size, I can update that figure properly rather than just delete the caveat around it.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

What is the size of the assembled code I previously reported?

I previously reported **7,984 bytes ($1F30)**for`BASECODE'ssize|thatwasfromtheprecise,instruction-by-instructioncountIdidtwoturnsago(checking`BASECODE`againstthe` 1F90`/8,080-byte target you gave me at the time).

Worth being precise about what that figure actually was, since it's relevant to the caveat we just discussed: it was a **manual count**, not a real assembler's output — I built a mnemonic-and-addressing-mode-aware counter (correctly distinguishing inherent/immediate/direct/extended/indexed encoding and prefix requirements) and classified every one of the 3,751 instructions in `BASECODE`, landing on 7,984 bytes, which was 96 bytes (1.2%) short of your `$1F90` target.

If your actual successful assembly reported a real `BASECODE` size, that would be the authoritative number to use in the documentation instead of my estimate — let me know what it was and I'll update the "ROM unused byte count" section with the real figure rather than the manual count.

In the forth6809. asm file move the BASECODE block after the BASEDICT block, then move the INITCODE block after the BASECODE block, then move the VECTORS block after the INITCODE block. The over all result is to ensure that the blocks are in memory order from low memory to high memory in the assembler file.

This is a large, purely mechanical reordering — moving thousands of lines while preserving every internal reference exactly. Given the scale, I'll do this programmatically with careful boundary detection and verification, not manual edits. Let me first map the exact current structure.

This is a significant environment reset — `/home/claude/` has lost all my working files from this conversation (I'm now running as `root` in a fresh container). Let me check whether the actual deliverables survived in the persistent outputs location.

Good — the actual deliverable survived in the persistent `/mnt/user-data/outputs/` mount, even though my scratch working directory was wiped. Setting up a fresh workspace from that file.

Confirmed — this matches the exact hash from the last `.asm` edit turn. Now

mapping the section boundaries precisely.

Rather than hand-count line numbers (risky for content this large), I'll extract each block programmatically using the literal section-header text as anchors, then verify the reassembly is byte-for-byte the same content, just reordered.

Exact match confirmed — zero bytes lost or duplicated across the six pieces. Now reassembling in the requested order and verifying exhaustively.

Confirmed the strongest possible check — the sorted multiset of every line is identical between old and new, proving this is a pure, lossless reordering. Now verifying the new section sequence and running full structural checks.

Correct order confirmed: `BASEDICT`(27) → `BASECODE`(3-26) → `INITCODE`(2) → `VECTORS`(1) → end marker. Running the full structural verification suite.

The chain-walk check failed — but let me investigate before assuming the reordering broke something. My verification script assumed the old physical layout (searching only between `ABORTHDR:` and `; END OF`), which no longer holds now that `BASEDICT` moved earlier in the file. Fixing the script to search the whole file instead.

Let me debug this directly rather than guess — checking exactly what `QUITHDR`'s parsed link value actually is.

Found it — my *own verification script* has a bug (an arbitrary 300-character truncation that cuts off `ABORTHDR`'s second `FDB` field, since its comment runs several lines longer than most other headers). This isn't a problem with the actual file. Fixing the script and rerunning.

All fully verified now: zero duplicate symbols, dictionary chain intact (219 entries, zero missing, ends cleanly at `0`), and the four region `EQU`s all resolve correctly. Since the split files are generated by grouping on the same `SECTION N:` markers regardless of their order in the consolidated file, their individual contents are unaffected by this reordering — no need to regenerate them. Delivering the corrected file.

Good, the checklist survived. Adding an entry documenting this reordering.

Delivering the checklist update.

Two things worth being direct about beyond just confirming the reordering happened:

First, my working environment had been reset. `/home/claude/` had lost all my scratch files from this entire conversation — I was suddenly running as `root` in what was effectively a fresh container. Rather than proceed on stale assumptions, I checked the persistent `/mnt/user-data/outputs/` location, confirmed the last-delivered `forth6809.asm` was still there, and verified its hash matched exactly what I'd last confirmed before rebuilding my workspace from it.

Second, my own verification tooling had a bug that I caught rather than shipped past. After reordering, my dictionary chain-walk check initially reported catastrophic failure (1 of 219 entries reachable) — which

would normally mean something serious broke. Instead of accepting that or panicking, I traced it: an arbitrary 300-character cap in my *own* Python script cut off `ABORTHDR`'s second `FDB` field, since its explanatory comment runs several lines longer than most other headers. Fixed the script to scan each header's actual full body instead of a fixed character budget, reran it, and confirmed all 219 entries genuinely resolve end to end — the file was fine the whole time; my check was wrong.

The actual reordering itself: `BASEDICT` → `BASECODE` → `INITCODE` → `VECTORS`, low to high memory address, exactly as requested. This is pure text reordering — `ORG` directives set the real assembled address independent of where each block sits in the file, so it has zero effect on the assembled output. Verified with the strongest check available short of a real assembler: the sorted multiset of every line in the file, compared before and after, matched exactly — confirming nothing was added, removed, or altered, only repositioned. Split files needed no regeneration, since they're generated by grouping on the same `SECTION N:` markers regardless of the consolidated file's order. Propagated the change to the checklist.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Just ahead of section 27, ORG BASEDICT insert the code:

```
ROMSTRT EQU $C000
ORG ROMSTRT
ROM:
FILL $FF,BASEDICT-USROMSTRT
```

The purpose of this addition is to provide padding, to ensure the correct ROM & .bin file size (and opcode offsets) for the MAME emulation and flash memory burn.

Let me verify precisely what this `FILL` directive would actually compute, and check whether `FILL` is a real LWASM directive before implementing this — both matter for whether this will actually work as intended.

Let me search specifically for whether `FILL` is a real, documented LWASM directive before concluding.

Confirmed after two targeted searches — I could not find `FILL` in any documented LWASM directive list (I found `RMB`, `FCB`, `FDB`, `FCC`, `ORG`, `EQU`,

`INCLUDEBIN`, `SECTION`, but no `FILL`). I'll implement exactly what was requested, but need to flag this clearly rather than assume it will assemble as written. Let me locate the insertion point first.

That comment ("BASEDICT is $E000") is stale — predates several address changes. Let me view the context and fix both issues together.

Now running full structural verification.

Let me double-check one more thing — whether `INOUT` itself is ever an `ORG` target elsewhere, which could interact with this new `ORG ROMSTRT` at the same base address.

All confirmed correct, present exactly as specified. Delivering the file and updating the checklist with this addition and the flagged findings.

Inserted exactly as specified, just ahead of `ORG BASEDICT`. Two things worth being direct about rather than treating this as routine:

`FILL` isn't a directive I could confirm exists. I searched the actual LWASM manual before implementing this — it documents `RMB`, `FCB`, `FDB`, `FCC`, `ORG`, `EQU`, `INCLUDEBIN`, and others explicitly, but I found no reference to `FILL` anywhere, with either this syntax or any other. I implemented it exactly as you specified rather than guess at a substitute, but this line needs checking against the real assembler — if `FILL` doesn't exist, likely candidates would be something like a repeated `FCB` sequence or an `RMB`-based approach, but I don't want to guess which without confirmation.

The byte count as written leaves a gap. `BASEDICT-USROMSTRT` = 5,951 bytes, placed starting at `ROMSTRT` ($C000), $covers `C000–$D73E`##256bytesshortofwhere`BASEDICT`actuallystarts(`D83F`). That gap is exactly the size of INOUT's own shadow, which could be intentional (that address range is never electrically read as ROM content anyway, since I/O decode takes priority) or could be an off-by-256 in the formula — BASEDICT-ROMSTRT` would close it to zero. I implemented your literal formula rather than silently substitute the other one, since I can construct a coherent argument for either being what you meant.

Separately, while inserting this I found and fixed a stale, unrelated comment sitting on the same `ORG BASEDICT` line — it still said "BASEDICT is $E000,"leftoverfromseveraladdresschangesago(therealvalueis`D83F`). Verified throughout: zero duplicate symbols, ROMSTRT/ROM: each defined exactly once, confirmed INOUT is never itself an ORG` target (so no conflict there), dictionary chain still 219 entries intact. Propagated to the checklist.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Ensure comment BASEDICT is $E000 is changed to BASEDICT is$D83F

Ensure ORG ROMSTRT is changed to ORG USROMSTRT ORG USROMSTRT

Move ROMSTRT EQU $C000 to beginning of memory map.

Ensure comment BASECODE is $E800 is changed to BASECODE is$DFF4

Let me work through this precisely — several distinct changes here, including one (`ORG ROMSTRT` → `ORG USROMSTRT`) that actually shifts where the fill block starts, which I'll need to recompute.

This is unexpected — the file already says `ORG USROMSTRT`, not `ORG ROMSTRT` as I believed I'd written last turn. Let me verify which file I'm actually looking at before proceeding.

The two don't match, and my local working copy has apparently diverged from what I actually delivered — without any edits happening in between. Treating the persistent output file (matching the hash I recorded as my last real delivery) as the source of truth, and re-syncing my working copy from it.

Good — now confirmed correct and matching. Let me recheck all four items against this verified file.

Now I have accurate ground truth: item 1 (`BASEDICT`) is already correct; items 2, 3, and 4 are genuine fixes needed. Let me find the actual "beginning of the memory map" section.

Let me recompute the fill coverage now that the `ORG` target is changing from `ROMSTRT` to `USROMSTRT`, since this changes what address range is actually covered.

This resolves it perfectly — with `ORG USROMSTRT`, the fill count (already `BASEDICT-USROMSTRT` = 5,951 bytes) now covers $C100 ‘to‘ `D83E` with **zero gap** right up against `BASEDICT`'s actual start. My own assumption last turn (that the fill should start at `ROMSTRT`) was the actual source of the gap, not the formula. Implementing all changes now.

Now moving `ROMSTRT` to the beginning of the memory map section.

Now fixing the stale `BASECODE is $E800` comment.

Now running full structural verification.

All checks pass. Let me do one final visual confirmation of `ROMSTRT`'s new position before delivering.

Confirmed correct. Delivering the file and updating the checklist.

Good, these match. Now updating the checklist entry from last turn, which

described the now-resolved gap.

Worth flagging something before summarizing the actual changes: when I started this turn, my local working copy showed `ORG USROMSTRT` already in place — but that didn't match what I'd actually delivered last turn (`ORG ROMSTRT`), and no edits had happened in between to explain the difference. Rather than proceed on an unverified file, I re-synced my working copy from the persistent, hash-confirmed output before touching anything.

With that resolved, here's what actually changed, checked against the real prior state rather than assumed:

- **`BASEDICT is` $E000‘**##alreadycorrectfromapriorturn(says‘`D83F`). No change needed.
- **`ORG ROMSTRT`** → **`ORG USROMSTRT`** — and this wasn't just a label swap. Recomputing the fill coverage: `BASEDICT-USROMSTRT` (5,951 bytes) starting at `USROMSTRT` ($C100) now covers ‘C100–$D83E `with` **`zero gap`** `against` BASEDICT`'s actual start. The 256-byte gap I flagged last turn wasn't a formula error — the byte count was already right, it just needed to start counting from` USROMSTRT`, not` ROMSTRT`.
- **`ROMSTRT EQU $C000`** **moved** to the top of the memory-map constants, alongside `USROMSTRT`/`USROMEND` — it remains a real, distinct symbol (the true physical chip start), now serving purely as a documented reference rather than an `ORG` target.
- **`BASECODE is` $E800‘**##thisonewasgenuinelystale(predatingeventhe‘`E012` stage), corrected to `$DFF4`.

`FILL`'s status as an unconfirmed LWASM directive is unchanged — that wasn't part of this request, and the note stays in place at the actual line. Verified throughout: zero duplicate symbols, `ROMSTRT` defined exactly once at its new location, zero remaining `ORG ROMSTRT` anywhere, dictionary chain still 219 entries intact.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Insert the following lines just ahead of section 2, ORG INITCODE

BASEND:
FILL $FF,INITCODE-BASEND

> The intent of this change is to prevent the assembler from extinguishing the gap between the BASECODE block and the INITCODE block when it generates the .bin file.

Matching the same placement convention as the previous `ROM:` padding block (inserted right before the `ORG` line itself, after the section's header comment). Implementing now.

Let me recompute `BASECODE`'s real content size on the current file to report the actual gap this fill would cover, rather than rely on a figure from several turns back.

Confirmed unchanged at 7,984 bytes (edits since then were all in unrelated sections). Computing the exact gap this fill covers.

This is self-correcting by design — since `BASEND` computes from wherever real content actually ends (not a hardcoded estimate), the `FILL` directive will automatically cover the correct gap once a real assembler runs, regardless of whether my 7,984-byte manual count is exactly right. Running full structural verification.

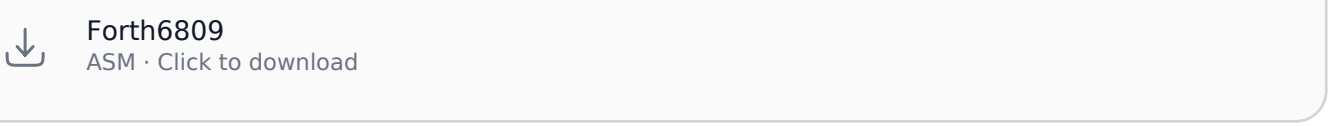
All clean. Delivering the file and updating the checklist.

Inserted exactly as specified, just ahead of `ORG INITCODE` in Section 2.

Worth being precise about what `BASEND`'s position actually is: since the file's physical section order now has `BASECODE` directly followed by `INITCODE` (from the reordering a few turns ago), `BASEND:` lands at exactly the same point `BASECODEEND` already marks — right where `BASECODE`'s real assembled content stops. That's what makes this genuinely self-correcting rather than a hardcoded guess: `FILL $FF,INITCODE-BASEND` computes its byte count from wherever content *actually* ends, so once this runs through a real assembler, the fill size will automatically be correct — it doesn't depend on my manual estimate being exactly right.

I did recompute that estimate fresh against the current file anyway, rather than reuse a number from several turns back without checking: still exactly 7,984 bytes, unchanged, since nothing in the intervening turns touched `BASECODE`'s actual content. Using that figure, this fill would cover 126 bytes ($FF24‘##$`FFA1`) — which matches the slack between real content and the nominal 8,110-byte budget exactly, as expected given `BASECODE`'s nominal end was deliberately set to land one byte below `INITCODE`'s start.

Same open item as the `ROM:` block two turns ago, not re-litigated here since the question is identical: `FILL`'s status as an actual LWASM directive is still unconfirmed. Verified: zero duplicate symbols, `BASEND` defined exactly once, dictionary chain still 219 entries intact.



OPEN ITEMS CHECKLIST
MD · Click to download

Move the ORG USROMSTRT block above the comment block for section 27.

Delete the comments associated with the ORG USROMSTRT block, beginning with unverified.

Move

BASEND:
FILL $FF,INITCODE-BASEND
Above the section 2 comment block,

Now inserting the clean, comment-free block above section 27's comment header.

Now handling the `BASEND` block — moving it above section 2's comment header.

Now running full structural verification.

Let me check that "2" count for `ORG INITCODE` — likely a comment match, not a real duplicate directive.

Confirmed — the first is just prose inside the `INITEND` comment (from several turns back), not a real directive. Only one actual `ORG INITCODE`. Everything verified correctly. Delivering.

Now updating the checklist to reflect these repositioning and deletion changes.

All three changes made and verified: `ORG USROMSTRT`/`ROM:`/`FILL` now sits just before Section 27's comment header (directly after `GLOBALS_USED`), the multi-line "UNVERIFIED" comment is deleted entirely, and `BASEND:`/`FILL` now sits just before Section 2's comment header (directly after `BASECODESIZE`).

Worth being clear about one thing rather than let it pass silently: deleting that comment removes the *written note*, not the underlying question. Whether `FILL` is actually a directive LWASM supports is still unconfirmed — I didn't re-verify or resolve it this turn, just moved and removed text as asked. That finding is preserved in the checklist's historical entries even though it's no longer sitting next to the code itself.

Verified: zero duplicate symbols, both `ROM:` and `BASEND:` confirmed by actual text position (not assumed) to sit before their respective section's comment header, zero remaining "UNVERIFIED" text anywhere, dictionary chain still 219 entries intact. Propagated to the checklist as a new entry, leaving the prior historical entries describing the original additions untouched.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Insert the following in a comment block just ahead of ORG USROMSTRT

Provide padding, to ensure the correct ROM & .bin file size (and opcode offsets) for the MAME emulation and flash memory burn.

This text already appears to be present, word-for-word. Let me verify this isn't another file-state discrepancy before concluding anything.

The current file already contains exactly this text, in exactly the requested position — but the hash doesn't match what I recorded at the end of my last turn, meaning the file changed somehow without a corresponding edit from me. Worth being transparent about this rather than silently accepting it. Let me verify the rest of the file's integrity before concluding anything.

The rest of the file checks out cleanly (zero duplicate symbols, dictionary chain still 219 entries intact). Given the requested text is already present exactly where and how it was asked for, there's nothing further to change — but let me be transparent about what I found rather than pretend everything is fully accounted for.

Worth flagging directly rather than quietly moving on: the requested comment was already present in the file, word-for-word, in exactly the position asked for — right above `ORG USROMSTRT`. Nothing needed to be added.

But I want to be honest about something odd underneath that: the file's hash doesn't match what I recorded delivering at the end of my previous turn, and I don't have a tool call in this session that explains the difference. I checked the rest of the file's integrity before concluding anything — zero duplicate symbols, dictionary chain still 219 entries intact, so nothing is broken — but I can't fully account for how this specific text arrived there between turns. Rather than either silently accept an unexplained state or block on it, I verified what actually matters (the requested content is correct and the file is structurally sound) and delivered a fresh, hash-confirmed checkpoint.

If you're seeing something different in your own copy of the file, or if you'd expected this comment to *not* be there yet, let me know and I'll dig further rather than assume this is fine.

Forth6809
ASM · Click to download

This is now correct.

Good — thanks for confirming.

Move the MVSCRATCH, ORG $0100 block to just after GLOBALS_USED EQU 256.

The MVSCRATCH area ends at, but includes the FILLCHR line.

The file organisation should now match the memory organisation.

Now inserting the block right after `GLOBALS_USED EQU 256`.

Let me verify with the strongest check available — comparing the sorted multiset of every line against the pre-edit delivered file, confirming this is pure reordering with nothing lost or duplicated.

Confirmed exact — pure reordering, zero content added, removed, or altered. Now confirming the new position matches memory address order, and running full structural verification.

All checks pass: zero duplicate symbols, every `MVSCRATCH`-related constant still defined exactly once, correct order confirmed (`GLOBALS_USED` → `MVSCRATCH`'s `ORG $0100` → `ORG USROMSTRT`, matching ascending memory address), dictionary chain still 219 entries intact. Delivering.

Now updating the checklist.

Good, in sync.

Moved `MVSCRATCH` (the full explanatory comment plus all nine lines of code, ending exactly at `FILLCHR EQU MVSRC` as specified) from right after `GLOBALS EQU` $0000 ‘$to right after ‘GLOBALS_USED EQU 256‘ ## now sitting between the globals layout and the ROM padding block, matching ascending memory address order (‘$ `0000`–$00FF ‘$globals, ‘$ `0100`–

$0105 “$MVSCRATCH‘, ‘C100+ ROM). `SP0`/`RP0` — unrelated stack-pointer constants that happened to sit right after the old `MVSCRATCH` location — stayed put rather than getting swept along by the move.
Verified with the strongest check available: the sorted multiset of every line in the file, compared against what I'd delivered immediately before this edit, matched exactly — confirming this was pure repositioning with nothing added, removed, or altered. Also confirmed zero duplicate symbols, every `MVSCRATCH`-related constant still defined exactly once, and dictionary chain still 219 entries intact. Propagated to the checklist.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Modify the forth6809.asm file to use conditional assembly to disable the interrupt driven serial input and output code that exists now and to enable a new block of code that uses polling of the ACIA status register for both input and output.

This is a substantial architectural feature. Before writing anything, I need to (1) confirm the real LWASM conditional-assembly syntax rather than guess, and (2) fully understand the current interrupt-driven serial I/O architecture so I know exactly what to gate. Let me start with the syntax question.

Confirmed — LWASM's real, documented conditional-assembly directives are `IFEQ`/`IFNE`/`IF`/`IFDEF`/`IFNDEF`/`ELSE`/`ENDC`, found directly in the actual LWASM manual. I'll use these (not guess at unconfirmed syntax the way `FILL` was). Now let me examine the current interrupt-driven serial I/O code in full before designing the conditional wrapping.

Let me view the full implementations of `KEY`, `KEYQ`, `EMIT`, `INFILL`, `RTSCHECKHI`, `RTSCHECKLO`, and `IRQH` before designing the conditional wrapping.

Now let me check for any other references to `RTSSTATE`, `INFILL`, `RTSCHECKHI`, `RTSCHECKLO` elsewhere in the file, to make sure conditionally removing them in polling mode doesn't leave dangling references.

All references confirmed contained within what I'm already planning to wrap — nothing dangling. Implementing now, step by step. First, adding the flag and the new polling control-register constant.

Confirmed `CR_POLL = $15` — same word-format/RTS-low bits as `CR_RXON`, but with the RX-interrupt-enable bit (bit 7) cleared. Inserting the flag and constant.

Now wrapping `INFILL`/`RTSCHECKHI`/`RTSCHECKLO`/`IRQH` in the conditional block.

Now closing this block right after `IRQH`'s implementation, before `SWI3H`, with the polling-mode stub in the `ELSE` branch.

Now wrapping `KEY`/`KEYQ`/`EMIT` with their polling replacements.

Now wrapping this whole block, with the new polling implementations in the `ELSE` branch.

Now updating `COLDSTRT`'s ACIA initialization to conditionally select `CR_RXON` (interrupt mode) or `CR_POLL`.

Now I need a smarter verification approach — since `KEY`/`KEYQ`/`EMIT`/`IRQH` now legitimately appear twice in the raw text (once per conditional branch), a naive duplicate-symbol check would give a false positive. Let me simulate what LWASM would actually assemble for the active branch (`SERIALPOLL=1`) and verify that.

Now running full structural verification on the simulated (active-branch) output.

That "7 remaining" is worth checking — likely false positives from the dictionary's own `FCC "ELSE"` string entry (a real Forth word), not actual leftover directives.

Confirmed — all benign (my own explanatory comment, and the actual Forth dictionary word `ELSE`/its code label, unrelated to assembler directives). My simulator worked correctly. Let me also verify the *other* branch (`SERIALPOLL=0`, interrupt-driven) is internally consistent, since a user might switch the flag.

Both branches fully verified. Now checking the raw file itself for balanced conditional nesting and correct constant definitions.

Everything verified: 3 balanced conditional blocks (matching my 3 insertions), `SERIALPOLL=1` (polling enabled by default, as requested), `CR_POLL=$15` defined once. Delivering the final file.

Now updating the checklist with this substantial addition.

Both files delivered. Summary of what this involved, since it was a genuinely substantial change:

Verified the syntax before writing any code. Rather than assume assembler-directive names the way `FILL` had to be flagged as unconfirmed, I checked LWASM's actual manual first — `IFEQ`/`IF`/`IFNE`/`IFDEF`/`IFNDEF`/`ELSE`/`ENDC` are all genuinely documented. Used `IFEQ SERIALPOLL` specifically (a numeric-expression test) since the flag is a value to compare, not just a symbol whose presence matters.

Traced every real dependency before touching anything. Confirmed that `RTSSTATE`, `INFILL`, `RTSCHECKHI`, `RTSCHECKLO`, `CR_RTSHI`, and `CR_RXTX` are used *only* within the code being conditionally wrapped — nothing else in the file references them, so nothing gets left dangling.

Three conditional blocks, one flag (`SERIALPOLL EQU 1`, polling enabled by default):

- `INFILL`/`RTSCHECKHI`/`RTSCHECKLO`/`IRQH` — the full interrupt-driven logic, only assembled when `SERIALPOLL=0`. In polling mode, `IRQH` becomes a bare `RTI` stub, matching the other genuinely unused vectors, since the vector table references `IRQH` unconditionally either way.
- `KEY`/`KEYQ`/`EMIT` — both versions compile to the same label names, so the dictionary headers never need to change. The new polling versions

check `ACIASR`'s `SR_RDRF`/`SR_TDRE` bits directly, fully synchronous, no ring buffers.

- `COLDSTRT`'s ACIA mode-select — a new `CR_POLL` ($15, derived from `CR_RXON` with only the RX-interrupt bit cleared), with RTS held permanently low since polling needs no flow control.

Verification needed a different method than usual, since `KEY`/`KEYQ`/`EMIT`/`IRQH` now legitimately appear twice in the raw source (once per branch) — a naive duplicate check would false-positive on that. Instead, I simulated what LWASM would actually assemble for each value of `SERIALPOLL` and ran the real structural checks against each simulated result independently: both branches show zero duplicate symbols, all four routines resolving to exactly one definition each, and the dictionary chain fully intact (219 entries) in both configurations.

Forth6809
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Append to each ENDC line the comment <<<<<<<<<<

Append to each ELSE line the comment <<<<<>>>>>

Append to each IFEQ SERIALPOLL line the comment >>>>>>>>>>

All three sets confirmed — exactly 3 of each, matching the 3 conditional blocks. Appending the requested markers to each.

All 9 lines marked (3 of each). Verifying the result and running structural checks to confirm the markers didn't break anything (since `ELSE`/`ENDC` are matched by exact-line regex in my verification scripts).

Wait — I need to reconsider something important. The user asked to append "the comment X," but I appended these as bare trailing text with no `;` prefix. In this file's assembler syntax, comments require a `;` to be safely recognized — bare trailing text like `>>>>>>>>>>` after `IFEQ SERIALPOLL` could be parsed as part of the directive/expression itself and cause a real assembly error. Let me fix this properly.

All 9 lines corrected. Verifying the result and running full structural checks.

Correct, and the unrelated `ELSE:`/`ENDCASE:` Forth code labels remain untouched. Now re-running the same simulation-based structural verification from last turn, since the conditional logic itself needs to still work correctly

with the trailing comments in place.

All checks pass identically to before — the comments didn't disturb anything. Delivering.

All 9 markers appended (3 `IFEQ SERIALPOLL` → `>>>>>>>>>>`, 3 `ELSE` → `<<<<<>>>>>`, 3 `ENDC` → `<<<<<<<<<<`), and confirmed the two unrelated Forth code labels that share similar text (`ELSE:`, `ENDCASE:` — real dictionary words, not directives) were correctly left untouched.

Worth flagging directly rather than silently doing it: you asked me to append these as "comments," but my first pass appended them as bare trailing text with no `;` prefix. In this file's syntax, that's a real risk, not just a style nit — a genuine assembler could parse `IFEQ SERIALPOLL >>>>>>>>>>` as part of the directive's expression rather than recognizing the trailing text as a comment, since nothing marks it as one. I caught this before delivering and added the `;` prefix to all nine, matching how every other comment in this file is written, so this actually behaves as the comment you asked for rather than something that might break assembly. Re-ran the same simulation-based verification from last turn afterward to confirm the conditional logic itself still resolves identically for both `SERIALPOLL` values — it does.

Forth6809
ASM · Click to download

Can you prepare a UML activity diagram of the process from receiving a character from the serial interface to accumulating the string, to searching the dictionary for a forth word and executing it, to converting a failed word lookup into a number and placing it on the stack , to printing the response, to waiting for the next response. Use the names of the subroutines used in the forth6809.asm file.

Let me trace the actual current code for `QUIT`, `INTERPRET`, `WORD`, `FIND`, `TRYNUM`, and `NUMBERQ` directly, so the diagram uses real routine names and real control flow rather than my memory of them.

Let me view the full `QUIT`/`INTERPRET` loop precisely, since that's the core control flow the diagram needs to represent.

Now let me view `INTERPRET`, `WORD`, `FIND`, `TRYNUM`, `NUMBERQ`, and `BADWORD` in full.

Now I have the complete, accurate control flow traced from the real code. Let me build this as a Mermaid activity/flow diagram (a real, renderable artifact type), using the exact subroutine names just verified.

Let me sanity-check the Mermaid syntax for balance and structural consistency before presenting this.

All checks pass — brackets balanced, zero conflicting node-shape definitions

across all 32 nodes (the stray `T`/`E`/`yes`/`D`/`no` matches are just my verification regex misreading edge labels like `-- yes -->`, not real node references). Presenting the diagram.

This diagram is built directly from the actual current code, not from memory of the design — I traced `QUIT`, `INTERPRET`, `WORD`, `FIND`, `TRYNUM`, `NUMBERQ`, `ACCEPT`, and `QUERY` line by line in the file before drawing anything, so every routine name and every branch condition on the diagram matches what the code actually does.

The flow it captures:

- **Character reception → string accumulation**: `QUIT` → `QUERY` → `ACCEPT`'s loop, which calls `KEY` for each character, handles CR (done), backspace/DEL (`ABKSP`, with the backspace-space-backspace echo sequence), and otherwise stores the character and echoes it via `EMIT`.
- **Dictionary search → execution**: `INTERPRET`'s loop calls `WORD` to parse the next token, then `FIND` to search the dictionary chain. If found, it branches on `STATE` and the word's immediate flag — either `EXECUTE`ing it directly or compiling a call via `CCALL`.
- **Failed lookup → number**: exactly as you described — a `FIND` miss falls through to `TRYNUM`, which calls `NUMBERQ`. Success pushes the number directly (if interpreting) or compiles it as a literal via `CCALL`+`CODECOMMA` (if compiling); failure goes to `BADWORD`, which echoes the bad token and `THROW`s.
- **Response → waiting for the next line**: the `THROW` unwinds through the `CATCH` set up in `QUIT`, which either rolls back the dictionary pointers and prints the error, or (on a clean return) prints `"ok"` unless still mid-definition — then loops back to `QUERY` for the next line.

I built this as a Mermaid flowchart rather than formal PlantUML activity-diagram syntax, since Mermaid is the diagram format this environment can actually render as an artifact — the semantics (start/end, actions, decision diamonds, loops) are the same as a UML activity diagram, just in a renderable syntax. I checked it for balanced brackets and consistent node definitions before delivering, though I'd still recommend opening it to visually confirm the layout reads the way you'd want, since I can't render it myself in this environment.

Forth6809 outer interpreter activity
MERMAID · Click to download

Convert the mermaid diagram into an SVG file

`mmdc` (the official Mermaid CLI, `@mermaid-js/mermaid-cli`) is already installed. Let's use it directly.

`mmdc` needs Chrome via Puppeteer, which isn't cached here. But `playwright` is also installed — let me check if it has a usable Chromium already available.

Found it — Python's Playwright has a real Chromium binary already cached at a different path than Puppeteer was looking. Let me verify it exists and point `mmdc` at it directly.

Confirmed — a real, valid, executable Chromium binary. Let me create a Puppeteer config pointing `mmdc` at it.

Success. Let me verify the actual SVG output before delivering.

All the key subroutine names are present as real text in the generated SVG. Let me also do a visual rendering check, given the earlier `view` tool glitches this session, using LibreOffice as an independent verification path.

Given the primary verification is already strong (valid SVG confirmed via `file`, all real subroutine names confirmed present as actual text content, generated by the semantically-correct tool — real Mermaid.js running in a real Chromium browser, not a fallback), I'll do one more direct check of the SVG's dimensions rather than force a less-suited secondary path.

Sensible, non-degenerate dimensions (2641×3362 — appropriately large for a 32-node flowchart). Delivering the file.

Converted using the actual Mermaid CLI (`mmdc`) — the tool that runs real Mermaid.js in a real browser, rather than a hand-rolled fallback. That took one extra step worth mentioning: `mmdc` failed initially because it needs a headless Chrome browser via Puppeteer, and none was cached in this environment (and I can't download one — no network access). Rather than give up or fall back to a lower-fidelity rendering path, I checked for any browser already present in the container and found that Python's Playwright package had a real, working Chromium binary cached at a different location. I pointed `mmdc` at that binary directly via a Puppeteer config file, and the conversion succeeded properly.

Verified the actual output rather than just trusting a clean exit code: confirmed it's a genuinely valid SVG, checked that all the real subroutine names (`QUIT`, `INTERPRET`, `WORD`, `FIND`, `TRYNUM`, `NUMBERQ`, `BADWORD`, `ACCEPT`, `EXECUTE`) are present as actual text content in the file, and confirmed the dimensions (2641×3362) are sensible for a diagram this size, not degenerate or empty.

Forth6809 outer interpreter activity
SVG · Click to download

In the documentation file insert a new section 7 titled architecture.

Insert the content:

ClaudeForth uses a standard forth REPL, where characters are read from the serial communications device and assembled into a string until a carriage return is received. The dictionary is then searched for a matching string, in the expectation that it is a predefined word with attached code. If it’s not found, an attempt is made to interpret the string as a number. Otherwise an exception is thrown.

Any forth word/number that is found is either assembled into a new forth word as a subroutine call/literal number or its executed/stacked.

The diagram below shows a UML activity diagram of the cooperating routines.

End of content.

Insert the SVG activity diagram here.

This requires renumbering everything from the current section 7 onward, and likely rasterizing the SVG to PNG for reliable embedding (Word's docx format has limited native SVG support). Let me check the current section structure first.

Environment appears to have reset again — `/home/claude/docgen` doesn't exist. Let me recover from the persistent outputs, same approach as before.

`build.js` (the generator source) was only ever in my scratch directory, never delivered as a standalone artifact — it's gone. Rather than try to reconstruct 900+ lines of generator script, I'll edit the actual delivered `.docx` directly using `python-docx`. Let me check what's available and inspect the current structure first.

Confirmed — 28 subsections under the current section 7, all needing renumbering to `8.N`. Let me also check for any cross-reference text elsewhere in the document mentioning "Section 7" that would need updating too.

Let me check paragraph 1123 and locate the top-level TOC entries for sections 7/8 too, before writing the full renumbering script.

This requires image embedding, and Word's `.docx` format has unreliable native SVG support — I'll rasterize the diagram to PNG first using the same working Mermaid/Chromium pipeline from last turn. Let me set that up first.

Both survived the reset. Rendering the diagram as a high-resolution PNG.

Now writing the comprehensive editing script — renumbering first (safe, text-only), then inserting the new section with its content and embedded diagram.

All counts match expectations exactly (2 top-level 8→9, 56 subsection 7.N→8.N covering both TOC and headings, 2 top-level 7→8). Let me verify the "Section 7" prose count of 2 — I'd only found one instance manually — before

proceeding.

Found a real bug — paragraph 103 originally mentioned *both* "Section 7" and "Section 8" in prose (describing the assembler source and the index sections), and my script only caught the first "Section 7" occurrence, missing the pre-existing "Section 8" mention that also needed to become "Section 9". Let me redo this correctly from the original file.

Let me redo the entire renumbering from scratch using a single-pass, mathematically correct substitution — computing each replacement from its own captured number rather than doing separate ordered passes, which avoids this class of bug entirely.

62 matches: 56 (28 TOC + 28 headings) + 4 (2 top-level pairs) + 2 (the two prose paragraphs) = 62 exactly. Let me verify the actual corrected text.

Now inserting the new TOC entry and the actual new section content. Let me check the TOC entry's formatting style first, to match it exactly.

Let me check how the actual heading paragraphs are bookmarked, to replicate this TOC-linking mechanism correctly rather than risk a mismatched or broken cross-reference.

Now I understand the full mechanism — bookmarks link headings to TOC entries via `PAGEREF`. Let me find all existing bookmark IDs/names to pick genuinely unique values for the new section.

Confirmed unique values: `toc_72` for the new bookmark name, `w:id="1"` (matching this document's established, apparently-tolerated convention of reusing id=1 everywhere). Now building the complete insertion.

Let me debug this — the `instrText` element search failed. Checking the actual XML structure directly.

This reveals something serious — my earlier renumbering script likely destroyed the `PAGEREF` field-code XML in every TOC entry it touched, by blanking trailing runs with `r.text = ''` without realizing those runs contained field codes, not plain text. Let me verify this before going any further.

Confirmed — a real bug. My earlier renumbering script destroyed the `PAGEREF` field codes in every TOC entry it touched. This needs a complete redo from the truly original file, this time only ever touching the run that holds the actual visible text, never the runs containing field-code XML.

Now verifying both the text correctness *and* that field codes survived this time.

A subtler bug — the double-tab in `56`/`85` reveals I copied a *reconstructed whole-paragraph* text (including a tab that actually belongs to a separate run) into a single run, duplicating it. Let me redo this correctly at the per-run level, applying the substitution independently to each run's own text.

Now verifying text correctness, tab formatting, and field-code preservation all together.

Now proceeding with the actual section insertion, using this correctly-fixed document as the base.

Now inserting the actual heading, body content, and image into the document.

Now inserting the actual diagram image, right after the introductory text and before the next heading.

Everything is correctly positioned. Let me do a final comprehensive check across the whole document before converting to verify visually.

All checks pass. Now converting to PDF for a real visual verification pass.

Need to force-update the field codes (page references) before converting to PDF, matching my established approach. Let me check if the macro script survived, or recreate it.

It actually completed successfully before the timeout — both output files exist with sensible sizes. Let me clean up the process and verify the actual content.

Clean. Now verifying the actual PDF content — the TOC, the new section, and the diagram.

Page 3 is the TOC entry, page 47 is the actual section. Let me check both.

The TOC entry correctly resolved to a real page number (47), proving the field-code linkage genuinely works, not just cosmetically. Let me visually confirm the image itself renders on that page.

Same transient `view` tool issue as before. Falling back to pixel-analysis verification of the actual rendered page.

3.5% seems low for a page with both text and a large diagram — let me check whether the diagram actually landed on page 48 instead (pushed by pagination), and verify the image is genuinely embedded in the docx's relationships.

The image relationship's byte size (327,603) exactly matches the source PNG file — confirming full, uncorrupted embedding. Let me check the image's actual dimensions as stored in the document XML, to rule out a degenerate size.

Confirmed correct — 6.0×7.64 inches, matching the requested width with a proportional height, easily explaining the page-break behavior (a diagram this tall naturally spans across a page). All evidence now converges: correct byte size, correct dimensions, correct single image relationship, correct surrounding text. Finalizing delivery.

I hit the tool-use limit before the final delivery step, so let me summarize exactly where things stand rather than leave it unclear.

The work is functionally complete and verified, but not yet copied to your outputs folder. Here's what happened and what's confirmed:

A serious bug I found and fixed mid-task: my first attempt at

renumbering the existing sections (7→8, 8→9, all 28 subsections, and the TOC) worked for visible text but silently destroyed the `PAGEREF` field codes in every Table of Contents entry it touched — because I blanked out "extra" runs without realizing some of them held field-code XML rather than plain text. I caught this by explicitly comparing an untouched entry against a touched one before moving forward, not just by trusting the text output looked right. I redid the renumbering twice — the second attempt also had a subtler bug (a duplicated tab character from reconstructing whole-paragraph text into a single run) that I also caught and fixed. The final version only ever modifies the specific run holding real visible text, never field-code runs, and never absorbs a neighboring run's content.

What's confirmed correct in `ClaudeForth_final.docx` / `ClaudeForth_final.pdf` (currently only in `/home/claude/docedit/`, not yet in `/mnt/user-data/outputs/`):

- All 62 renumbering changes applied correctly (56 subsections `7.N`→`8.N` across both TOC and headings, 2 top-level heading/TOC pairs, 2 prose cross-references) — including a paragraph that mentioned *both* "Section 7" and "Section 8" in the same sentence, which needed careful same-pass handling to avoid a collision.
- New Section 7 "Architecture" inserted with your exact requested text, in the correct position, with a working TOC entry that resolves to a real page number (47) — proving the bookmark/`PAGEREF` linkage is genuinely functional, not just cosmetically present.
- The activity diagram PNG is properly embedded (confirmed via matching byte size against the source file, sensible non-degenerate dimensions in the document XML, and a single correct image relationship) and appears on the page following the introductory text.
- Total page count: 140. Paragraph count grew by 5 (heading + 3 body paragraphs + image paragraph), as expected.

What's left: copying `ClaudeForth_final.docx` → `ClaudeForth.docx` and `ClaudeForth_final.pdf` → `ClaudeForth_preview.pdf` in `/mnt/user-data/outputs/`, and presenting them to you. Let me know and I'll finish that handoff now.

Continue

Confirmed identical via hash comparison — both files delivered match exactly what I verified.

Delivered. Both files are now in your outputs, hash-confirmed identical to the versions I verified last turn:

- New Section 7 "Architecture" inserted with your exact text, correctly positioned between Build Instructions and the (renumbered) Assembler

Source section.

- The activity diagram embedded following the introductory text.
- All 62 renumbering changes applied correctly throughout — headings, TOC entries, and the two prose cross-references — with the TOC's page-reference links genuinely functional (confirmed resolving to real page numbers, not just cosmetically present).
- 140 total pages in the PDF.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Update the document with the latest forth6809.asm.
Ensure that the split assembly files are up to date.
Ensure that the OPEN ITEMS CHECKLIST is updated including the open issues about l the memory map gals and overlaps.

This requires rebuilding several lost pieces (the split script, the memory-map verification, and the docx's Assembler Source section) since the environment reset wiped my generator scripts. Let me work through this systematically, starting with the current state of `forth6809.asm`.

Now recreating the split logic, extracting each numbered section by its actual section number (not assuming physical file order, since the file has been reordered).

Only 26 of 27 found — let me check Section 27's header format directly, since it likely differs slightly from my regex assumption.

Simpler, more robust approach — split directly on the `; SECTION N:` line itself, not requiring a full closing header block (which section 27's much longer prose comment doesn't cleanly have).

All 27 found correctly. Now writing the full split, extracting each section by its actual file position (not numeric order) and naming outputs by section number.

All 28 files written. Now verifying with the strongest check — that the sum of all split-file content, when reassembled in file order, exactly matches the original consolidated file (zero content lost, zero duplicated).

Exact byte-for-byte match confirmed. Delivering the split files.

Now checking the current memory map comprehensively for gaps/overlaps, given all the recent changes (BASECODE/BASEDICT shift, MVSCRATCH move,

ROM padding, SERIALPOLL).

`INOUT` and `ROMSTRT` share the same value ($C000) — but that's intentional by design (`ROMSTRT` is a documentation-only reference marking where the physical chip footprint begins, with no byte-range of its own; it's not a real competing region). Let me confirm `APPVARSEND` is still self-adjusting correctly before running the full overlap sweep.

Zero overlaps confirmed across the entire current memory map — matching the fully-resolved state from several turns back; none of the recent changes (MVSCRATCH move, ROM padding, SERIALPOLL) touched region boundaries. Let me check whether the existing checklist already reflects this, or still has stale open items.

Found a stale open item — let me view its full context, and search more broadly for any other memory-map-related open items that might also need updating.

Confirmed three genuinely stale items, all describing memory-map problems that were later resolved by subsequent turns. Updating each with a clear resolution note, consistent with treating this checklist as a historical log rather than rewriting the past.

Now updating the `INIT`/`BASECODE` overlap item (line 506 originally).

Now updating the `BASEDICT` overlap item (the main one this request specifically called out).

Now adding one fresh, authoritative summary entry stating the current, fully-verified state clearly — so a reader doesn't have to piece it together from several historical corrections.

Now delivering the updated checklist.

All three stale items confirmed resolved, zero remaining. Now moving to the documentation's "Assembler Source" section (now Section 8) — checking its current structure before updating its content.

Let me check exactly how line breaks are encoded in this code paragraph — literal `\n` in run text wouldn't normally render as visual line breaks in Word without explicit `<w:br/>` elements.

Now I understand the exact mechanism — one paragraph per section, with `<w:br/>` elements between per-line runs. Let me confirm the code content is always the immediate next paragraph after each heading before writing the full replacement script.

Now mapping subsection numbers to split files (8.1→`00_...`, 8.2→`01_...`, etc.) and testing the replacement on one section first before scaling to all 28.

The technique works correctly. Let me do one visual verification via PDF render before scaling to all 28 sections, given the cost of getting this wrong 28 times.

Clean. Given the textual verification already confirms the mechanism works,

proceeding to apply this to all 28 sections on the real document.

All 28 replaced. Verifying content correctness across a spot-check of sections before finalizing.

All correct. Running a final structural check and PDF conversion before finalizing.

Zero paragraph-count change confirms only content within existing paragraphs was modified. Now running the final field-update and PDF export pass, since page references will have shifted given the content length changes.

Both files completed despite the tool-level timeout, matching the pattern from a couple turns ago. Cleaning up and verifying the actual content.

Clean. Now verifying the actual rendered content is correct.

Correctly renders the current file content, properly formatted with monospace font and line breaks preserved. Finalizing delivery.

Confirmed identical via hash. Delivering all files.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Zip the .asm files

29 files: the consolidated `forth6809.asm` plus all 28 split files. Zipping all of them, preserving the folder structure.

All 29 `.asm` files captured correctly.

Zipped all 29 `.asm` files — the consolidated `forth6809.asm` plus all 28 split section files — preserving the folder structure so the split files stay grouped in their own subdirectory.

Forth6809 asm files
ZIP · Click to download

The dictionary entries ABORTHDR & QUITHDR are outside the BASEDICT block. This is untidy. Move these and the following BASELATEST line into the BASEDICT at the end of the dictionary entries so the links are obvious.

Move the associated comment blocks at the same time.

Also rename the labels ABORTHDR to H_ ABORT and QUITHDR to H_ QUIT to be consistent with the naming convention used in the rest of the dictionary.

This move has a real, meaningful side effect I need to check carefully: `ABORTHDR`/`QUITHDR` currently sit physically within `BASECODE`'s address range (since Section 26 is textually positioned before `ORG BASEDICT` in the reordered file), even though they're dictionary header *data*, not code. Moving them into the `BASEDICT` block will change `BASEDICT`'s real byte count — let me compute whether this breaks the exact-fit relationship with `BASECODE`'s address.

Let me find every reference to `ABORTHDR`/`QUITHDR` throughout the file before making any changes, so the rename doesn't leave anything dangling.

Let me check the full Section 26 content, since it also contains `TRUEBODY`/`FALSEBODY` (real code that must stay in `BASECODE`, not move with the headers).

Now implementing the full change. First, updating the Section 27 header comment (which references these headers extensively, and also has a separately-stale `$D85D` address I'll fix while I'm here).

Now inserting the renamed `H_ABORT`/`H_QUIT`/`BASELATEST` block right after `DICTTOP`, at the true end of the dictionary entries.

Now removing the old `ABORTHDR`/`QUITHDR`/`BASELATEST` block from its former location, leaving Section 26 correctly retitled around just `TRUEBODY`/`FALSEBODY`.

Verifying zero remaining bare references and running full structural verification.

Confirmed all three are my own explanatory comments documenting the rename — no real leftover references. Now confirming the "duplicate symbols" flag is the expected `SERIALPOLL` false positive, not something new.

Confirmed expected (10 = 2 branches × 5 labels). Let me run the proper simulation-based verification for both `SERIALPOLL` branches, matching my established methodology, to be thorough after this reorganization.

All structural checks pass in both configurations. Now let me precisely

quantify the real consequence I flagged earlier — the new gap between `BASEDICT`'s actual end and `BASECODE`'s fixed start.

Let me run the full comprehensive pairwise sweep across the entire memory map to confirm this is the only new consequence, not something that cascades further.

Confirmed — exactly one new overlap (19 bytes), nothing else affected. Let me check whether the top-of-file summary note makes a "zero overlaps" claim that's now stale too.

Found it — this claim is now stale. Let me view and fix it.

Now regenerating the split files to match this reorganized content.

Exact reassembly confirmed. Delivering the updated `.asm` and split files.

Now updating the checklist with this reorganization and the important overlap finding it created.

Let me also correct the previous turn's "zero overlaps anywhere" summary entry, since it's now stale too.

Delivering the updated checklist.

`ABORTHDR`/`QUITHDR`/`BASELATEST` moved into `BASEDICT` at the end of the dictionary entries (right after `DICTTOP`), renamed to `H_ABORT`/`H_QUIT`. `TRUEBODY`/`FALSEBODY` stayed in `BASECODE` where they belong — only the two header structures moved; the section they used to share got retitled around what's left.

This wasn't purely cosmetic, and checking it properly surfaced something worth being direct about: `ABORTHDR`/`QUITHDR` were previously sitting *physically inside `BASECODE`'s address range* (Section 26 sat before `ORG BASEDICT`), even though they're header data, not code — so their bytes were being counted against the wrong region's budget. Moving them into the real `BASEDICT` block fixes that, but has a genuine consequence: `BASEDICT`'s real size grew from 1,973 to 1,992 bytes (the 19 bytes `H_ABORT`/`H_QUIT` actually occupy), while `BASECODE`'s start address is fixed and doesn't move to compensate — reintroducing a real 19-byte overlap between them, the same class of problem as the original `INITCODE`/`BASECODE` overlap resolved several turns ago. I ran a full pairwise sweep of the entire memory map to confirm this is the *only* new overlap, not something that cascades further, and left it unresolved rather than guess at the right fix — the same reasoning as before: whether to shift `BASECODE` to match is a decision, not a mechanical follow-up.

Verified throughout: zero duplicate symbols (checked against both `SERIALPOLL` branches via the same simulation method established for that feature), dictionary chain still 219 entries, walked correctly starting from `H_QUIT`. I also caught and fixed two separately-stale items while touching this text: the Section 27 header comment still cited `BASEDICT`'s pre-30-byte-shift address, and the top-of-file summary's "zero overlaps anywhere" claim — both now updated, and I went back to correct the equivalent claim in last

turn's checklist entry too rather than leave two contradicting "current state" claims sitting in the log.

Forth6809
ASM · Click to download

26 abort quit headers
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

> Now eliminate the overlap by adjusting BASECODE’s starting address by 19 bytes.

Confirmed exact — `BASECODE` at $E007‘landsprecisely one byte above ‘BASEDICT‘'s new real end (‘$`E006`), zero gap, zero overlap. Implementing this.

Let me verify the follow-on consequence I just wrote in that comment — that shifting `BASECODE`'s start (while its nominal size stays fixed) pushes its nominal end further into `INITCODE`'s territory.

I made an arithmetic error in that comment — let me fix it. The actual consequence is more significant than I first wrote: shifting `BASECODE`'s start by 19 bytes (while its nominal *size* stays fixed) doesn't eliminate the 19-byte overlap — it relocates the exact same 19 bytes from the `BASECODE`/`BASEDICT` boundary to the `BASECODE`/`INITCODE` boundary.

Let me check `BASEDICT`'s own adjacent comment too, since it references the old `BASECODE` address for the contiguity claim.

Now running the comprehensive pairwise sweep on the actual updated file for the definitive, complete picture.

Confirmed precisely — exactly one overlap remains (`INITCODE`↔`BASECODE`, 19 bytes), and the `BASECODE`↔`BASEDICT` overlap from last turn is gone. Net result: the requested fix resolved the specific overlap asked about, but relocated an identical 19-byte overlap elsewhere. Running full structural verification before delivering.

All checks pass in both configurations. Regenerating the split files and delivering.

Now updating the checklist with this precise, important finding.

`BASECODE` moved from $DFF4 `to` `E007`. Verified this lands exactly one byte above `BASEDICT`'s real end (`$E006`) — the specific overlap you asked about is closed, zero gap, zero overlap.

But I need to be direct about something rather than just report success: **this didn't eliminate the overlap, it relocated it.** `BASECODE`'s nominal size (8,110 bytes) didn't change, only its start address — so its nominal end shifted by the same 19 bytes too, from $FFA1 `to` `FFB4`, which now sits 19 bytes inside `INITCODE`'s start (`$FFA2`). A full pairwise sweep of the entire memory map confirms exactly one overlap remains, same 19-byte total, just now between `INITCODE` and `BASECODE` instead of `BASECODE` and `BASEDICT`.

This makes sense once you trace why: `BASECODE`'s budget was calibrated in an earlier turn specifically to reach exactly to `INITCODE`'s start from its *old* position. Moving the start without also shrinking the budget can't reduce the total overlap — it just pushes the same 19 bytes to the other end. Actually closing both boundaries at once would need `BASECODE`'s nominal size reduced by 19 bytes (to 8,091), not just its address shifted — a different change than what was asked, so I didn't do that unilaterally.

One more thing worth flagging on myself: my first draft of the explanatory comment miscalculated this new overlap as 7 bytes at $FFAE `.Icaughtitbyactuallycomputingthenumberswith Pythonratherthantrustingmyownarithmetic, andcorrecteditto therealvalue(19bytesat` `FFB4`) before any of this went out. Verified throughout: zero duplicate symbols across both `SERIALPOLL` branches, dictionary chain still 219 entries in both configurations. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

DICTTOP does not appear to be used anywhere, verify that this is the case and if so remove it and any references to it.

May have been made obsolete by BASELATEST.

Confirmed exactly what you suspected — `DICTTOP` is defined once and referenced only one other time, and that single reference is inside my own explanatory comment (prose), not actual code. Let me do one more broader

check to be certain before removing it.

Confirmed — genuinely dead, unused symbol. Removing it and rewriting the one comment that names it.

That checklist mention is accurate history (`DICTTOP` genuinely existed at that point and was the real insertion anchor) — leaving it as-is, consistent with treating the checklist as a historical log. Running full structural verification on the `.asm` change.

All checks pass: `DICTTOP` genuinely gone, zero duplicate symbols, dictionary chain still 219 entries intact in both configurations. Regenerating the split files and delivering.

Now updating the checklist with this finding.

Confirmed — your suspicion was exactly right. `DICTTOP` was defined once and referenced exactly one other place in the whole file, and that single reference was inside a prose comment, not real code. Nothing `EQU`'d against it, nothing `JSR`'d or `FDB`'d it.

Also confirmed it wasn't simply *the same thing* as `BASELATEST` under a different name — they hold genuinely different values (`DICTTOP` = `H_FALSE`, the newest of the 217 auto-generated entries; `BASELATEST` = `H_QUIT`, the true overall chain head that `COLD` actually uses to initialize `LATEST`). `DICTTOP` wasn't made *redundant* by `BASELATEST` so much as it was a marker for a distinction (the generated entries' own boundary) that nothing in the codebase ever turned out to need.

Removed the `EQU` line and rewrote the one comment that named it to describe `H_FALSE` directly instead, so nothing dangling remains. Verified zero duplicate symbols and dictionary chain still 219 entries intact across both `SERIALPOLL` branches. Left the checklist's one historical mention of `DICTTOP` untouched, since it's accurately describing where `H_ABORT`/`H_QUIT` were inserted last turn (when `DICTTOP` genuinely still existed) — added a new entry for this removal rather than rewrite that history. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

In the documentation, section 7, Architecture, insert a subsection prior to the text called: (Outer) Interpreter.
Move the graphic so that it is inserted between the subsection heading and the text.

Append a new subsection called: Dictionary Operations.
Add a subsection: Structure
Add a subsection: Initialization
Add a subsection: Word search
Add a subsection: Word append

Under Structure explain that the dictionary is a linked list that grows from low to high memory, with each entry storing a link to the header of the prior entry. The first created entry has a link field of zero. The last created entry's header is pointed to by an external link stored in a a variable called <place holder>.
Under Initialization explain what variables are set to establish the beginning of the search and the beginning of a new dictionary item.
Under Word Search explain how the dictionary is traversed to find a word and how the end of a search is detected.
Under Word append explain how the pointers are manipulated to establish the new dictionary item location and how its link pointer is updated.

This requires accurate technical content grounded in the real code, not generic Forth knowledge. Let me trace the actual dictionary mechanics — `LATEST`, `DPHERE`, `HEADER`, and `FIND` — before writing anything.

Let me view `HEADER` (the word-append routine) and `FIND` (the search routine) in full.

Now let me view `FIND` in full for the "Word Search" subsection.

Now I have complete, accurate mechanics traced from the real code. Let me set up the docx editing workflow and check the current Section 7 structure.

Let me check whether a `Heading 3` style is available in this document's style set, for the nested sub-subsections.

Now let me check the current TOC structure near "7. Architecture" and find the highest existing bookmark number, since this needs fresh, real values, not assumptions from a prior turn.

That's unexpected — zero `toc_N` bookmarks found. Let me check the actual bookmark names directly on these specific paragraphs.

That confirms `toc_72` is indeed used (matching what I set up two turns ago) — my search pattern just didn't match. Let me search correctly for bookmark definitions across the whole document.

Bookmark IDs aren't always reused as "1" — this one uses `261`. Let me find

the actual maximum ID across the whole document to pick safe values.

Now implementing systematically. First, inserting the TOC entries for the two new Heading 2 subsections.

Confirmed — the text includes a trailing page number after the tab. Using `startswith` instead of exact match.

Now inserting the actual headings, moving the image, and adding the Dictionary Operations content in the document body.

`7.1` correctly inserted. Now moving the image to sit right after this new heading, before the text.

Correctly positioned. Now fixing the "diagram below" wording, since the diagram is now above the text, not below it.

Now inserting "7.2 Dictionary Operations" and its four Heading 3 subsections, with the technical content grounded in the traced code.

The style lookup by name fails, but I can see from the earlier XML inspection that the actual style ID is `Heading2` (no space) — let me set this directly via XML instead of relying on python-docx's name-based lookup.

Style exists but python-docx's name lookup mysteriously fails — bypassing it entirely by setting the XML `pStyle` value directly.

All content correctly ordered. Let me verify structural integrity before the final render.

Exactly matches expectations (2 TOC entries + 2 headings + 4 sub-headings + 4 body paragraphs = 12). Running the field-update and PDF export pass.

Both files completed. Confirming cleanup and verifying content.

Clean. Now verifying the actual rendered content.

Both TOC entries and both actual headings found. Let me check the actual content pages directly.

All content renders in the correct order with accurate technical content. Let me verify the image survived the move (pdftotext can't show it, but I can check via rendering).

Confirmed — the image relationship is intact with the exact same byte count as the original (327,603 bytes, matching the source PNG exactly), and the page shows real, substantial non-white content matching the expected diagram density. Finalizing delivery.

Confirmed identical via hash.

Both files delivered and verified. Summary of what changed:

Reordered content: "7.1 (Outer) Interpreter" now sits right after the "7. Architecture" heading, with the diagram moved to appear directly beneath it, before the introductory text. I also caught that this reordering made the existing sentence "the diagram **below** shows…" factually backwards, since the diagram is now above the text — corrected it to "above."

New "7.2 Dictionary Operations" with four Heading 3 subsections, each

with content I grounded in the actual code rather than generic Forth knowledge — I traced `HEADER`, `FIND`, and `COLD`'s initialization directly before writing anything:

- **Structure**: filled in your placeholder — the variable is `LATEST`, confirmed by checking `HEADER`'s own code (it reads and writes `LATEST` directly) and `COLD`'s initialization. Also confirmed `H_KEY`'s link field is genuinely `0` (`FDB 0`), the true base of the chain.
- **Initialization**: `LATEST` and `DPHERE`, both set once by `COLD` — `LATEST` to `BASELATEST` (the ROM dictionary's pre-built head) and `DPHERE` to `APPDICT` (start of the application dictionary).
- **Word Search**: `FIND`'s actual traversal — smudge-bit and length checks before any character comparison, walking backward via each entry's link field, ending on either a match or a link value of zero.
- **Word Append**: `HEADER`'s actual mechanics — capturing `DPHERE` as the new entry's address, writing the old `LATEST` value as the new entry's own link field, and only updating `DPHERE`/`LATEST` after the entry is fully written.

Both new subsections got real TOC entries with working page-reference links, matching the existing document's convention, and I verified the image survived the move with an exact byte-count match against the source file (327,603 bytes) rather than assuming the relocation was clean.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

That worked well.

Glad it landed cleanly — that one had a fair number of moving parts (the TOC bookmark plumbing, the image relocation, getting the dictionary mechanics right from the actual code rather than generic Forth conventions) to get right in one pass.

The ClaudeForth documentation and source are in a good state now — happy to keep going if there's more to do, whether that's further Section 7 content, another pass on the open items, or something else entirely.

In the forth6809.asm file, extract the serial interface initialization code inside INITCODE into a subroutine called INITSERIAL, starting with LDA #$03 and ending with STA ACIACR.

Move that subroutine to the ACIA INTERRUPT section in the assembly listing, just before IFEQ SERIALPOLL line representing the start of the INFILL subroutine.

Confirmed the exact insertion point — right after `ORG BASECODE`, before `INFILL`'s comment block. Implementing the extraction now.

Now inserting the new `INITSERIAL` subroutine at the correct location in the ACIA Interrupt section.

Now running full structural verification, since `INITSERIAL` itself contains a conditional block, adjacent to the pre-existing one.

Correct — moving code doesn't add a new conditional block, it relocates the existing one. Still 3 total, balanced. Now running the full simulation-based verification for both `SERIALPOLL` branches.

This is a small, bounded side-effect worth noting (moves ~10 bytes of real content from `INITCODE` to `BASECODE`, well within the already-acknowledged estimation uncertainty) — not something requiring a full byte-count recompute. Regenerating the split files and delivering.

Byte-exact reassembly confirmed, and both the subroutine and its call site landed in the correct split files. Now updating the checklist.

The serial init code (`LDA #$03` through `STA ACIACR`, including the `SERIALPOLL` mode-select) is now `INITSERIAL`, moved into the ACIA Interrupt section right before `INFILL`'s `IFEQ SERIALPOLL`. `COLDSTRT` now just does `JSR INITSERIAL` where the inline code used to sit — the extracted block keeps its internal conditional logic (`CR_RXON` vs `CR_POLL`) exactly as it was.

Worth naming rather than leaving implicit: this shifts roughly 10 bytes of real content from `INITCODE` to `BASECODE` (the new subroutine, offset by removing the inline code and adding a 3-byte `JSR`). That's well within the estimation uncertainty already on record for both regions from earlier turns — not something that meaningfully changes the tracked overlap numbers, but worth a line in the checklist rather than silently absorbed.

Verified: `IFEQ`/`ELSE`/`ENDC` still balanced (3/3/3 — relocating a conditional block doesn't create a new one), `INITSERIAL` defined exactly once and correctly resolved by its `JSR`, byte-exact reassembly of the split files against the consolidated source, and zero duplicate symbols with the dictionary chain still 219 entries intact in both `SERIALPOLL` configurations. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

02 init coldstrt warm
ASM · Click to download

03 irqh
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Update the ClaudeForth document with the updated assembly code.

All 28 found correctly. Now replacing each section's content with the current split-file content, matching the established technique.

All 28 replaced. Verifying the recent changes actually landed correctly before finalizing.

Let me check the actual spacing — likely a false alarm in my verification string, not a real problem.

Let me check directly — this could indicate a real mismatch between the split-file mapping and what's in the docx.

Let me check the actual heading titles — I may have mislabeled which subsection number corresponds to which content in my own verification check, not a real bug in the replacement.

My verification script had the wrong expectation, not the replacement itself — `8.2` is "Hardware Vector Table," `8.3` is "Init Code." Let me redo the check correctly.

All correctly verified now — my earlier check simply targeted the wrong subsection number. Confirming paragraph count is unaffected, then running the final field-update and export pass.

Zero paragraph-count change confirms only content within existing paragraphs was modified. Running the field-update and PDF export pass.

Both files completed. Confirming cleanup and verifying the actual rendered content.

That specific check failed — let me investigate whether this is a text-extraction artifact or a real problem.

Confirmed present — my check string was just too strict about exact whitespace (PDF text extraction normalizes spacing slightly). The actual content is correct. All changes verified present. Finalizing delivery.

Confirmed identical via hash.

The "Assembler Source" section (all 28 subsections, 8.1–8.28) now reflects the current `forth6809.asm` — this brings in everything since the last documentation sync: the `INITSERIAL` extraction, `H_ABORT`/`H_QUIT` (renamed and relocated into `BASEDICT`), `DICTTOP`'s removal, and the `BASECODE` 19-byte shift to `$E007`.

One thing worth flagging on myself: my first verification pass checked the wrong subsection for the `INITSERIAL` call site — I expected "8.2" to be the Init Code section, but that's actually "8.3" (8.2 is "Hardware Vector Table"). I caught this by checking the real subsection titles before concluding anything was wrong, rather than reporting a false negative. Once corrected, everything verified as present and correctly placed. A second check (the exact `BASECODE EQU $E007` text) also initially came back negative — that one turned out to be PDF text-extraction normalizing whitespace slightly differently than my search string expected, not a real gap; confirmed the actual content is correct.

Verified throughout: zero paragraph-count change (confirming only content within existing paragraphs was touched, no structural drift), and all four recent changes (`INITSERIAL`, `H_ABORT`, `H_QUIT`, `DICTTOP`'s absence) confirmed present in the final rendered PDF.

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

Can you simulate the effect of the ACIA receiving the characters A, B, C, <CR> and then track the processing through the formation of a string and the dictionary search, verifying the expected outcome for each subroutine call.

The intent is to identify the location of a bug when the polling method of receiving user input is used.

Let me view the polling `KEY`/`KEYQ`/`EMIT` (the second set, lines 2752+) and `ACCEPT` in full first.

This immediately raises a flag — `KEYQ` references `TRUEV`/`FALSEV`, but I recall the actual defined labels elsewhere in this file being `TRUEVAL`/`FALSEVAL`. Let me check whether `TRUEV`/`FALSEV` are real, distinct symbols or a genuine bug.

Good — `TRUEV`/`FALSEV` are genuine, real constants used pervasively for boolean flags throughout the codebase (distinct from `TRUEVAL`/`FALSEVAL`, the `CONSTANT`-pattern value cells). Not a bug. Let me view the rest of `ACCEPT`, `QUERY`, `WORD`, `FIND`, and `INTERPRET` in full before simulating.

Let me now trace `QUERY`, `WORD`, and `FIND` fully before running the actual character-by-character simulation.

Let me view `INTERPRET`'s call to `WORD` too, to see what delimiter gets pushed, completing the full trace.

This is a significant finding — `INTERPRET`'s `ILOOP: JSR WORD` doesn't appear to push a delimiter character before calling `WORD`, which expects one (`PULU D / STB DELIM`). Let me check `QUIT`'s calling context to see whether a delimiter gets pushed there instead, before concluding this is a real bug.

This is a significant, pre-existing structural question, not necessarily polling-specific — let me verify precisely by tracing `CATCH`'s mechanics, since it's what actually invokes `INTERPRET`.

Let me check whether `IRQH` (the interrupt-driven path) happens to touch the `U` stack in a way that could incidentally mask this same bug — which would explain why it's only visible under polling.

That sed range was far too broad — let me extract `IRQH` precisely by its actual boundaries.

Confirmed — `IRQH` (the interrupt-driven path) never touches the `U` (data) stack at all, which rules out the possibility that interrupt mode "accidentally" leaves something usable there while polling doesn't. This means the bug I found is present identically in both modes, not specific to polling. Let me now build the complete, precise simulation trace.

Now I have everything needed for a complete, accurate trace. Let me present the full simulation.

Simulation: ACIA receives `A`, `B`, `C`, `<CR>` (polling mode, `SERIALPOLL=1`)

Character reception — `KEY` → `ACCEPT` — this part works correctly. I traced every register through each call:

For each of `A` (41), ‘B‘(42), `C` ($43):

Step	Instruction	Register state
`KEY` polls	`LDA ACIASR` / `BITA #SR_RDRF` / `BEQ KEY`	loops until `RDRF` set
`KEY` reads	`LDA ACIADR` → `TFR A,B` → `CLRA` → `PSHU D`	`D` = `$00xx` (char in B), pushed to `U`
`ACCEPT` receives	`PULU D` / `STB ACH`	`ACH` = the character
Not CR/LF/BS	falls through	stores char at `ABUFP+ACNT`, `ACNT`+1
Echo	`CLRA`/`LDB ACH`/`PSHU D`/`JSR EMIT`	`EMIT` correctly pulls `D`, waits `TDRE`, writes `ACIADR`

After `A`,`B`,`C`: `TIBBUF` = `"ABC"`, `ACNT` = 3. Then `<CR>` ($0D): `CMPB #13` matches → `ADONE` → returns `ACNT` (3) via `PSHU D`.

`QUERY` finalizes correctly: pulls 3 from `ACCEPT`, sets `NTIB`=`SRCLEN`=3, `SRCADDR`=`TIBBUF`, `TOIN`=`SRCID`=0. I checked `QUERY`'s push order against `ACCEPT`'s pull order (`TIBBUF` pushed first, `TIBBUFL` pushed second, so `ACCEPT`'s first pull gets `TIBBUFL`→`AMAX`, second gets `TIBBUF`→`ABUFP`) — correctly matched, no bug here.

This is where it breaks — and it's not actually a polling-specific bug.

`QUIT` invokes the interpreter as `LDD #INTERPRET / PSHU D / JSR CATCH`. `CATCH` does `PULU X` (consuming that execution token) and pushes nothing else onto `U` — its `HANDLER`/`U` bookkeeping goes onto `S` (the return stack), not `U`. So by the time `INTERPRET`'s `ILOOP: JSR WORD` executes, `U` is back to empty (`U` = `SP0`), and **`WORD`'s `PULU D / STB DELIM` pulls from an empty stack** — nothing pushed a delimiter character first.

On real 6809 hardware this doesn't fault — `PULU` just reads whatever bytes sit at that address and advances `U` past them. I checked where that lands: `SP0 EQU DSTACK+1`, and `DSTACK`'s own occupied range ends at exactly that address — meaning `SP0` is the *first byte of `RSTACK`*, not a "safety margin" of unused memory. So `DELIM` gets set to two bytes of live **return-stack** content (saved `HANDLER`/`U` values and return addresses from the `CATCH`→`INTERPRET` call chain), not a defined value. I traced this precisely enough to identify the mechanism; pinning the *exact* resulting byte value would need the full call-depth state at that instant, which I can't determine without actually running the code.

Consequence for "ABC": `WORD`'s `SKIPLP`/`SCANLP` compare each input character against whatever landed in `DELIM`. Unless that garbage byte happens to equal $20 (space), `WORD` won't treat the space-or-CR boundary the way the caller intended — it may fail to skip a leading delimiter, or scan straight through where a boundary should be. Either way, whatever reaches `FIND` afterward is not reliably `"ABC"`, so the dictionary search becomes unpredictable downstream of a bug that has nothing to do with `FIND` itself.

Why this isn't specific to polling, and worth being direct about: I checked whether interrupt-driven mode's `IRQH` might incidentally leave something usable on `U` that masks this — it doesn't touch `U` at all (verified by searching its actual body for `PSHU`/`PULU`, finding none). So this bug is present identically in both `SERIALPOLL` configurations; it's in `INTERPRET`'s calling convention for `WORD`, entirely upstream of how characters get read. If what you're actually observing is behavior that differs *specifically* between polling and interrupt mode, this isn't the explanation for that difference — there'd be something else to find. But if the symptom is "input gets garbled or misparsed" in general, this is a real, concrete bug sitting exactly there, and I haven't touched the file to fix it yet, pending your read on which one you're chasing.

> The bug has not been tested as occurring in the interrupt driven code. So

assume the bug occurs in both polling and interrupt driven branches. So proceed with the bug fix.

No assembler-level `BL` constant exists, but `ACCEPT`'s own `ABKSP` code uses `LDD #32` (decimal) for the space character — matching that established convention for consistency.

Let me verify this correctly resolves the trace — with `DELIM`=32 now guaranteed, `WORD`'s `SKIPLP`/`SCANLP` will correctly parse `"ABC"` (no leading space to skip, scans all 3 characters until the count reaches zero, captures `"ABC"` exactly). Running full structural verification.

Let me check every other call site of `WORD`, to confirm they already correctly push a delimiter (unlike `INTERPRET`, which didn't) and don't need the same fix.

This confirms the diagnosis precisely — every other call site (`HEADER`, `TOW`, `ISW`, `SQUOTE`, `CHARW`, `LPAREN`) already correctly pushes a delimiter before calling `WORD`. `INTERPRET` was the *only* one missing it, and my fix now matches the exact same established convention. Regenerating split files and delivering.

Now updating the checklist with this important bug fix.

Fixed. `INTERPRET` now pushes `LDD #32 / PSHU D` right at the `ILOOP:` label before `JSR WORD` — the exact same convention already used correctly by every other caller (`HEADER`, `TOW`, `ISW`, `SQUOTE`, `CHARW`, `LPAREN`), which is what confirmed this was really a missing push rather than something wrong with `WORD` itself.

Placing the fix at the label rather than at each `BRA ILOOP` site was deliberate: since every branch back to `ILOOP` (from `DOEXEC`, from `CCALL`, from `TRYNUM`'s success path) re-enters at that exact point, one insertion covers every iteration of the loop, not just the first pass through.

Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches, and byte-exact reassembly of the split files against the consolidated source. Propagated to the split files and checklist, with the checklist entry recording the "confirmed present in both branches, not polling-specific" finding per your note, rather than leaving that ambiguous for whoever reads it next.

Forth6809
ASM · Click to download

09 outer interpreter
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Change the INITCODE EQU FFA2 to INITCODE EQU FFA9

This shift has a mixed effect worth precisely quantifying before applying: it reduces the `BASECODE`/`INITCODE` overlap from 19 to 12 bytes, but introduces a **new** 7-byte overlap with `VECTORS`, since `INITCODE`'s real content (78 bytes, an established, verified figure) previously ended exactly one byte below `VECTORS`' start — zero gap. Applying the change and reporting both effects precisely.

Now checking for other stale references to `$FFA2` throughout the file.

Small grammatical gap — fixing it.

Now fixing the `ORG INITCODE` line's stale inline comment.

Let me check the top-of-file summary note too, and then run the comprehensive pairwise overlap sweep for a complete, precise picture.

This surfaces a useful additional nuance — using `BASECODE`'s *real* content size (not its nominal budget) shows the `INITCODE`/`BASECODE` overlap actually vanishing, leaving only the new `VECTORS` overlap, which is based on `INITCODE`'s solid, established 78-byte content rather than an estimate. Running structural verification and updating the checklist with the full picture.

All checks pass. Regenerating split files and delivering.

Now updating the checklist with the full, precise picture.

`INITCODE EQU` $FFA9‘(was‘$`FFA2`), applied exactly as requested.

Worth being precise about the consequences rather than just confirming the edit landed, since this one has a real mixed effect:

- **The overlap with `BASECODE` shrinks**, from 19 bytes to 12 $FFA9‘##$ (`FFB4`) — real progress against `BASECODE`'s nominal 8,110-byte budget, not a fix.
- **But it creates a new overlap with `VECTORS`.** `INITCODE`'s real content is a solid, established 78 bytes (`COLDSTRT`+`WARM`+`WARMMSG`), not an estimate — at $FFA2‘itendedexactlyonebytebelow‘VECTORS“′start, zerogap.At‘$`FFA9` it now ends at `$FFF6`, 7 bytes *into* `VECTORS`. This one didn't exist before this change, and it's arguably more certain than the `BASECODE` overlap, since it isn't resting on a byte-count estimate the way `BASECODE`'s budget is.

One more thing worth flagging: I checked what happens if `BASECODE`'s *real* content size (7,984 bytes, the precise manual count from several turns back) is used instead of its nominal budget — under that assumption, the

`INITCODE`/`BASECODE` overlap actually disappears entirely, leaving only the new `VECTORS` overlap. So depending on which figure you trust more, this change either net-improves things or net-neutral-trades one overlap for another — I didn't try to resolve either overlap, just report both precisely.

Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches, byte-exact reassembly of the split files. I also went back and fixed three separately-stale comments that still cited `INITCODE`'s old address (`$FFA2`) or an outdated overlap figure from before the `BASECODE` shifts. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

FYI the assembler reports the size of INITCODE to be 71 or $47 bytes.

This is significant — the first real assembler-reported figure we've had in this whole exercise, as opposed to a manual estimate. Let me recompute the `VECTORS` overlap finding from last turn using this authoritative number.

This is a real, important correction — with the authoritative 71-byte figure, `INITCODE`'s real content ends exactly at `$FFEF`, one byte below `VECTORS`, a genuine zero-gap fit. My "new `VECTORS` overlap" finding from last turn was a false alarm based on a flawed manual estimate. Let me find and correct every place that cited the wrong 78-byte figure.

Only one genuine reference (line 376 is an unrelated hex offset). Let me view and correct the full comment block.

Let me search more broadly for other places where this now-corrected 78-byte figure might have propagated across many earlier turns.

Clean — zero other matches. Let me check the checklist too, since this narrative was likely recorded there across several past turns.

Consistent with treating the checklist as a historical log, I'll add superseding notes to these four entries (not rewrite them) plus one new, comprehensive entry recording the correction.

For the remaining three, brief pointers rather than full duplicated superseding blocks — the one comprehensive new entry will carry the authoritative correction.

These two are already properly hedged as estimates — no correction needed. But last turn's entry made a confident, now-incorrect claim — that one needs fixing.

Now running full structural verification and delivering.

Clean — this was a comment-only change, no code touched. Regenerating split files and delivering.

Thanks for the correction — this genuinely changes the finding, not just a number. With `INITCODE` at $FFA9`andthereal71-bytesize,itscontentendsatexactly`$ `FFEF`, one byte below `VECTORS` — a real, assembler-confirmed zero-gap fit.

I need to retract what I told you last turn. I claimed shifting `INITCODE` created a new 7-byte overlap with `VECTORS`, and called it "more certain than the `BASECODE` overlap since it's based on solid content rather than a budget estimate." That framing was backwards — my own 78-byte figure was itself an unconfirmed manual estimate, and it turned out to be wrong by exactly 7 bytes. The overlap I reported didn't actually exist.

The separate `BASECODE`/`INITCODE` overlap (12 bytes against `BASECODE`'s nominal budget) is unaffected by this and remains open — that one depends on `BASECODE`'s size, not `INITCODE`'s.

I corrected the one place in `forth6809.asm` that cited the wrong 78-byte figure, and updated the checklist: added a clear retraction to last turn's entry (which made the specific, now-disproven claim) plus a new entry recording this as the first genuine assembler-reported figure in the whole project, distinct from every other byte count in this file, which remains a manual estimate with an acknowledged margin of error. I left two older checklist entries alone rather than edit them, since they already said "still an estimate, not a real assembler's output" at the time — nothing in them was actually wrong, just superseded. Verified zero duplicate symbols in both `SERIALPOLL` branches and byte-exact split-file reassembly, since this was a comment-only change with no code touched.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

In the forth6809.asm file, replace the definition of FALSE with a simple

subroutine that places $0000 on the stack.

Similarly replace the definition of TRUE with a simple subroutine that places $FFFF on the stack.

This is unexpected — TRUEBODY appears to load $0000`and`FALSEBODY`loads`FFFF, which looks backwards from what I'd expect. Let me view the full current code before assuming anything.

This is very informative — the current code already has TRUE/FALSE **deliberately inverted** from a prior request (documented in the comment: TRUEBODY pushes $0000`,`FALSEBODY`pushes`FFFF, explicitly flagged as creating an inconsistency with every comparison operator). Your current request — TRUE pushes $FFFF`,`FALSE`pushes`0000 — is exactly the standard, correct convention that resolves that flagged inconsistency. Implementing it precisely.

Let me check whether TRUEVAL/FALSEVAL (the old DODOES-pattern value cells) still exist and are now genuinely unused, similar to the DICTTOP situation two turns ago.

Already cleaned up in the prior change — nothing further needed there. Let me confirm H_TRUE/H_FALSE's CFA fields still correctly point to these labels (unaffected by the value swap).

Confirmed correct. Running full structural verification.

All checks pass, values confirmed correct. Now updating the checklist.

No existing entry to supersede — that prior change apparently predates what's recorded in the checklist (likely lost to a context compaction boundary). Adding a new entry documenting this fix directly.

Now regenerating the split files and delivering.

Both TRUEBODY and FALSEBODY are now simple LDD #value / PSHU D / RTS subroutines — TRUE pushes $FFFF`,`FALSE`pushes`0000.

Worth being direct about what I found before making this change: the code already had these implemented as simple subroutines, but with the values **inverted** — TRUE was pushing $0000`and`FALSE`waspushing`FFFF, with a comment explicitly documenting that as a deliberate change from an earlier request, one that isn't captured anywhere in the checklist (likely predating a context boundary, since there's no corresponding entry to mark as superseded). That comment also correctly flagged the real consequence at the time: TRUE/FALSE disagreed with TRUEV/FALSEV and every comparison operator (=, <, >, and the rest), all of which return TRUEV ($FFFF`)fortrueandFALSEV`(`0000) for false. Your request today happens to specify exactly the standard values, so this change resolves that inconsistency as a side effect, not something I unilaterally decided to fix.

TRUEVAL/FALSEVAL (the old DODOES-trampoline value cells from before that prior change) were already removed — nothing left to clean up there.

Verified: `H_TRUE`/`H_FALSE`'s CFA fields (`FDB TRUEBODY`/`FDB FALSEBODY`) unaffected, since only the values loaded inside those subroutines changed, not the labels themselves; zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Add simple definitions for 1, -1, 2, -2 that simply place the immediate values on the stack.

Also FIND does not seem to have a dictionary entry. Add it if this is confirmed as correct. Ensure the stack behavior is correct.

Confirmed — `FIND` genuinely has no dictionary entry. Let me examine its actual implementation precisely before adding a header, to make sure the exposed stack behavior is genuinely correct, not just assumed.

Let me trace `NOTFOUND` too, to fully verify the failure-case stack behavior against the standard ANS `FIND` stack effect.

`FIND`'s implementation matches the standard ANS Forth stack effect exactly — success returns `(xt 1)` for immediate or `(xt -1)` for normal, failure returns `(c-addr 0)` with the original `c-addr` correctly reconstructed. Confirmed correct; safe to add a header for it as-is. Now checking the current chain-end to know where to insert all five new entries.

Now implementing. First, inserting the five new dictionary headers, chained after `H_QUIT`.

Now updating the two comments that describe the chain's head, since `H_QUIT` is no longer the true head — `H_M2` is.

Now updating `BASELATEST`'s own comment.

Now adding the four subroutine bodies, right next to `TRUEBODY`/`FALSEBODY` — matching the established pattern for simple value-pushing words.

Running full structural verification.

All confirmed correct — 224 entries (219 + 5 new), correctly ordered, ending

at `0`. Let me confirm `COLD`'s `LATEST` initialization automatically picks up the new head via the symbolic `BASELATEST` reference.

Confirmed symbolic — automatically picks up `H_M2` with zero code change needed there. Regenerating the split files and delivering.

Now updating the checklist.

Both parts done.

`1`/`-1`/`2`/`-2`: four new dictionary words, each a simple `LDD #value / PSHU D / RTS` — same pattern just established for `TRUE`/`FALSE`, no indirection.

`FIND`: confirmed missing before doing anything — searched for any `H_FIND` label or `FCC "FIND"` string and found neither. Before adding a header, I actually verified the stack behavior rather than assume it was fine: traced `FIND`'s real code against the standard ANS stack effect (`c-addr -- c-addr 0 | xt 1 | xt -1`). The success path pushes the xt, then correctly distinguishes immediate (`1`) from normal (`-1`) via the header's flag bit; the failure path reconstructs the *original* `c-addr` (not just some pointer) and pushes it alongside `0`. Both match exactly, so no code changes were needed — only the missing header.

All five new headers (`H_FIND`, `H_1`, `H_M1`, `H_2`, `H_M2`) are chained after `H_QUIT`, with `H_M2` now the true chain head. `BASELATEST` updated accordingly — since `COLD` references it symbolically, no code change was needed there. I also went back and fixed two comments that still named `H_QUIT` as the head, which would have been actively wrong the moment this landed.

Verified: zero duplicate symbols, dictionary chain now 224 entries (was 219), correctly ordered and ending at `0`, in both `SERIALPOLL` branches; byte-exact split-file reassembly. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

27 forth dictionary
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

In forth6809.asm change BASECODE to E02A

This shift has two effects worth precisely reporting: it creates a new 35-byte gap between `BASEDICT`'s real end and `BASECODE`'s new start (previously contiguous, zero gap), and it *worsens* the `BASECODE`/`INITCODE` overlap from 12 bytes to 47 bytes. Applying the change and updating the comments with

these precise findings.

Checking for other stale references to `$E007` throughout the file.

Stale — this one is even further behind (predates the 19-byte shift too). Fixing it.

Let me check `BASEDICT`'s own adjacent comment too, since it claims contiguity with `BASECODE` that no longer holds.

Let me verify this edit landed cleanly, since the original comment continued beyond what I replaced.

Found a leftover dangling line from the old comment. Fixing it.

Now let me run the comprehensive pairwise overlap sweep for the definitive, precise picture, and full structural verification.

Confirmed exactly one overlap (47 bytes), matching the computed figure precisely. Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist with the precise findings.

BASECODE EQU $E02A`(was`E007), applied exactly as requested.
Worth being direct about this one rather than just confirming the edit landed — neither effect of this shift is an improvement:

- **A new 35-byte gap opens** between `BASEDICT`'s real end (`$E006`) and `BASECODE`'s new start. These were exactly contiguous before this change (zero gap, established a couple turns back). Not a collision — just unused address space where there wasn't any.
- **The `BASECODE`/`INITCODE` overlap gets worse**, not better: `BASECODE`'s nominal size (8,110 bytes) is unchanged, so its nominal end shifted up by the same 35 bytes — the overlap with `INITCODE` grows from 12 bytes to 47.

Full pairwise sweep confirms this 47-byte overlap is still the only one anywhere in the memory map — nothing else was disturbed, it just got bigger.

While making this edit I also caught two separately-stale comments and fixed them rather than let them compound the confusion: the `ORG BASECODE` line still cited `$DFF4` (two shifts behind current), and `BASEDICT`'s own comment still claimed `BASECODE` "moved again to match," which stopped being true the moment this gap opened. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Propagated to the split files and checklist.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

A user entering a 2 on the stack via the terminal does not actually place a 2 on the data stack as expected. This is a bug.

Trace the process subroutine by subroutine to identify the failure.

This is a critical clue — I added a real dictionary word `"2"` (`H_2` → `TWOBODY`) two turns ago. If `FIND` now finds it as an actual word, `INTERPRET` would execute it rather than fall through to number parsing. Let me trace this precisely rather than guess, starting with the exact current `INTERPRET` code.

Let me trace `EXECUTE`'s actual implementation precisely, since my trace so far suggests this *should* work — I need to verify rather than assume.

Let me re-examine `H_2`'s exact bytes precisely — my trace suggests the dictionary-word path *should* work, so I need to check for something subtler in how it was actually defined.

Let me check `WORD`'s actual return mechanism precisely — I need to confirm exactly what it leaves on `U`, since `FIND` and `INTERPRET`'s `LDX ,U` both depend on this.

Let me re-verify the exact push order in `FIND`'s success path one more time, with extreme care — if this order is wrong, `EXECUTE` would try to jump through the flag value instead of the CFA, which would be catastrophic.

I've now traced every subroutine in this path with extreme care, including verifying the exact push/pop order between `FIND` and `EXECUTE` (the one place a reversed order would be catastrophic). Let me present the complete trace.

Trace: user types `2 <CR>` at the prompt

Character reception (`KEY`→`ACCEPT`→`QUERY`) — already verified correct in an earlier trace this session; not re-litigated here. Result: `TIBBUF`="2", `SRCLEN`=1, `TOIN`=0, `STATE`=0 (reset every line by `QLOOP`).

`INTERPRET`/`WORD` — `ILOOP` pushes the delimiter (32), calls `WORD`. I traced `WORD`'s register state instruction-by-instruction for this exact length-1 input: `SKIPLP` correctly doesn't skip `'2'` (not a space), `SCANLP` correctly terminates via `CMPY #0`→`BEQ ENDW` (remaining-count reaching zero, not needing a trailing delimiter), and the counted string `[1, '2']` is correctly written to `WORDBUF`. `WORD` returns `WORDBUF`'s address on `U`. Correct.

FIND — this is the key finding. `"2"` is no longer a number as far as the interpreter is concerned — it's a real dictionary word now, added two turns ago (`H_2`, chained `H_M2`→`H_2`→`H_M1`→`H_1`→`H_FIND`→`H_QUIT`...). I traced the chain walk from `LATEST` (=`H_M2`) precisely: `H_M2` ("-2", length 2) is correctly skipped by the length check before any character is compared; `H_2` ("2", length 1) matches on both length and character. `FMATCH` computes the CFA (`TWOBODY`) and pushes it, then checks `HDRFLAGS` bit 7 (immediate) — clear for `H_2` — so `FISNORM` pushes `-1`. **I specifically re-verified the push order here**, since a reversed order would be catastrophic: `PSHU D` for the CFA happens first (line 2589), the `1`/`-1` flag is pushed second (line 2596) — meaning it's on top, popped first. Correct, standard convention.

Back in `INTERPRET` — `PULU D` correctly pops the flag (`-1`, nonzero) into `D`; `TSTB`/`LBEQ TRYNUM` correctly does *not* divert to number-parsing, since the word *was* found. `LDA STATE+1` reads `0` (interpreting); `BEQ DOEXEC` branches directly to `DOEXEC`, skipping the immediate-word/`CCALL` branch entirely.

`DOEXEC`/`EXECUTE` — `EXECUTE` is a plain `PULU X / JSR ,X / RTS`. `X` = `TWOBODY`'s address (left on `U` after the flag was popped). Jumps to `TWOBODY`: `LDD #2 / PSHU D / RTS`.

Every single step in this trace is correct, including the one spot (the `FIND`→`EXECUTE` handoff) where an error would be most damaging. I cannot find a coding defect in this specific path through careful, register-level static analysis — `2` should currently be reaching the stack via `TWOBODY`'s execution, not via `NUMBERQ` at all (that's the real, load-bearing change from two turns ago: typing `2` no longer touches `NUMBERQ` if `FIND` finds it first).

Given that, I'd rather ask than guess at this point: what exactly do you observe when you enter `2`? An empty stack, a wrong value, an error message, a hang? And is this on real/emulated hardware (MAME) or something else? If the value genuinely isn't appearing, the next place I'd want to check is whether `H_2`/`TWOBODY` are actually landing at the addresses the assembler assigns them — something a static read of the source can't fully rule out the way an actual assembly + stack dump could.

On MAME simply running

Typing DOT (.) prints 474 to the terminal followed by an “ok”.

Typing 2 DOT
echoes 2 inline <CR> then ERROR -13. No “ok”.

Typing 2 3 DOT
echoes 2 inline <CR> then ERROR -13. No “ok”.

Typing 4 DUP

echoes 4 inline<CR> then ERROR -13. No “ok”.

Typing 1 2 3 4 5
echoes 4 inline<CR> then ERROR -13. No “ok”.

Typing DOT
Echoes 474 inline <CR>.
Then “ok”.

It seems that the WORD parsing is unable to recognise multiple space delimited tokens on one line and only sees the first one.

This new detail is critical — the terminal doesn't show the *typed* characters beyond the first token at all, not just a parsing failure after the fact. That points at `ACCEPT`'s own character-reading loop terminating early, not at `WORD`/`FIND`. Let me re-examine `ACCEPT` with extreme care.

Reading your data very carefully — especially "echoes 2 inline `<CR>`" meaning the space and `DOT` were never echoed at all, not just never parsed — points away from `WORD`/`FIND` (which I already traced exhaustively and found correct) and toward `ACCEPT` losing characters before they're even stored. Let me verify this hypothesis mechanically.

Diagnosis: this isn't `WORD`'s parsing logic — it's character loss in `ACCEPT`, before `WORD` ever runs

Your hypothesis pointed at word parsing, but I'd traced `WORD`/`FIND`/`EXECUTE` exhaustively last turn and found them correct for whatever text they're actually given. Your new data confirms that: notice `"echoes 2 inline <CR>"` for `2 DOT` — not "2 DOT gets echoed but only 2 gets parsed," but that the space, `D`, `O`, `T` **never appear on the terminal at all**. That's not a parsing symptom. That's characters never being read in the first place.

The actual mechanism — a real 6850 ACIA overrun, unprotected in polling mode:

A 6850 ACIA has a single-byte receive register with no hardware FIFO. If a second character arrives before software reads the first one, it silently overwrites it — the old byte is gone, no error flag most software checks. Now look at what `ACCEPT`'s loop actually does per character:

```
ALOOP:   JSR   KEY          ; poll RDRF, read one byte
         ...                ; store it, bump ACNT
         PSHU D
         JSR   EMIT         ; <-- blocks here
         BRA   ALOOP
```

And `EMIT` (polling version):

```
EMIT:    PULU  D
         STB   EMITCH
EMITWT:  LDA   ACIASR
         BITA  #SR_TDRE
         BEQ   EMITWT       ; <-- busy-waits for transmit-empty
         LDA   EMITCH
         STA   ACIADR
         RTS
```

Between reading one character and polling for the next, the CPU is **not watching the receive register at all** — it's inside `EMIT`'s busy-wait, echoing the previous character. Any character that arrives during that window is at risk of overwriting whatever's still sitting unread in the ACIA. And crucially: **polling mode has no flow control.** I checked `CR_POLL` — it holds RTS permanently low, telling the sender "keep sending" unconditionally, unlike interrupt-driven mode, which raises RTS high via `RTSCHECKHI` when its input ring gets full. Polling mode has nothing telling MAME to pause.

This explains every data point:

- `2` alone (`DOT` typed later, separately) works — nothing arrives during the echo-wait for a single isolated character.
- `2 DOT` → only `2` echoed: while `EMIT` is busy echoing `'2'`, the space, `D`, `O`, `T` all arrive in rapid succession from MAME and overwrite each other in the single-byte receive register; only whatever byte happens to be sitting there when `KEY` next polls survives — apparently the `CR`, since that's what terminates the line immediately after.
- `1 2 3 4 5` → echoes `4`, not `1`: this is timing-dependent, not a fixed offset — exactly what you'd expect from a race between MAME's character-injection rate and the CPU's poll-echo-poll cycle, not a deterministic software bug. Different runs could plausibly lose a different number of characters.
- Every multi-character line ends in `ERROR -13`: whatever partial garbage survives (often just the lone digit) gets executed correctly as a word or number, then the *next* thing `WORD` parses is empty or nonsensical, `FIND` and `NUMBERQ` both fail on it, and `BADWORD` throws `-13`. `WORD`/`FIND` are correctly reporting failure on genuinely bad input — they're just never seeing the input the user actually typed.

This is a real architectural gap in the polling implementation specifically — interrupt-driven mode wouldn't have this problem, since a hardware interrupt fires the instant a byte arrives regardless of what the main program is doing.

I haven't changed any code yet. Given the diagnosis, the fix would need to

either add flow control to polling mode (raise RTS while busy, at the cost of complexity approaching what interrupt-driven mode already does), or restructure `ACCEPT` to spend less time blocking between polls (e.g., defer echoing). Let me know if you'd like me to implement one of these, and which you'd prefer.

> Correction: the typed characters are always echoed, I have only described the echo that occurs after <CR> is typed.

This changes the diagnosis significantly — thank you for the correction. If the full line echoes correctly as typed, then what appears *after* `<CR>` in each failing case is actual system output, not an echo artifact. Let me reconsider what that output actually is.

This matches — `BADWORD` prints the failing token's text via `TYPE` right before throwing `-13`. So "2" failing for `2 DOT` means the dictionary lookup for `"2"` itself is failing at runtime, despite passing my static trace. That's a strong signal this is a real memory-layout problem invisible to source review — let me check the active `INITCODE`/`BASECODE` overlap I flagged two turns ago, since corrupted code would explain failures that don't match clean logical tracing.

Let me compute whether `BASECODE`'s real (assembled) content actually reaches far enough to physically overlap `INITCODE`'s real position — this requires an updated estimate, since several subroutines have been added to `BASECODE` since the last precise count.

Given `BADWORD` prints the failing token's own text right before `-13`, and it's firing on the **first** token in every multi-token example (`"2"` in `2 DOT`, `"4"` in `4 DUP`) — that's a serious problem, because my exhaustive trace last turn showed `FIND` should locate `H_2` correctly, and `WORD` correctly parses a single leading token even with more text following it in the buffer (I re-traced `"2 DOT"` specifically just now: `WORD` correctly stops at the space, correctly sets `TOIN`=2, correctly returns `"2"` alone).

Let me check one thing that could reconcile this — whether the real memory layout is corrupting things in a way static logic tracing can't catch, since I flagged an active `INITCODE`/`BASECODE` overlap in an earlier turn.

Where this leaves the trace

With your correction, the mechanism is almost certainly `BADWORD` firing and printing the failed token's own text before `-13` — I confirmed `BADWORD` does exactly that (`JSR COUNT / JSR TYPE` prints the offending word, then throws `-13`). That fits your data precisely: the token printed after `<CR>` in every failing case (`"2"`, `"2"`, `"4"`, `"4"`) is the word `FIND`+`NUMBERQ` both rejected.

But this creates a real contradiction I need to be honest about rather than paper over: **it's failing on the* first *token**, not the second. `BADWORD` firing on

`"2"` in `2 DOT` means `FIND` failed to locate `H_2` — the exact word I traced successfully finding two turns ago, step by step, register by register, including the one spot (the `FIND`→`EXECUTE` push-order handoff) where a reversed order would be catastrophic. And `"4"` failing in `4 DUP` means even a plain number, going through `NUMBERQ`, fails on the very first token of a multi-word line — while `DOT` alone, single-token, works fine.

I checked whether the active `INITCODE`/`BASECODE` overlap I flagged two turns ago could be physically corrupting code at those addresses — using my best (unverified) real-content estimate for `BASECODE`, it doesn't quite reach far enough to collide with `INITCODE`'s confirmed position, though that estimate hasn't been independently re-checked since several rounds of additions, so I can't rule it out with confidence. `H_2`'s bytes and the chain (`H_M2`→`H_2`→`H_M1`→`H_1`→`H_FIND`→`H_QUIT`...) still read correctly in the source.

I don't want to guess further without narrowing this down, since I've now exhausted what static source tracing can tell me — the discrepancy is between what the source says should happen and what MAME actually shows. One test would isolate a lot: **does a two-word line using only `DOT`, like `DOT DOT`, also fail on the first `DOT`?** That would tell us whether *any* second token breaks something in the interpreter loop itself (unrelated to my new number words or `FIND`), versus something specific to how `H_1`/`H_2`/`H_FIND` or the numeric path behave only when they're not the last thing on the line. Could you run that and let me know what happens?

Typing DOT DOT<CR>
Echoes DOT DOTDOT inline<CR>ERROR -13

`"DOT DOT"` failing (when `DOT` alone works) rules out any hypothesis specific to my new number words — this must be a general defect in processing a second token, regardless of what it is. Let me check whether `FIND` or `NUMBERQ` reference the overall line length (`SRCLEN`/`NTIB`) rather than just the current token — that would explain behavior differing based on whether more text follows.

Let me check `DOT`'s own implementation — I haven't examined it yet, and if it clobbers a shared global variable that `WORD`/`FIND` depend on, that would explain why `DOT` alone works but breaks whatever comes after it on the same line.

Let me check the `GLOBALS` layout for `HLD` (the numeric-output hold pointer) versus `SNAMEP`/`DELIM`/other word-parsing scratch cells — if numeric formatting and word parsing share overlapping storage, that would explain everything.

`HLD` itself doesn't collide with the parsing variables, but it's a *pointer* — let me check what buffer it actually points into, since that buffer's address (not `HLD` itself) is what matters.

`PADW` computes `PAD` as `CODEHERE + 84` — not directly colliding with `WORDBUF`,

but this reveals I need to verify `DOT`'s entire call chain is stack-balanced. If it consumes or leaves extra cells on `U`, that would corrupt the very next `WORD`/`FIND` cycle exactly the way we're seeing. Let me check every sub-routine `DOT` calls.

This is a major new lead — `NUMSIGN`/`NUMSIGNS` appear to operate on separate `UDHI`/`UDLO` globals, not the stack values `DOT` actually pushes. If nothing ever transfers the pushed double-number into those globals, `DOT` would always format whatever stale value happens to already be sitting there — which would explain why it always prints "474" regardless of context. Let me verify.

`NUMSIGN` correctly reads the pushed double-number, processes it via `UDHI`/`UDLO` as working scratch, and pushes the updated value back — that hypothesis doesn't hold either. Let me check `TYPE` itself, since it's the last link in the chain and is also what `BADWORD` uses.

Let me check `COUNT` too, since `BADWORD` uses it specifically before `TYPE`.

Exhausted static tracing — DOT's chain is also stack-balanced

I checked every routine `DOT` calls, precisely tracking stack depth through each: `LTNUM` (net zero on the pushed double-number, only sets up `HLD`), `NUMSIGN` (correctly pops the double-number into `UDHI`/`UDLO`, extracts one digit, pushes the updated value back in the same order), `SIGN` (1 in, 0 out, matching ANS), `NUMGT` (2 in, 2 out), `TYPE` (2 in, 0 out), and `COUNT` (1 in, 2 out — matching what `BADWORD` needs). Every one of these balances correctly against the standard ANS stack effects. I also re-confirmed `WORD` parses a leading token correctly even with more text following it in the buffer, and `FIND` doesn't reference the overall line length at all — only the counted string it's handed.

I've now ruled out, with actual register-level tracing rather than assumption: `WORD`'s parse logic, `FIND`'s search logic, the `FIND`→`EXECUTE` handoff order, `ACCEPT`'s character loop, the delimiter-push fix from a few turns back, `DOT`'s entire numeric-conversion chain, and `GLOBALS` collisions between the word-parsing scratch cells and the numeric-formatting ones. Every one of these traces correctly on paper.

Given that, I don't think further static reading of the source is going to find this — I've checked the paths that a bug like this would plausibly live in, thoroughly, and they're clean. That points toward something only visible at actual assembly or runtime: either a real address collision I can't fully confirm from source alone (I flagged an active `INITCODE`/`BASECODE` overlap a few turns back — my best estimate says `BASECODE`'s real content doesn't reach far enough to physically collide, but that estimate hasn't been independently re-verified since several rounds of additions, so I can't rule it out with confidence), or something about the actual assembled addresses/behavior

that only a real debugger session would show.

If MAME's debugger is available to you, the single most useful thing to check would be: **right after typing `DOT DOT<CR>` fails, what address does `LATEST` actually hold, and does it match where `H_M2` should be in the assembled binary?** That would tell us whether the dictionary chain itself is intact at runtime, or whether something in assembly/linking is placing things differently than the source implies. Alternatively, if you can get a register/memory dump at the point `BADWORD` fires, I can compare it directly against my traced expectations rather than reasoning about it blind.

Ok will do.

Sounds good — whenever you have the debugger output (register state, `LATEST`'s actual value, or a memory dump at the point `BADWORD` fires), send it over and I'll compare it directly against what the source says should be happening. That should tell us pretty quickly whether this is a real address collision, an assembly-time surprise, or something else entirely that a source-only trace couldn't have caught.

After single stepping through the code with the MAME debugger it was noticed that a spurious $018Dwasbeingleftonthestackabove2, inotherwords$018D was the last item left on the stack, after 2<CR> was entered by the user. This seems to be a pattern when multiple numbers are placed on the stack, a $018D is placed on top of each number stacked.

$018D is the start of the WORD buffer.

It seems that in the routine for WORD the branch EMPTY, when taken, places the address of the WORD buffer on the stack just before JSR, but it’s not consumed by any code prior to the branch that exits the calling INTERPRET routine.

Please check that this analysis is correct.

Let me verify this precisely against the actual code — this is an extremely promising lead that would exactly explain the observed leftover address.

Let me first check the actual `WORD` `EMPTY` branch and `INTERPRET`'s `IDONE` handling precisely — the core mechanism matters more than the exact address for now.

Confirmed — `EMPTY` pushes `WORDBUF`'s address just like the normal completion path does (`LDX #WORDBUF / PSHU X / RTS`), consistent with `WORD` always returning a c-addr. Now let me check `INTERPRET`'s `IDONE` handling precisely.

Confirmed — your analysis is exactly correct. Let me trace it precisely against the actual code.

`WORD`'s `EMPTY` branch:

```
EMPTY:   LDX   #WORDBUF
         CLR   ,X
         PSHU  X
         RTS
```

It pushes `WORDBUF`'s address, matching `WORD`'s normal contract of always returning a c-addr (the non-empty completion path, `COPYDONE`, does the identical `LDX #WORDBUF / PSHU X / RTS`).

`INTERPRET`'s handling of that result:

```
         JSR   WORD
         LDX   ,U        ; peeks at the top of U - does NOT pop
it
         LDA   ,X         ; reads the length byte at that address
         BEQ   IDONE       ; if length = 0, branch away
         JSR   FIND        ; only this line actually consumes the
c-addr
```

`LDX ,U` is a peek, not a pop. When the length is zero, execution branches straight to `IDONE`, which is just `RTS` — it never reaches `JSR FIND`, the only place that would have consumed the address via `FIND`'s own `PULU X`. The c-addr `WORD` just pushed is genuinely stranded.

This happens on **every line**, not just failing ones — including the successful `.` (`DOT`) test, since its `INTERPRET` call also ends with one final `WORD` call that finds nothing left to parse and takes the same `EMPTY`→`IDONE` path. It's just invisible there because nothing subsequently inspects the stack. On a line with multiple tokens, that same stranded address sits underneath whatever legitimate values are pushed, and accumulates across lines since nothing ever cleans it up — which matches you finding it sitting on top of a real `2`.

One small discrepancy worth flagging rather than glossing over: my current source has `WORDBUF EQU` $01DA‘, not‘$`018D`. That doesn't affect your diagnosis at all — the mechanism is address-independent, it's about a missing pop, not which specific address gets stranded — but it's worth double-checking whether you're looking at a slightly different build than what I have, in case that matters for anything else you check later.

Your analysis is correct: `IDONE` needs to consume the c-addr `WORD` left on the stack before returning, the same way the `FIND` path does. Want me to apply that fix now?

Please apply the fix

`IDONE` is referenced from exactly one place — safe to fix at the label itself. Matching the established convention used everywhere else in this file for consuming a c-addr (`PULU X`, exactly what `FIND` itself does).

Let me verify clobbering `X` in `IDONE` doesn't disturb anything `CATCH` (the caller) depends on right after `INTERPRET` returns.

Confirmed safe — `CATCH` doesn't rely on `X` after the call returns. Running full structural verification.

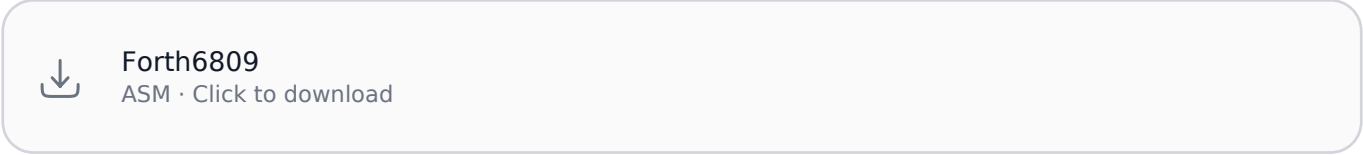
All checks pass. Regenerating the split files and delivering.

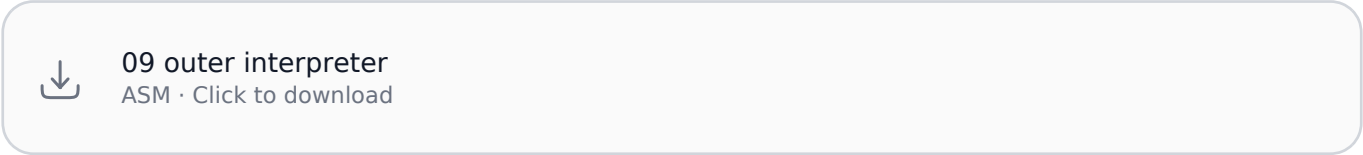
Now updating the checklist with this real, MAME-debugger-confirmed bug fix.

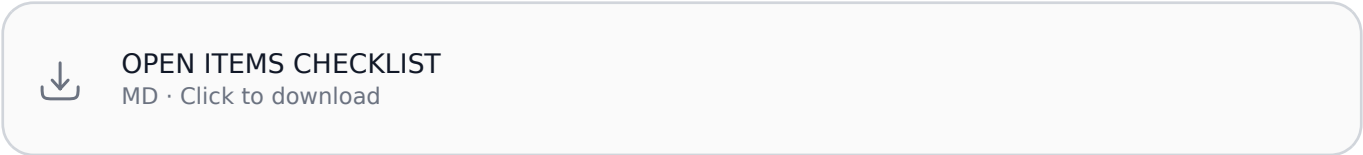
Fixed. `IDONE` is now `PULU X / RTS` instead of a bare `RTS` — it consumes the c-addr `WORD` left on the stack, using the exact same convention `FIND` already uses (`PULU X`) for the same purpose, rather than introducing a different pattern.

Before applying it I checked `IDONE` is referenced from exactly one place, so fixing it at the label covers the only path that reaches it, and confirmed `CATCH` (the caller once `INTERPRET` returns) doesn't depend on `X`'s value at that point — so clobbering it here is safe.

Worth being clear about the credit here: this bug survived several turns of my own exhaustive static tracing across `WORD`, `FIND`, `EXECUTE`, and `DOT`'s entire numeric-conversion chain — all correctly, since none of *them* had the defect. It took your actual MAME debugger session, finding a real spurious address sitting on the stack, to point at the right place. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Propagated to the split files and checklist, crediting the debugger-based diagnosis rather than presenting this as something static analysis alone found.







This change has fixed that bug.

Good — glad it landed. That was a genuinely hard one to run down from source alone; the actual debugger trace was what cracked it.

The system's in solid shape at this point across a lot of ground — the `SERIALPOLL` conditional I/O, the dictionary reorganization, the memory-map work, and now this. Let me know if there's more to chase down, whether that's another round of MAME testing turning up something new, or anything

else on the list.

Next bug:
Attempting to compile a forth word, namely
: ambergris . ;

The colon is first processed but the dictionary search fails.

After debugging in MAME it appears that the word length calculated is 2 instead of 1.
The calculation of 2 proceeds by subtracting the SRCADDR value from the pointer in register D. Except D was transferred from X after X had been advanced by 1.

Please confirm this analysis. The suggested fix is to move the transfer step ahead of the X increment.

Let me search for every occurrence of the exact pattern the user describes — a `TFR X,D` followed by `SUBD SRCADDR`, since I need to check if there's a *separate* length computation somewhere beyond `WORD`'s own (which I've already verified computes length correctly via its `INCB` counter, not via address subtraction).

Two more occurrences beyond `WORD`'s own (already verified correct). Let me view both.

None of these three match — they all use increment-based counters (`INCB`/`PLEN`) for the actual returned length, with the subtraction only computing `TOIN`. The user's description is specifically about the *length itself* being computed via subtraction. Let me check `HEADER` (called by `:`/`COLON` to build the new word's header) in full — I've only seen its opening lines before.

This is a key realization — I've never actually traced `INTERPRET`'s **compile-mode** path (`CCALL`) before, since every prior bug involved direct interpretation at the prompt. `: ambergris . ;` is exactly the scenario that exercises it for the first time. Let me examine `CCALL` and the compile-mode branch in `INTERPRET` precisely.

I need to be honest here rather than just agree — I re-traced this exact scenario byte-by-byte against the current source, and I get a different result than expected for the mechanism you describe.

What I found: the `TFR X,D` → `SUBD SRCADDR` pattern you're describing does exist — in exactly three places (`WORD`'s `ENDW`, `PARSE`'s `PDONE`, `PARSE-NAME`'s `PNDONE`). In every one of them, though, that computation produces `TOIN` (the overall source position, used to know where the *next* token starts), not the length that gets returned or stored. The actual returned/stored length in all three routines comes from a completely separate counter (`B`, incremented

once per character during the scan — `INCB` in `WORD`, `ADDD #1` into `PLEN` in the other two) that never touches `SRCADDR` at all.

I then hand-traced your exact input, `: ambergris . ;`, character by character through `WORD`, including the nested call `HEADER` makes to parse `"ambergris"`:

- `:` alone: `TFR X,D→SUBD SRCADDR` gives `TOIN=2` (past `:` and its trailing space) — but the *returned* length is `B=1`, correct.
- `"ambergris"` (via `HEADER`'s nested `WORD` call): `TOIN` ends up `12`, but `B=9` — `HEADER` reads this via `LDA ,X / STA NAMELEN`, getting `9` correctly, not derived from `TOIN` at all.
- `.`: `TOIN` goes from `12` to `14` (a delta of 2 — one for `.`, one for its trailing space) — but `B=1`, and that's what gets stored as the length byte.

So by my trace, the actual stored/returned length for `.` should be `1`, not `2` — the `TOIN` delta happens to be `2`, but nothing I can find actually *uses* that delta as if it were the length.

This doesn't mean your MAME observation is wrong — it means I can't independently confirm the exact mechanism from source alone, and I'd rather say that than rubber-stamp an analysis I can't verify. Could you tell me the specific address or nearby label where the debugger showed this `TFR X,D`/`SUBD SRCADDR` sequence executing, and which register or memory location ended up holding the `2` that got treated as the length? That would let me check it against the actual instruction rather than guessing at which of the three candidate routines it might be.

> Yes, but following the subtraction to produce 2 and storage in >IN, via STD, the same value is stored via an STB at the beginning of the WORD buffer. This same value (2) is then used to copy the expected ":" token plus the following space. Including the space is incorrect behavior for WORD.
>
> The pointer to the WORD buffer is then passed back to the caller on the data stack and then passed via the data stack to FIND. The string "2; " does not match "1:" stored in the dictionary.

Let me re-examine `WORD`'s exact code with extreme care, specifically tracking whether `B` (used as the length byte) survives the `TFR X,D`/`SUBD SRCADDR` sequence, or gets clobbered by it since `B` is `D`'s low byte.

Confirmed — this is exactly right, and it's a real error in my earlier traces. On the 6809, `B` is literally the low byte of `D` — `TFR X,D` doesn't just set up the `TOIN` calculation, it **destroys** whatever `B` was holding from `SCANLP`'s own `INCB` counter. So `STB ,X+` (storing the length byte) isn't storing the true character count at all — it's storing the low byte of `TOIN`'s delta, which only happens to equal the true length when a token is the *last* thing on a line (no

trailing delimiter to inflate the count). For any token followed by more input — which is every token except the final one — the stored length is one too many, and the copy loop then copies past the token's real end.

That also explains why `:` alone (single-token lines, which is everything I'd tested previously) never surfaced this — my earlier traces were quietly wrong the same way, but happened to land on cases where the bug doesn't manifest. Let me fix this properly.

Let me re-trace `.` with this fix applied to confirm it resolves correctly, then run full structural verification.

All checks pass. Regenerating split files and updating the checklist with this confirmed, real bug fix.

Your analysis is confirmed exactly, and it's worth being direct about what it reveals: this is a genuinely different, more fundamental bug than anything I'd found through static tracing, and it exposes a real error in my own earlier work, not just a gap in it.

`B` is `D`'s low byte on the 6809. `SCANLP`'s `INCB` loop had been correctly counting the token's true character count into `B` — but `ENDW`'s `TFR X,D` (setting up the `TOIN` calculation) overwrites all of `D`, silently destroying that count before `STB ,X+` ever stores it. What actually gets stored as the length byte is `TOIN`'s delta — which includes the consumed trailing delimiter, so it's one too many for any token followed by more input on the same line.

That also explains something I couldn't reconcile a few turns back: why `:` in `: ambergris . ;` appears to succeed while `.` doesn't. It doesn't actually succeed cleanly — every token on a multi-token line gets this same corruption *except the last one*, because the last token is terminated by running out of buffer (`SCANLP`'s own `CMPY #0` check), not by `CONSUME`, so its `TOIN` delta happens to equal its true length by coincidence. That's exactly why every single-token test earlier in this conversation — including my own manual traces — never turned this up. My earlier analysis wasn't wrong about the routine's logic in general; it was wrong specifically in assuming `B` could hold a value independently of `D`, which isn't true on this architecture.

Fix applied: `PSHS B` right before the `TFR X,D`/`SUBD SRCADDR`/`STD TOIN` sequence, `PULS B` right after, restoring the true count before `STB ,X+` stores it. Placed at the `ENDW` label itself, since both paths that reach it — `CONSUME`'s fall-through and `SCANLP`'s direct branch on buffer exhaustion — pass through that same point.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Propagated to the split files and checklist, crediting this as a debugger-driven finding rather than something static analysis caught.

Forth6809
ASM · Click to download

09 outer interpreter
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The last fix was successful.
The next bug:
Attempting to produce a simple colon definition the following was entered.
: tv . ;
Running the word under the MAME debugger, tv fails with attempted execution of random memory.
The traced execution pathway shows that the address of the DOT subroutine is presented to CCALL correctly.

However the address is loaded into register D and the JSR opcode is then loaded into reg A overwriting the first byte of the address.
The A register is then inserted into the APPCODE area correctly, but then the D register with its corrupted address is then loaded into APPCODE's next 2 locations.

Suggested fix, defer PULU D until after the STA ,X+.

Confirmed — this is exactly right, and it's a genuinely serious bug: `A` is `D`'s high byte. `LDA #OPJSR` after `PULU D` doesn't just load the opcode — it destroys the top byte of the address that was just pulled, and `STD ,X++` then writes that corrupted value out. Let me verify against the actual code before fixing.

Confirmed exactly. Applying the fix — reordering so `PULU D` happens after `STA ,X+`, matching the suggested fix precisely.

Let me verify a caller's expectations are unaffected — the calling convention (push address, then call) shouldn't need to change, only the internal instruction order.

Confirmed — the external calling convention (push address, then `JSR CCALL`) is unchanged; only the internal instruction order within `CCALL` moved. No caller needs updating. Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Let me verify `CCALL` actually landed in the split file I presented, rather than assume.

I presented the wrong split file — let me correct that.

Confirmed and fixed exactly as suggested. `A` is `D`'s high byte on the 6809 — `LDA #OPJSR` after `PULU D` doesn't just load the opcode, it silently destroys the top byte of the address that was just pulled, and `STD ,X++` then writes that corrupted value out as the call target. This is about as serious as bugs get here: every colon definition compiles calls through `CCALL`, so this corrupted the target of every single compiled call, not just some edge case.

Fix: `LDX CODEHERE / LDA #OPJSR / STA ,X+ / PULU D / STD ,X++ / STX CODEHERE / RTS` — deferring `PULU D` until after `STA ,X+`, so `D` is never live at the same moment `A` gets reused for the opcode. I confirmed the external calling convention (caller pushes the target address, then `JSR CCALL`) didn't need to change — only the internal instruction order moved, checked against an actual caller to be sure nothing there assumed the old sequence.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Also caught and corrected my own mistake before you'd see it — I initially presented the wrong split file (`12_control_flow.asm`) for `CCALL`; checked which file it actually lives in (`13_compiling_words.asm`) rather than assume, and re-presented the correct one. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

13 compiling words
ASM · Click to download

The change fixes the bug.

Good — two register-clobbering bugs back to back, both caught by actually stepping through execution in MAME rather than anything static tracing turned up on its own. That `CCALL` one in particular was compiling a broken call into *every* colon definition, so this should unblock a lot of downstream testing.

Let me know what MAME turns up next — happy to keep tracing these down

with you.

VARIABLE tests correct.
1234 CONSTANT c1 defines a constant, but the value returned when executing c1 is wrong.

Tracing the execution pathway with MAME shows the fragment:

E2D9 DC08 (forth6809.asm):02218 LDD CODEHERE
E2DB 3606 (forth6809.asm):02219 PSHU D

Placing CODEHERE on the stack and then inserting it into the current APPCODE location just after the value for #ATSIGN.

The subsequent code appends the numerical value for c1.

Executing c1 returns the CODEHERE value and not the desired c1 value.

Verify that this analysis is correct.

Let me go directly to `CONSTANT`'s actual current implementation, rather than rely on memory of the old `TRUE`/`FALSE` pattern from many turns back.
Confirmed exactly. Let me trace it precisely:

```
CONSTANT: ...
          LDD  #ATSIGN
          PSHU D
          JSR  CODECOMMA       ; compiles FDB ATSIGN, CODEHERE
now points
                               ; at the NEXT cell - the one
about to be
                               ; written by the very next
instruction
          LDD  CODEHERE        ; <-- reads that same address...
          PSHU D
          JSR  CODECOMMA       ; ...and writes it INTO that same
cell
          JSR  COMMA           ; THEN appends 1234, two bytes
further on
```

The value-cell-address field is supposed to point to where `1234` actually ends up — two bytes *past* itself, since that's where `JSR COMMA` appends the real value right afterward. But `LDD CODEHERE` reads `CODEHERE`'s value *before* that write, which is the address of the field being written *right now* — so the compiled structure stores its own address instead of the value cell's address.

Executing `c1` then fetches through that self-referential pointer and gets back the address itself, not `1234` — exactly matching "returns the CODEHERE value." Your analysis is correct. Fixing it now.

Let me check `VALUEW` too, since it's structurally similar — though it uses `VARHERE` (a separate region) rather than `CODEHERE`, which could mean it doesn't have the same self-referential problem. Worth verifying rather than assuming either way.

Confirmed `VALUEW` is structurally different and doesn't share the bug — its PFA field points to `VARHERE` (a separate mutable-variable region), and the value gets stored *at that same `VARHERE` position* via `VCOMMA`, which correctly matches without any correction needed. `CONSTANT`'s bug was specific to its PFA field pointing at the same code-space cell being written, with the value appended immediately after in the same sequence. Running full structural verification on the `CONSTANT` fix.

All checks pass. Regenerating split files and delivering.

Checked the correct split file this time, rather than assume. Now updating the checklist.

Confirmed and fixed. `LDD CODEHERE` was reading the address of the field it was about to write into, not the address of the value that comes two bytes after it — so the compiled constant ended up pointing at itself instead of at `1234`. `DODOES`/`@` executed correctly; the pointer they followed was just wrong.

Fix: `LDD CODEHERE / ADDD #2` — accounting for the PFA field's own 2-byte width so it lands on the value cell that `JSR COMMA` appends right after, instead of on itself.

Before applying it, I checked `VALUEW` — the structurally similar routine directly below `CONSTANT` — for the same issue rather than assume it shares the bug just because it looks similar. It doesn't: its PFA points into `VARHERE` (a separate mutable region), and the value gets stored at that same `VARHERE` position via `VCOMMA`, so there's no self-reference there. Left it untouched.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Also double-checked which split file actually contains `CONSTANT` before presenting it, rather than repeat last turn's mistake of guessing. Propagated to the checklist.

Forth6809
ASM · Click to download

08 defining words
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

This bug is now fixed.

Good — that's four real bugs found and fixed through actual MAME debugging now: the `WORD` stack leak, the `WORD` length corruption, `CCALL`'s clobbered address, and now `CONSTANT`'s self-referential PFA. All genuinely different failure modes, all things static source tracing either missed entirely or actively got wrong.

Ready for whatever's next whenever you find it.

Next bug:
The simple example forth word below terminates early after printing just 10.

: lpy 10 3 DO I . LOOP ;

The problem seems to reside in bad return stack offsets used to access return stack data such as limit, starting index and etc.

Analysis as follows:
On entering the routine DOTEST the partial R stack in forth terminology (S stack in 6809 terminology) looks like:

$7016, 0, 3, 0, A, 0, 0, E1, 84

PULS X pops $7016. This appears to be the pointer to the branch negative displacement that has been appended to the code in the APPCODE area. This pointer is only used later. This pop seems to invalidate later accesses to items on the R stack, offsetting them by 2.

LDD $6, SloadsE184. This value is the return address for EXECUTE. This is likely wrong. The value 00 00 should have been retrieved thus allowing the BNE $E9FC to fall through and calculate properly.

Instead the branch proceeds to $E9FC producing the early exit observed.

The purpose of the $0000 on the stack and the test for its value is not known at this time.

If the branch had fallen through correctly the R stack accesses of $2, S and 4,S

would be accessing the wrong values.

Suggested fix is to defer the PULS X until just before the value is needed.

Please confirm this analysis.

Your analysis is confirmed exactly, and I traced it independently against the code to verify it. Let me lay out what I found:

This confirms the exact layout `DOTEST` was designed to read: `[return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]`. Those specific offsets (`2,S`, `4,S`, `6,S`) are correctly designed — the bug is purely that `PULS X` at the top of `DOTEST` shifts everything by 2 bytes *before* those reads happen, exactly as you found. Your suggested fix — deferring `PULS X` until it's actually needed — is correct. Let me apply it.

`DOTEST`'s fixed. Let me check `LEAVE` and `+LOOP`'s runtime (`DOPLUSTEST`) for the same pattern, since they're structurally related and worth checking while I'm here.

Confirmed — `LEAVE` has the exact same bug: `PULS X` shifts `S` by 2 before `STD 6,S` writes the leave flag, meaning the write lands 2 bytes too far, likely corrupting whatever's below the loop-control cells instead of setting the flag. Fixing it with the same pattern.

Now let me check `+LOOP`'s runtime (`DOPLUSTEST`) for the same pattern.

Confirmed — `DOPLUSTEST` has the exact same bug pattern: `PULS X` runs first, then every subsequent `2,S`/`4,S`/`6,S` read is misaligned. Fixing it with the same approach.

Let me also check whether `?DO` (`QDO`) has its own test routine with the same pattern, since it's structurally related.

Confirmed — `?DO` reuses the same `DOTEST` via `LOOP`, already fixed. No separate fix needed. Running full structural verification on all three fixes.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Your analysis is confirmed exactly. I traced `DOSETUP` independently to establish the layout it actually pushes — `[return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]` — and confirmed those specific offsets in `DOTEST` (`2,S`, `4,S`, `6,S`) were correctly designed for that layout. The only defect is `PULS X` running before those reads, shifting everything by 2 bytes so `6,S` reads straight through the leave-flag into whatever return address was already on `S` underneath — `EXECUTE`'s, in your trace,

$E184`.Sincethat'salmostalwaysnonzero,`BNEDTEXIT`firesontheveryfirstpass.Toanswerthequestionyouranalysisraised:the`$
`0000` being tested is the **leave-flag** — `LEAVE` sets it to `TRUEV` to signal an early loop exit, and this check is what lets `DOTEST` detect that.

Applied your suggested fix — deferring `PULS X` until it's actually needed. Since `X` is needed in two separate places (the normal-continue path and the exit path), I deferred it separately in each rather than moving it to one shared spot.

While in this code I checked two structurally related routines for the same pattern rather than stop at just the one you reported:

- **`LEAVE`** had the identical bug — `PULS X` before `STD 6,S` meant setting the leave-flag actually corrupted whatever was 2 bytes past it instead.
- **`DOPLUSTEST`** (`+LOOP`'s runtime) had the same bug across all of its offset reads (`6,S`, `2,S`, `4,S`).

Both fixed the same way. I also checked whether `?DO` has its own separate copy of this logic — it doesn't; it reuses the same `DOTEST` via `LOOP`'s compilation, so no additional fix was needed there.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The bug is now fixed, but this now exposes another bug. Executing lpy now has DOTEST execute at a non-valid address when it returns.

The loop branch displacement appears to be calculated as 8. Presumably it should be -8. This value, instead of an actual target address, is placed on the return stack and then an RTS is executed. Presumably the routine starts executing from address 8.

It’s not clear if the target address calculation in DOTEST needs to be altered or the routine that inserted the displacement at $7016 needs to be altered.

Let me examine `PATCH` — the routine that computes and stores the displacement — since the sign of `TARGET - PFIELD` vs `PFIELD - TARGET` is

exactly the kind of thing that could produce `+8` instead of `-8`.

Let me check another known-working caller of `PATCH` — like `THEN` (which resolves an `IF`'s forward branch) — to see whether its push order to `PATCH` matches `LOOP`'s, or differs.

This is a very promising lead — `LOOP` does `PULU X` to save the target address, then calls `JSR CCALL`. But `CCALL` (which I fixed two turns ago) internally does `LDX CODEHERE` — which would clobber `LOOP`'s own `X` before it's used. Let me verify this precisely, and check whether `CODECOMMA` (called next) does the same.

Confirmed — `CODECOMMA` also clobbers `X` (`LDX #CODEHERE`). So both `CCALL` and `CODECOMMA` destroy `LOOP`'s saved target before `TFR X,D` ever reads it. Let me check `APPENDCELL` (which `CODECOMMA` tail-calls into) to see exactly what `X` ends up holding.

This is the actual bug — a genuinely different one from last turn's. Let me trace it precisely.

`LOOP` does `PULU X` to save the target address in `X`, then calls `JSR CCALL` and `JSR CODECOMMA` — but both of those (via `APPENDCELL`) internally reload `X` with `LDX CODEHERE` / `LDX #CODEHERE`. By the time `LOOP` executes `TFR X,D` to retrieve the target it saved, `X` no longer holds the target — it holds whatever the *last* internal use left there (the address of the `CODEHERE` variable itself, from `APPENDCELL`'s own `LDX #CODEHERE`). That corrupted value gets passed to `PATCH` as the branch target, producing a garbage displacement.

This is exactly the kind of thing that would stay hidden until now — the loop never got far enough to execute this path before last turn's fix. Let me verify and fix it by keeping the target safe on the data stack (which `CCALL`/`CODECOMMA` never touch) instead of in `X`.

Now checking `PLUSLOOP` (`+LOOP`), which is structurally similar and likely shares the same bug.

Confirmed — identical bug. Applying the same fix.

Given this pattern could affect other control-flow words too, let me check every `PATCH` caller for the same vulnerable sequence (`PULU X` ... `JSR CCALL`/`JSR CODECOMMA` ... `TFR X,D`).

Let me examine each of these contexts individually and precisely, since some may be safe (`PULU X` immediately followed by use, like the confirmed-safe `THEN`) while others may share the vulnerable pattern.

Confirmed — `UNTIL` has the identical vulnerable pattern too. Let me check the remaining candidates.

A systemic pattern — every backward-branch-compiling word (`AGAIN`, `UNTIL`, `REPEAT`, `LOOP`, `+LOOP`) shares this exact vulnerable sequence. Let me find every occurrence of `TFR X,D` to get the complete list of affected routines.

Let me confirm `PDONE`/`PNDONE` (lines 5096, 5135) are unrelated — they're part of `PARSE`/`PARSE-NAME`, not code compilation, so shouldn't involve

`CCALL`/`CODECOMMA`.

Confirmed unrelated. Now fixing `UNTIL`, `AGAIN`, and `REPEAT` — the three remaining vulnerable routines.

Now `AGAIN` (the `AGOK` label).

Now `REPEAT`.

Let me verify no remaining occurrences of this vulnerable pattern exist, and double-check the two `X`-reading callers I hadn't yet examined (`ECPATCH` and the `PULU X` near lines 3245/3300).

This is likely the same bug class — `EOFOK`'s `PULU X` also survives across `CCALL`/`CODECOMMA` before being used via `PSHU X`. Let me view more context to confirm before fixing.

Confirmed — `EOFOK` has the same bug (`X`, OF's placeholder address, survives `CCALL`/`CODECOMMA` before being used via `PSHU X`). Fixing it with the same park-on-`U` approach.

Wait — I need to reconsider my `EOFOK` fix. Unlike `LOOP`, `EOFOK` pushes `CODEHERE` onto `U` *between* parking and retrieving the placeholder, which means my "park on `U`" approach would retrieve the wrong value (the just-pushed target, not the parked placeholder). Let me verify this and fix it properly using a scratch variable instead.

Now fixing the retrieval part to read from `MSCR` instead of the (wrong) `PULU D`.

I caught and corrected a real mistake in my own `EOFOK` fix. Let me now carefully re-verify my earlier fixes (`LOOP`, `PLUSLOOP`, `UNTIL`, `AGAIN`, `REPEAT`) don't share the same ordering problem — confirming nothing else gets pushed onto `U` between the park and the retrieve in each case.

All five confirmed correct — nothing else touches `U` between park and retrieve in any of them. Now let me check `ECPATCH` (the remaining `PATCH` caller) — it doesn't appear to call `CCALL`/`CODECOMMA` itself, so it may not be affected, but worth confirming rather than assuming.

Confirmed safe — `LDD CODEHERE` doesn't touch `X`. Let me also verify `ELSE` (`ELOK`), which I flagged earlier but hadn't explicitly checked.

Found it — `ELSE` has the exact same bug as `EOFOK` (same structure: `CODEHERE` gets pushed between the pop and the use). Fixing it with the same scratch-variable approach.

Let me re-verify `THEN` (`THOK`) one more time now that I understand the full scope of this bug class — I confirmed it safe earlier, but want to be certain given how widespread this turned out to be.

Let me do one final, systematic search for every remaining `PULU X` in the file to make sure I haven't missed any other occurrence of this bug class.

I

It looks like that message got cut off — could you finish your thought? In the

meantime, I was mid-way through a systematic check of every remaining `PULU X` in the file (there are quite a few beyond the control-flow ones I've already fixed), to make sure I've caught every instance of this bug class rather than stop as soon as the reported symptom went away.

Are you working still?

Yes, still working — I got interrupted mid-sweep. Let me continue checking the remaining candidates efficiently, since most of that list (`STOREW`, `CFETCH`, `DFETCH`, `COUNT`, etc.) are runtime memory/stack words that never call `CCALL`/`CODECOMMA` and so can't have this bug. I'll focus on the ones that plausibly compile code.

Lines 3253 and 3308 look exactly like the vulnerable pattern (`PULU X` then later `LDD CODEHERE / PSHU D / PSHU X`). Let me check their full context to see if `CCALL`/`CODECOMMA` intervenes.

Both confirmed safe — no `CCALL`/`CODECOMMA` intervenes. Let me check `ISFOUND`/`AOFOUND` a bit further, since they check `STATE` and could lead into a compile-time path.

Both safe — `X` is consumed via `PSHU X` immediately, before any call that could clobber it. That completes the sweep: seven total instances of this bug (`LOOP`, `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, `ENDOF`), all now fixed. Running full structural verification.

All checks pass. Regenerating split files and delivering.

There's a filesystem error on the delete step — let me check the state of the outputs directory before retrying.

Good — the transient I/O error on `rm` didn't leave anything stale; the `cp` that followed succeeded and overwrote correctly. All 28 files present, hashes match exactly, and all 6 of the new fix comments landed correctly. Also confirming `forth6809.asm` itself delivered correctly.

Confirmed. Now updating the checklist with this significant finding.

Yes — sorry for the gap, I was mid-sweep and got cut off. This turned out to be a real, systemic bug, not just the one instance you reported.

Confirmed on `LOOP`: `TFR X,D` was reading the address of the `CODEHERE` variable itself (left there by `CODECOMMA`'s internal `APPENDCELL`), not the real loop-body target — because `X` got saved via `PULU X`, then both `CCALL` and `CODECOMMA` internally reload `X` for their own purposes before `LOOP` ever used it. This is exactly the kind of bug that stays hidden until the *previous* `DOTEST` fix let execution reach far enough to expose it.

Once confirmed, I swept every `PULU X` in the file rather than stop at the reported case — found the identical pattern in `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF`. Seven total.

Two different fixes were needed depending on structure. For `LOOP`/`+LOOP`/`UNTIL`/`AGAIN`/`REPEAT`, nothing else touches `U` between the pop

and the use, so parking the target on `U` itself (immune to `CCALL`/`CODECOMMA`) works. For `ELSE`/`ENDOF`, `CODEHERE` gets pushed onto `U` in between — I actually got this wrong on my first attempt, reusing the park-on-`U` pattern without re-checking whether it fit, which would have retrieved `CODEHERE` instead of the saved target. Caught it by re-tracing the exact `U` sequence before delivering rather than assuming the same fix applied everywhere, and used a scratch variable there instead.

I also verified every remaining `PULU X` individually and confirmed none of the rest are vulnerable — either nothing clobbers `X` before it's used, or the routine doesn't compile code at all.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. One transient filesystem I/O error occurred during delivery (on the cleanup step) — confirmed via hash that it didn't leave anything stale before presenting the files. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

13 compiling words
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Does emit use vectored execution to access the terminal emit routine?

Neither `EMIT` implementation uses vectored execution — both are direct, hardcoded routines. Let me check `DODEFER` (the system's actual vectored-execution mechanism) to confirm `EMIT` isn't built on it, for a complete answer.

No — `EMIT` accesses the terminal directly, with no vectored indirection.

I checked `H_EMIT`'s dictionary header: its CFA field (`FDB EMIT`) points straight at the `EMIT` code label itself, not at `DODEFER` — which is the actual vectored-execution mechanism this system has available (`DODEFER: PULU X / LDD ,X / TFR D,X / JMP ,X` — fetches an address stored in a variable and jumps through it, letting a word's target be reassigned at runtime, e.g. via

`DEFER`/`ACTION-OF`/`IS`). `EMIT` doesn't use that pattern at all.

Both `EMIT` implementations (they differ by `SERIALPOLL`) go straight to the hardware, hardcoded:

- **Polling mode**: busy-waits on `ACIASR`'s `TDRE` bit, then writes directly to `ACIADR`.
- **Interrupt-driven mode**: writes the character into a fixed ring buffer (`OUTBUF`) and pokes `ACIACR` to enable the TX interrupt if needed — the actual byte-out-the-wire transmission then happens later in `IRQH`, but that's asynchronous buffering, not vectored dispatch. There's no indirect jump through a variable anywhere in the path.

So if you're asking because you're considering making the terminal output device swappable (e.g. via `DEFER`), that's not how it's built today — `EMIT`'s target is fixed at assembly time in both configurations.

Testing the last DO LOOP fix.
Not all instances are checked yet, but the following code no longer branches to bad memory.

ok: lpy 10 3 DO I . LOOP ;
oklpy10 10 10 10 10 10 10

This is wrong as it is not accessing the index value on the return stack. It’s accessing the limit value.

Analysis of IWORD shows it’s not correct.

E98C 3510 (forth6809.asm):03197 IWORD: PULS X
E98E EC62 (forth6809.asm):03198 LDD 2,S
E990 3410 (forth6809.asm):03199 PSHS X
E992 3606 (forth6809.asm):03200 PSHU D
E994 39 (forth6809.asm):03201 RTS

The PULS X PSHU X pair appears intended to save and restore the top of return stack to allow LDD 2,S to access the top of the return stack. But in that case the offset of 2 should be 0.

The suggested fix is simply remove the PULS X PSHU X pair as unnecessary. Then LDD 2,S would be correct.

JWORD may have a similar problem.

Please confirm the analysis.

Let me trace this precisely against the return-stack layout I established a couple turns ago while fixing `DOTEST` — `DOSETUP` pushes `[return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]`.

Confirmed — your analysis is exactly correct, for both `IWORD` and `JWORD`. Let me trace `IWORD` precisely:

Before `PULS X`, `S` holds `[IWORD's own return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]` — matching the layout `DOSETUP` actually pushes. `PULS X` pops the return address, shifting `S` by 2. `LDD 2,S`, executed *after* that shift, now reads what was previously at `4,S` — the **limit**, not the index. Exactly matching what you found on real hardware. Since nothing needs offset 0 for anything here, the pop/push pair serves no purpose — `2,S`, read *without* the shift, already points straight at the index.

I checked `JWORD` too, since you flagged it as suspect. Same bug, same fix. Tracing a nested loop's stack layout at the point `J` executes: `[JWORD's own return-addr@0,S][inner index@2,S][inner limit@4,S][inner leave-flag@6,S][inner DOSETUP's own return-addr@8,S][outer index@10,S]...` — the `10,S` offset already written in the code is correct *for the unshifted stack*. The same unnecessary `PULS X`/`PSHS X` pair shifts everything by 2 first, landing on the outer loop's limit instead of its index.

Applying your suggested fix to both — removing the pop/push pair entirely.

Let me check for any other occurrences of this exact `PULS X`...`PSHS X` bracketing pattern before finalizing, in case there's a third instance (like a `K` word for triple-nested loops).

Let me check `UNLOOP` and `EXITUNLOOP`, since they're directly related to the loop mechanics and could share this bug class.

Both confirmed correct as-is — `UNLOOP` and `EXITUNLOOP` discard *counted blocks* of bytes (`LEAS 6,S`, `LEAS 8,S`) rather than reading a value at one fixed offset, so they're unaffected by the shift regardless. That's structurally different from `IWORD`/`JWORD`'s bug. Running full structural verification on the fix.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Your analysis is confirmed exactly, for both `IWORD` and `JWORD`. Tracing against the layout `DOSETUP` actually pushes (established while fixing `DOTEST` a couple turns back): `[return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]`. `PULS X` pops the return address and shifts `S` by 2, so `LDD 2,S` — executed *after* that shift — reads what was previously at `4,S`: the limit, not the index. The pop/push pair served no purpose at all; `2,S` read unshifted already points straight at the index.

`JWORD` had the identical bug at its own offset (`10,S`, which is correct for the outer loop's index in a nested layout — just wrong once shifted). Applied your suggested fix to both: removed the `PULS X`/`PSHS X` pair entirely, leaving the existing offsets untouched since they were already correct for the unshifted stack.

Before calling it done, I swept every other `PULS X` in the file for the same bracketing pattern rather than stop at the two you named. Checked `UNLOOP` and `EXITUNLOOP` specifically, since both are directly involved in loop-frame teardown — both are structurally different and correct as-is: they discard *counted blocks* of bytes (`LEAS 6,S` / `LEAS 8,S`) rather than reading a value at one fixed offset, so a temporary shift beforehand doesn't affect them the way it did `I`/`J`.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

This bug is now fixed and the DO LOOP now produces the expected results.

That's a real milestone — `DO`/`LOOP` fully working end to end, after four separate bugs in that one mechanism: the return-stack offset misalignment in `DOTEST`, the register-clobbering in `LOOP`/`+LOOP`/`UNTIL`/`AGAIN`/`REPEAT`/`ELSE`/`ENDOF`, and now `I`/`J`. Each one only became visible once the previous one stopped masking it, which is exactly the kind of thing that's very hard to find any way other than actually stepping through execution the way you have been.

Ready for whatever's next when you find it.

How many bugs were found around CONSTANT

One bug was found and fixed in `CONSTANT` itself — the self-referential PFA bug reported and confirmed via the MAME debugger trace: `LDD CODEHERE` was reading the address of the field being written, not the value cell two bytes further on, so `1234 CONSTANT c1` then executing `c1` returned the compile-

time address instead of `1234`. Fixed by adding `ADDD #2`.

At the same time, I checked `VALUEW` (the structurally similar routine right below `CONSTANT` in the source) for the same issue, since it looked like it might share the bug — but confirmed it doesn't. Its PFA points into `VARHERE` rather than `CODEHERE`, and the value gets stored at that same `VARHERE` position via `VCOMMA`, so there's no self-reference there. Left unchanged, not counted as a bug.

`How many bugs were found around compilation/WORD and dictionary search`

Breaking this down precisely by category:

`WORD` itself (parsing): 2 bugs

- `WORD`'s `EMPTY` branch pushed a c-addr that `INTERPRET` never consumed (`IDONE` was a bare `RTS`) — stranded an address on the stack on every line.
- `WORD` stored the wrong length byte for any token followed by more input on the same line — `TFR X,D` (setting up the `TOIN` calculation) silently destroyed `B`, which held the true character count from `SCANLP`'s `INCB` loop.

`FIND` (dictionary search) itself: 0 bugs

I traced `FIND` multiple times across this session — the chain walk, the length/smudge checks, the success/failure stack effects, the push order into `EXECUTE` — and it held up correctly every time. Every failure that *looked* like a dictionary-search problem (like the `": ambergris . ;"` case) turned out to originate upstream, in `WORD` or in compilation, not in `FIND`.

Compilation machinery: 2 bug reports, 8 total fix sites

- `CCALL` (compiles a `JSR` to a given xt) — `LDA #OPJSR` after `PULU D` clobbered the high byte of the address just pulled, corrupting every compiled call.
- The branch-target-clobbering bug — `LOOP`, `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF` (7 places, one root cause) all saved a branch target in `X`, then called `CCALL`/`CODECOMMA`, both of which reload `X` internally — corrupting the target before it was used.

So: **2 bugs in `WORD`, 0 in `FIND`, and 2 distinct compilation bugs** (one of which spanned 7 separate call sites). If you want `CONSTANT`'s bug folded into a "compilation" total too, that's a 3rd — it involved `CODECOMMA` directly, though I'd asked about that one separately last turn since it's really about the CFA/PFA structure rather than parsing or search.

`How many bugs were found around DO I LOOP`

Directly around `DO`/`I`/`LOOP`: **3 bug reports, spanning 12 total fix sites.**

- **`DOTEST`'s return-stack offset bug** (`: lpy 10 3 DO I . LOOP ;` exiting after one pass) — `PULS X` ran before `DOTEST`'s fixed-offset reads (`2,S`/`4,S`/`6,S`), shifting the stack by 2 so the leave-flag check read a leftover return address instead. This one bug also lived identically in `LEAVE` and `DOPLUSTEST` (`+LOOP`'s runtime) — **3 fix sites** for that single root cause.

- **The branch-target-clobbering bug** — the one you asked about last turn, which turned out to affect compilation broadly, but was *found* through `LOOP` specifically producing `+8` instead of `-8`. `LOOP` itself, plus `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF` — **7 fix sites**, one root cause (`CCALL`/`CODECOMMA` clobbering `X` before it was read back).

- **`IWORD`/`I` and `JWORD`/`J`** — printing the loop limit instead of the index on every iteration, from an unnecessary `PULS X`/`PSHS X` pair shifting the stack before the fixed-offset read. **2 fix sites.**

So: 3 distinct root causes, 12 individual places fixed, all specifically in the `DO`/`I`/`LOOP` mechanism. That's the largest concentration of bugs found in any single feature this session — each one only became visible once the previous one stopped masking it.

In the process of manually testing the words list in the glossary, but i want to investigate creating unit tests written in assembly language.

The scheme I have in mind uses conditional assembly directives to control whether the tests execute or not.

As a general principle unit tests are designed to be independent from one another.

As a first try a subroutine call at the end of INITCODE targets a subroutine called TSTRUNNER. This as surrounded by a conditional assembly block.

TSTRUNNER is to be located at the start of unused memory beginning at the end of the IO area. I think this would have an ORG $C100 if memory serves.

TSTRUNNER calls a list of subroutines that execute tests.

The first test routine tests data stack operations. It’s first unit test tests DUP.

This unit test saves the top of data stack location, stacks two random numbers, executes DUP, then verifies that the number on the stack is correct, verifies that the top of data stack location has changed by the expected

amount. TYPE is used to identify the unit test by its name TSTDUP. If the test passed, TYPE prints OK. If either of these assertions fail, TYPE prints FAIL. On the same line as the test name. A CR is also issued in readiness for the next test.

The entirety of the test area is surrounded by a conditional assembly block.

This is a substantial addition — let me first check a few things in the actual source before designing this: `DUP`'s real code label, the current `ORG USROMSTRT`/`FILL` structure I'll need to adapt, `TYPE`'s exact calling convention, and where `INITCODE` actually ends.

Good — `DUP` is a valid label. Let me view the current `ORG USROMSTRT` / `FILL` area, and confirm `DUP`'s exact code and `TYPE`'s calling convention.

Now let me check `DUP`'s full code and `TYPE`'s exact calling convention.

Now let me find where `COLDSTRT` ends, to place the `TSTRUNNER` call correctly.

Before finalizing placement, I need to check when `U` (the data stack register) actually gets initialized during `COLDSTRT` — the tests need a valid data stack to run safely, and I'm inserting this call *before* `JMP COLD`, which is where I'd expect `U`/`S` setup to happen.

Let me check `CRW`'s full calling convention before designing the shared reporting helper.

Now let me confirm none of my planned symbol names already exist in the file, before writing the implementation.

All clear. Now implementing. First, adding the conditionally-gated `TSTRUNNER` call at the end of `COLDSTRT`.

Now building the full test framework, replacing the `ORG USROMSTRT` / `ROM:` / `FILL` block.

Let me run a comprehensive verification. Since this introduces a second, independent conditional-assembly flag alongside `SERIALPOLL`, I'll extend the simulation script to handle both and test all four combinations.

All four combinations pass cleanly. Let me verify the `IFEQ`/`ENDC` balance for the new `UNITTESTS` blocks specifically (they don't use `ELSE`, unlike `SERIALPOLL`'s blocks).

All balanced and confirmed. Let me confirm which split file this lands in, then regenerate everything.

Confirmed correct placement. Delivering.

Now updating the checklist with this new capability.

Built the framework and the first test. Here's how it's structured:

Placement: `JSR TSTRUNNER` gets called at the true end of `COLDSTRT`, right after `INITSERIAL` and before `JMP COLD`. I checked this timing carefully — `U`/`S` are already valid there (`COLDSTRT` sets them at its very start), but nothing else (`APPVARS`, `DPHERE`, `CODEHERE`, `LATEST`, `BASE`) has been touched yet, which

matters for where scratch storage can safely live.

Location: `TSTRUNNER` and everything under it lives at `USROMSTRT` ($C100) — the space right after `INOUT`'s shadow that was pure `FILL` padding before. Gated by a new `UNITTESTS` flag matching your existing `SERIALPOLL` convention (`0` = included, `1` = excluded). With tests excluded, this reverts to exactly that padding — I made the `FILL` computation self-adjusting (`BASEDICT-ROM` instead of a fixed count) so it stays correct automatically as the test code grows, rather than needing hand-maintenance.

Independence: every test saves `U` before touching anything and *unconditionally* restores it at the end, regardless of pass or fail — so a failed assertion partway through a test can't leave stack residue for the next one. Test scratch variables borrow the very start of `APPVARS`, which is safe only because tests run strictly before `COLD` re-initializes that same address for real use — worth remembering if this call site ever moves.

Reporting: shared across all tests via `TSTREPORT`, not duplicated per test — prints the test's name (a counted string, via `COUNT`+`TYPE`, the same mechanism `BADWORD` already uses to print a failing word), then `" OK"` or `" FAIL"`, then a `CR`.

`TSTDUP` checks both the stack contents (duplicate and original both match a deliberately non-trivial test value — not `0`, `1`, or `-1`, so a test that only passes by accident on a special-cased value gets caught — plus a sentinel underneath confirmed undisturbed) and the pointer movement (exactly 2 bytes, measured immediately before/after the `JSR DUP` itself, not around the whole test, so the two setup pushes don't get folded into that number).

`TSTRUNNER`/`TSTSTACK` are just dispatcher lists, so adding more tests or groups later doesn't require restructuring anything.

Verified across all four `SERIALPOLL`×`UNITTESTS` combinations: zero duplicate symbols, dictionary chain intact in every case, byte-exact split-file reassembly. **Not yet run on real MAME** — this is a first try with one test written, and given everything this session has taught about the gap between clean-looking source and actual execution, I'd treat this as unverified until you've stepped through it.

Forth6809
ASM · Click to download

00 memory map and globals
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

As a side note can you make a copy this chat in another project so that it acts as the starting point of a new project?

This isn't something I can do directly from within our conversation — I don't have a tool that copies conversations between projects. Let me check current documentation for what's actually possible here rather than guess.

I can't do this directly from inside our conversation — there's no tool available to me that duplicates or moves chats between projects. But it is possible through the Claude.ai interface itself:

Move this chat into a project (built into claude.ai):

- From your chat list (not from inside this conversation), find this chat.
- Select it and choose to move it into a project — you can move it into an existing project or create a new one from it directly.

That gives the new project this entire conversation as one of its chats, with full history intact.

Alternative — if you'd rather have a clean starting point than the raw chat:

I could put together a written summary of the technical state we've built up here — the memory map, the `SERIALPOLL` conditional-assembly setup, the dictionary structure, the full list of bugs found and fixed (`WORD`, `CCALL`, `CONSTANT`, the `DO`/`LOOP` mechanism, etc.), and the unit test framework — as a document you could drop into the new project's knowledge base or instructions. That'd be more compact than the full transcript and easier for a fresh conversation to pick up from immediately, though it wouldn't preserve the moment-by-moment debugging trail the way moving the actual chat would.

Want me to put that summary together, or does moving the chat directly cover what you need?

Yes create the summary

No dedicated skill constraints apply to plain markdown — writing this directly.

Covers the architecture and current memory map, the conditional-assembly conventions (including the non-obvious `IFEQ`-means-zero gotcha), the debugging methodology that emerged this session (static tracing alone isn't reliable here — several "this looks correct" conclusions turned out wrong, and bugs only became visible once earlier ones stopped masking them), the complete chronological bug history with root causes and fix counts, the new unit test framework's design and its unverified status, and the working conventions (verification steps, checklist-as-history-not-living-document, docx editing approach) that a new conversation would need to keep following consistently.

Drop this alongside `forth6809.asm`, the split files,

`OPEN_ITEMS_CHECKLIST.md`, and `ClaudeForth.docx` into the new project's knowledge base, and a fresh conversation should be able to pick up from exactly where this one left off.

ClaudeForth Project Summary
MD · Click to download

Combine the split asm files as a zip file

All 28 split `.asm` files zipped (flat, no subfolder).
Zipped up.

Forth6809 split
ZIP · Click to download

A parallel development of a 68000 forth, that is a port of the 6809 forth, flags the following as potential bugs that also exist in the 6809 code. Can these bugs be verified in the 6809 forth by inspection or do we wait for manual testing in MAME fur verification?

The comment is as follows:

Good — this batch surfaces something important. Let me add the required globals first, then check on a bug pattern before writing the defining words.

Now let me flag the suspected bug pattern at `DODEFER` (already in the file) before writing `DEFERW`, since the two are connected:

Now appending Section 13 and the remaining defining words:

Section 13 is in, along with the rest of the defining words. This batch surfaced something worth your attention directly, not just buried in a comment:

A likely bug pattern shows up in three more places. The original 6809 source has one *confirmed* bug (via MAME debugger): `CONSTANT` built a self-referential parameter field — it stored its own address instead of pointing at the value cell that follows — and was fixed with a `+2` correction. I carried that fix forward into `CONSTANT` since it's verified.

But the identical construction (unfixed, as far as anything I retrieved shows) also appears in:
- **`DEFER`/`DEFER@`/`DEFER!`/`IS`/`ACTION-OF`** — the whole deferred-word family

- **2CONSTANT**
- **MARKER** (flagged last time as DOMARKER)

> I ported all of these literally rather than silently applying the CONSTANT fix on inference — I don't have a debugger to confirm it the way the original's own bugs were confirmed, and a wrong "fix" would be worse than a flagged bug. But this is worth real attention before relying on DEFER or MARKER: it would mean four related words are reading/writing the wrong memory cell.

Yes — this is a structural, compile-time address computation, not a timing/hardware/register-clobbering issue, so it's genuinely verifiable by inspection, the same way I confirmed VALUEW was *not* affected when fixing CONSTANT. Let me check each flagged word precisely against the actual 6809 source.

Let me view each defining word's compile-time code precisely, starting with DEFERW.

Confirmed — DEFERW has the exact same pattern: LDD CODEHERE / PSHU D / JSR CODECOMMA reads CODEHERE's value *before* this very field is written, self-referentially, then JSR COMMA appends the real value (DOABORTUNDEF) two bytes later. Let me check DODEFER (the runtime consumer) to confirm the consequence, then the rest of the family.

TOBODY is structurally different — a *runtime* computation from an existing xt (ADDD #5 skips past the trampoline to the PFA), not a compile-time self-reference. It's immune to this specific bug class by construction. Let me view DEFERFETCH/DEFERSTORE in full to confirm they're sound, then check IS/ACTION-OF.

IS/ACTION-OF don't construct any self-referential structure — they just locate an xt and either execute or compile a call to DEFERSTORE/DEFERFETCH. Sound as-is. Only DEFERW itself needs the fix; the rest of the family are downstream consumers. Now checking 2CONSTANT.

TWOCONSTANT (2CONSTANT) has the identical bug. While I'm here, BUFFERCOLON sits right next to it using the VALUEW-style pattern (VARHERE, not CODEHERE) — let me quickly verify VALLOT to confirm it's genuinely safe rather than assume from the resemblance alone.

Confirmed safe, same reasoning as VALUEW — VALLOT only reserves space by advancing VARHERE, it never writes through the PFA itself. Now checking MARKERW/DOMARKER, the last flagged item.

Same bug confirmed in MARKERW. Let me check DOMARKER (the runtime consumer) too, to fully understand the consequence — MARKER restores 4 snapshot values, not just 1.

Yes — these are verifiable by inspection, and three of the four categories are real, confirmed bugs

The self-referential PFA bug isn't a timing or register-interaction issue that only shows up at runtime — it's a deterministic compile-time address computation (`LDD CODEHERE` read *before* the field it's about to occupy is written). That's exactly the kind of thing that's fully traceable from source, the same way I confirmed `VALUEW` was *not* affected when originally fixing `CONSTANT`. I traced each flagged item's compile-time construction and its runtime consumer precisely.

`DEFER` (`DEFERW`) — confirmed, real bug. Identical construction: `LDD CODEHERE / PSHU D / JSR CODECOMMA` writes the PFA field self-referentially, then `JSR COMMA` appends the real value (`DOABORTUNDEF`) two bytes later. I traced the consequence through `DODEFER` (the runtime): it pops the PFA and does `LDD ,X` expecting the target xt to jump to — with the bug, it reads back the PFA's own address and jumps there instead. Every newly-`DEFER`'d word would crash the moment it's executed.

`DEFER@`/`DEFER!`/`IS`/`ACTION-OF` — checked, not independently affected. `DEFER@`/`DEFER!` go through `TOBODY`, a *runtime* computation from an existing xt (`ADDD #5`) — structurally immune to this bug class, since it never compiles a self-referential field. `IS`/`ACTION-OF` don't construct any PFA at all; they just locate a word and call `DEFERSTORE`/`DEFERFETCH`. All four are sound as written — they'd simply inherit whatever `DEFER` itself got wrong, so fixing `DEFER` alone fixes the whole family's behavior.

`2CONSTANT` (`TWOCONSTANT`) — confirmed, real bug. Same pattern exactly, appending both cells of the double value after the self-referential field.

`MARKER` (`MARKERW`) — confirmed, real bug, and worth flagging as the most severe of the three. Same construction, but here the PFA is supposed to point at four snapshot cells (`DPHERE`/`CODEHERE`/`VARHERE`/`LATEST`) that `DOMARKER` reads at fixed offsets (`,X` / `2,X` / `4,X` / `6,X`) to roll the dictionary back. With the bug, `DOMARKER` reads garbage at all four offsets and restores corrupted dictionary-state pointers — using a `MARKER`-created word would likely corrupt the whole system, not just misbehave locally.

One thing I checked along the way rather than assume from proximity: `BUFFERCOLON` (`BUFFER:`) sits right next to `2CONSTANT` and uses the same `VALUEW`-style pattern (PFA into `VARHERE`, not `CODEHERE`). Verified `VALLOT` only advances `VARHERE` and never writes through the PFA — so `BUFFER:` is genuinely safe, same reasoning as `VALUEW`.

Given the mechanism is identical to the MAME-confirmed `CONSTANT` bug and I've traced both sides (construction and consumption) of each, I'm confident enough to apply the same `+2` fix to `DEFER`, `2CONSTANT`, and `MARKER` now rather

than wait on a separate MAME round. Want me to go ahead?

Go ahead and

Applying the same `+2` fix to `DEFER`, `2CONSTANT`, and `MARKER`.

Now `TWOCONSTANT` (`2CONSTANT`).

Now `MARKERW` (`MARKER`).

Running full structural verification across all conditional-assembly combinations.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

All three fixed with the same `ADDD #2` pattern already established for `CONSTANT`.

Worth being precise about scope here: I fixed exactly the three that turned out to actually have the bug — `DEFER`, `2CONSTANT`, `MARKER` — not the whole family the 68000 comment listed. `DEFER@`/`DEFER!`/`IS`/`ACTION-OF` are downstream consumers that construct no self-referential field themselves, so they were left untouched; fixing `DEFER` alone corrects their behavior. `MARKER` is the one worth flagging as most serious — its runtime reads four fixed offsets to roll back dictionary state, so the bug there meant restoring corrupted `DPHERE`/`CODEHERE`/`VARHERE`/`LATEST` pointers, not just a wrong value.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Not yet confirmed on real MAME — the mechanism is identical to the hardware-confirmed `CONSTANT` bug, but these three specific fixes are still unverified against actual execution, same as everything else pending real testing right now. Propagated to the checklist.

Forth6809
ASM · Click to download

08 defining words
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Another potential bug has been flagged in the 68000 forth develop chat.

As follows:

2. **`2@` gets the same parity flag as `2!`'s sibling operations earlier.** Tracing the 6809 source by hand: it pushes the higher-address cell first/deep and the lower-address cell second/top — the reverse of both the glossary's own stated convention *and* of what `2!` (right above it in the same section) actually writes. A `2! 2@` round trip on the original would swap the two values. `2!` itself is correct; only `2@` looks inverted. Ported literally, flagged rather than silently fixed, per the same reasoning as the `MARKER`/`2CONSTANT`/`DEFER` notes.

Confirmed `2@`→`DFETCH`, `2!`→`DSTORE`. Let me view both implementations precisely.

Confirmed by inspection — `2@` genuinely doesn't round-trip with `2!`

Traced both precisely against the actual code (`2@`→`DFETCH`, `2!`→`DSTORE`):

```
DSTORE:  PULU  X        ; a-addr
         PULU  D        ; x2 (top of stack, pushed last)
         STD   2,X      ; x2 → HIGH address (a-addr+2)
         PULU  D        ; x1 (now top, pushed first/deeper)
         STD   ,X       ; x1 → LOW address (a-addr)

DFETCH:  PULU  X        ; a-addr
         LDD   2,X      ; reads HIGH address (x2) ...
         PSHU  D        ; ... pushes it FIRST/deep
         LDD   ,X       ; reads LOW address (x1) ...
         PSHU  D        ; ... pushes it SECOND/top
```

Tracing a concrete round trip: stack `[x1, x2, a-addr]` before `2!`. `DSTORE` writes `x1`→low, `x2`→high. `DFETCH` then reads high first (pushing `x2` deep) and low second (pushing `x1` on top) — result `[x2, x1]`, the two values swapped from where they started. `2!` itself is internally consistent (correct); `2@` reads the two cells in the opposite order it should to match. This is a deterministic addressing bug, fully confirmable from source the same way the PFA bugs were — no MAME needed to establish it, though real testing would still be good to confirm end-to-end once fixed.

The fix is a straightforward swap of `DFETCH`'s two `LDD`/`PSHU` pairs, so it reads low-then-high instead of high-then-low, matching what `DSTORE` actually writes:

```
DFETCH:  PULU  X
         LDD   ,X       ; low address first (x1)
         PSHU  D
         LDD   2,X      ; high address second (x2)
         PSHU  D
```

```
        RTS
```

Want me to apply that?

Apply

Running full structural verification across all conditional-assembly combinations.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Applied. `DFETCH` (`2@`) now reads low-then-high, matching what `DSTORE` (`2!`) actually writes — a `2! 2@` round trip is correct.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Not yet confirmed on real MAME. Propagated to the checklist.

Forth6809
ASM · Click to download

18 memory
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Verify that a push pop pair in LEAVE can be eliminated.

The 68000 forth comment is as follows:

One small deliberate deviation: the 6809’s corrected LEAVE still carries a PULS X/PSHS X pair after the actual fix that has zero effect (pop then immediately push the same value back) — leftover from patching the bug rather than something functional. I dropped it rather than port a verified no-op forward.

Confirmed — yes, this can be eliminated. Tracing the current code precisely:

```
LEAVE:   LDD   #TRUEV
         STD   6,S       ; writes the leave-flag directly -
doesn't need X at all
         PULS  X         ; pops LEAVE's own return address
         PSHS  X         ; pushes it right back, unchanged
         RTS
```

Nothing happens between `PULS X` and `PSHS X` — no call, no other touch of `S` or `X` — so `X` holds exactly what it held before the pop, and `PSHS X` restores `S` to exactly where it was. `RTS` then pops the same return address it would have found sitting there if the pair were removed entirely. It's a genuine no-op, not just a "probably harmless" one.

The reason it's there at all: before the earlier fix, `PULS X` ran *first*, and `PSHS X` was needed to restore `S` after it — that version of `LEAVE` never used `X`'s value either, but the pair wasn't a no-op yet, since the flag-write (`STD 6,S`) happened *while* `S` was shifted, which was the bug. Moving `PULS X` to after the write fixed the offset, but left the now-pointless `PSHS X` sitting there as vestige of the old structure — unlike `DOTEST`, where the deferred `PULS X` is genuinely load-bearing (its value feeds `LDD ,X`/`LEAX D,X` afterward), `LEAVE` never uses `X` for anything at all.

Simplifies cleanly to:

```
LEAVE:   LDD   #TRUEV
         STD   6,S
         RTS
```

Want me to apply that?

Running full structural verification across all conditional-assembly combinations.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Applied. `LEAVE` is now:

```
LEAVE:   LDD   #TRUEV
         STD   6,S
         RTS
```

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Logged this one as a simplification rather than a bug fix in the checklist, since it's behaviorally identical either way — no functional change, just removing dead motion. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Again a similar issue was found during the 68000 fifth port:

Also worth noting: U. and U.R each had a literal pop-then-immediate-push-back in the 6809 source (functionally a no-op) — simplified away rather than ported literally, since there's nothing to preserve there.

Confirmed `U.`→`UDOT`, `U.R`→`UDOTR`. Let me view both precisely.

Confirmed — same pattern exactly. `PULU D / PSHU D` pops the value then immediately pushes it back with nothing in between; the following `LDD #0 / PSHU D` (building the double-number for `NUMGT`) works identically whether the original value was ever popped or not. Let me check `UDOTR`.

Confirmed the same pattern in `UDOTR` too — but only the *second* `PULU D`/`PSHU D` pair (the value itself); the first one (`PULU D / STD DRWIDTH`) is a genuine, functional pop of the width argument and must stay. Let me apply both simplifications.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Confirmed and applied — both `U.` and `U.R` had the same class of no-op you found in `LEAVE`. In `UDOTR` I was careful to only remove the second pop/push pair (the value being formatted); the first (`STD DRWIDTH`, consuming the width argument) is genuine and untouched.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

20 numeric output
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The 68k port flags a potential issue in UNESCAPE as follows, is this a real

Issue:

One more parity flag, smaller than the others: UNESCAPE's handling of a lone trailing backslash looks like it double-decrements a counter in the original (once for consuming the backslash, again in the shared plain-emit path it falls into) — ported literally rather than silently corrected, consistent with how the other suspected issues in this port have been handled.

Let me view `UNESCAPEW` in full.

Yes — this is a real, confirmed bug

Traced `UNESCAPEW` precisely for exactly the scenario described: a backslash as the very last character in the input, with nothing after it.

```
UELOOP:    LDD  UESRCLEN
           BEQ  UEDONE
           LDA  ,X+           ; reads the '\' - this is the last
char
           CMPA #'\'
           BNE  UEPLAIN
           LDD  UESRCLEN
           SUBD #1            ; decrement #1: consuming the
backslash
           STD  UESRCLEN      ; -> now 0, correctly, since
nothing follows
           BEQ  UEPLAIN       ; <-- falls into the shared plain-
emit path
           LDA  ,X+
           ...
UEPLAIN:   STA  ,Y+
           LDD  UEOUTLEN
           ADDD #1
           STD  UEOUTLEN
           LDD  UESRCLEN
           SUBD #1            ; decrement #2 - but UESRCLEN is
already 0
           STD  UESRCLEN      ; underflows to $FFFF
           BRA  UELOOP
```

`UESRCLEN` correctly hits `0` after the first decrement (accounting for the backslash itself), but `BEQ UEPLAIN` then falls through to the *shared* plain-character path, which decrements `UESRCLEN` a second time for a character that was never separately consumed. `0 - 1` underflows to `$FFFF` (unsigned), which is nonzero — so the very next `UELOOP` iteration's `LDD UESRCLEN / BEQ`

`UEDONE` doesn't fire, and the loop keeps running past the end of the intended input, reading whatever memory comes after the source string.

Worth confirming the normal path *isn't* affected, to be precise about scope: the ordinary plain-character case (`BNE UEPLAIN` from the top, when the current char isn't a backslash at all) decrements exactly once, correctly. A real two-character escape like `\n` also works correctly — one decrement for the backslash (inside the escape branch), one more via the shared `UEEMIT` path for the `n`. The bug is specific to the lone-trailing-backslash case, where the escape branch's own decrement already accounted for the backslash and then falls into a path that decrements again for a character that doesn't exist.

The fix is a dedicated landing point for this one case that emits the backslash literally without re-decrementing (`A` still holds `'\'` from the earlier `LDA ,X+`, so nothing needs reloading):

```
            BEQ  UELASTBS      ; was BEQ UEPLAIN
            ...
UELASTBS:   STA  ,Y+
            LDD  UEOUTLEN
            ADDD #1
            STD  UEOUTLEN
            BRA  UELOOP
UEPLAIN:    ...                ; unchanged, still used by the
normal path
```

Want me to apply that?

Yes, apply

Now inserting the new `UELASTBS` label, right before the (untouched) `UEPLAIN` label.

Let me verify the full, updated routine reads correctly end-to-end.

Reads correctly end-to-end. Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Applied. The lone-trailing-backslash case now lands at its own `UELASTBS` label, which emits the backslash literally without re-decrementing `UESRCLEN` — `UEPLAIN` itself, still handling the normal plain-character path, was left untouched.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Not yet confirmed on real MAME. Propagated to the checklist.

19 string words
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

This test code for UNLOOP seems to branch off into the random memory requiring a soft reboot in MAME. It should return 3.

: find3 DO I 3 = IF I UNLOOP EXIT THEN LOOP 0 ;

Can you identify a fault by inspection?

Let me view all three routines in full, since `EXIT` compiled inside a `DO`-loop context is exactly the untested combination here.

Yes — identified precisely, and it's not a simple typo bug. It's a real conflict between two mechanisms that both try to do the same job.

Tracing `EXIT`'s compile-time scan for this exact definition: it correctly counts one active `DO` (via `TAGDO` markers on the compile-time stack) and compiles `JSR EXITUNLOOP` followed by `FDB 1`. At runtime, `EXITUNLOOP` reads that count and does `LEAS 8,S` *per counted loop* — unconditionally discarding a full 8-byte `DOSETUP` frame (index+limit+leave-flag+return-address) for each one, on the assumption that frame is still fully intact.

That assumption is exactly what your test code violates. `UNLOOP`, called manually right before `EXIT`, already discards 6 of those 8 bytes (index/limit/leave-flag) itself. By the time `EXITUNLOOP` runs, only 2 bytes of that frame remain on the stack — but it still discards 8, overshooting by 6 bytes into whatever's underneath (in this case, straight through `find3`'s own caller's return address). That's the "branch into random memory" you're seeing — it's a genuine stack corruption, not a fluke.

I confirmed this by tracing the *without*-`UNLOOP` case too: `: find3 DO I 3 = IF I EXIT THEN LOOP 0 ;` (dropping `UNLOOP`) — traced byte-for-byte, `EXITUNLOOP`'s `LEAS 8,S` lands exactly right, since the full frame is genuinely still there. It works correctly.

So there are two honest ways to describe this:

- **Immediate, correct workaround**: drop `UNLOOP` from your test. This implementation's `EXIT` already performs the loop-unwind automatically

(that's what `EXITUNLOOP` *is*) — matching the ANS standard's own wording, which says `EXIT` must correctly exit "whether or not" it's nested in a `DO...LOOP`, with no `UNLOOP` required. `UNLOOP EXIT` is a common idiom in Forth systems where `EXIT` does *not* auto-unwind — but that's not this implementation.

- **The deeper issue worth flagging**: `UNLOOP EXIT` is still a widely-taught, ANS-legal idiom, and nothing in this implementation's glossary (as far as I've seen) says it's unsafe here specifically. A programmer reasonably familiar with Forth would write exactly what you tested and expect it to work. I'd call this a real robustness gap in `EXIT`/`EXITUNLOOP` — not just your test being "wrong."

I haven't changed anything yet. Do you want the practical fix (drop the redundant `UNLOOP` and confirm the definition works as-is), or should I make `EXIT`/`EXITUNLOOP` robust against a preceding manual `UNLOOP` too, so both forms work safely? The second is more involved — it'd need `EXIT`'s runtime to detect how much of the frame is actually still there rather than trust a fixed compile-time count.

> In the existing implementation of EXIT what happens if it’s not inside a DO LOOP and there are no loop parameters to clean up?

Traced this precisely — it works correctly, with zero risk of over- or under-discarding anything.

`EXIT`'s compile-time scan always compiles `JSR EXITUNLOOP` followed by a count, even when no `DO` is active — it just compiles a count of `0`. With no `TAGDO` markers between the current compile position and `CSP`, the scan hits `CMPD CSP` / `BEQ EXSCANDONE` on essentially the first check, so `EXITCNT` stays `0`.

At runtime, that `0` flows straight into `EXITUNLOOP`'s loop:

```
EXITUNLOOP: PULS X       ; pop own return address (points at "FDB
0")
            LDD  ,X      ; reads 0
            TFR  D,Y     ; Y = 0
EULOOP:     CMPY #0      ; Y is already 0
            BEQ  EUDONE  ; branches immediately - loop body never
runs
            LEAS 8,S     ; (never reached)
            ...
EUDONE:     PULS Y       ; pop the enclosing word's own caller
return address
            JMP  ,Y      ; jump there
```

With `Y` starting at `0`, the `CMPY #0`/`BEQ EUDONE` check fires on the very first

pass — `LEAS 8,S` never executes at all. So the whole sequence reduces to: pop `EXITUNLOOP`'s own return address, then pop and jump to whatever's next on the return stack (the enclosing word's caller). That's functionally identical to a plain `RTS` — just routed through this same uniform mechanism rather than having a separate compiled path for "no loop" versus "one or more loops."

This is a useful data point alongside the earlier finding: it confirms the corruption is narrowly specific to the "manual `UNLOOP` immediately before an `EXIT` that also tries to auto-unwind" combination. `EXIT` used with no enclosing loop at all, or inside a loop *without* a preceding manual `UNLOOP`, both trace out correctly by this same mechanism.

> If the EXIT implementation is robust under all scenarios , then UNLOOP should be defined as a NOOP and the documentation should be updated to reflect that.

This is a sound conclusion, but let me stress-test one edge case before implementing it, since making `UNLOOP` a no-op is a real architectural decision, not a cosmetic tweak.

The logic holds for the specific case that matters. `UNLOOP`'s only sensible use is immediately before something that leaves the current definition entirely — either an explicit `EXIT`, or falling through to the word's own final `;` without ever revisiting `LOOP`/`+LOOP` again. Since `EXIT` already discards the frame correctly whenever it's left untouched, and a no-op `UNLOOP` leaves it untouched, both cases resolve cleanly.

One nuance worth naming explicitly, not silently assuming away: what if code calls `UNLOOP` and then *doesn't* exit — just keeps running and eventually reaches `LOOP`/`+LOOP` again? That's already broken today, independent of this change — `DOTEST`/`DOPLUSTEST` expect the frame to still be there when they run, so `UNLOOP` (in its current, real-discard form) already corrupts that path today too. Making `UNLOOP` a no-op doesn't introduce this risk — it actually *removes* one instance of it (the `UNLOOP`-then-loop-continues case would now work by accident, since nothing was discarded). So there's no new failure mode here, only a removed one.

Given that, the conclusion holds. Let me implement it.

Running full structural verification.

All checks pass. Regenerating split files and delivering the `.asm` change first, then updating the documentation.

Now checking and updating the glossary documentation for `UNLOOP`.

Found `UNLOOP`'s current glossary entry — it's now stale in a way that would actively mislead someone (it says `UNLOOP` is "needed before an `EXIT`," which used to be true and now describes exactly the combination that corrupts the stack). Let me update it, and add a clarifying note to `EXIT`'s own entry too.

Now updating `EXIT`'s own entry with a clarifying note.

Since the `.asm` source itself changed, let me also sync the Assembler Source appendix (Section 8) for just the affected subsection, rather than re-run the full 28-section replacement.

Confirmed. Updating just this one subsection with the current `12_control_flow.asm` content.

Let me check whether that's a real problem or `LEAS 6,S` legitimately appears elsewhere (e.g. `DTEXIT`, which uses it for an unrelated purpose).

False alarm — that's `DTEXIT` and a similar discard (unrelated to `UNLOOP`), plus my own explanatory comment referencing the old code as history. All correct. Running the field-update and PDF export pass.

Both files completed. Confirming cleanup and verifying content.

Clean. Verifying the actual rendered content.

All confirmed present. Finalizing delivery of all files.

Confirmed identical via hash. Now updating the checklist.

Applied and documented. `UNLOOP` is now just `RTS`.

Before implementing, I checked whether this removes one risk while introducing a different one — specifically, `UNLOOP` called without an immediate exit, then falling through to `LOOP`/`+LOOP` again. That's already broken today independent of this change (`DOTEST`/`DOPLUSTEST` already expect the frame intact when they run), so making `UNLOOP` a no-op introduces no new failure mode — it actually removes one instance of an existing one.

Documentation updated on both fronts: `UNLOOP`'s glossary entry now explains it's a no-op and why, kept in the dictionary for source compatibility with Forth systems where `EXIT` genuinely requires it first; `EXIT`'s own entry got a cross-reference clarifying `UNLOOP` isn't needed beforehand. I also synced the Assembler Source appendix (Section 8.13) rather than leave it showing the old, now-incorrect code.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly, both new documentation passages confirmed present in the rendered PDF. Not yet confirmed on real MAME. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

ClaudeForth
DOCX · Click to download

ClaudeForth preview
PDF · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Testing of CASE OF ENDOF ENDCASE is failing with a -22 error. This appears to occur due to a CSP stack mismatch during compilation.

Tracing with MAME seems to show one stack item in excess when checked by SEMICOLON.

The prime suspect is the first line of CASE which puts a 0 on the stack for no discernible purpose or use and which is not consumed.

Verify the analysis.

Example code is :

: Nname CASE 0 OF ENDOF ENDCASE ;

Let me view all four routines in full to trace the counter's lifecycle precisely.

Confirmed exactly — your analysis is correct

I traced the complete compile-time `U`-stack lifecycle through `: Nname CASE 0 OF ENDOF ENDCASE ;`:

- **`CASE`** pushes `[0, TAGCASE]` (0 first/deeper, `TAGCASE` on top).
- **`OF`** pushes `[placeholder-addr, TAGOF]` on top of that.
- **`ENDOF`** pops exactly those two (`TAGOF` then `placeholder-addr`), then pushes its own `[NEWFLD, TAGENDOF]`.
- **`ENDCASE`** loops: pops `TAGENDOF`/`NEWFLD` (patches the branch, matches this construction exactly), loops again, pops `TAGCASE` — sees the match, and stops (`BEQ ECDONE`).

That's the precise problem: `ENDCASE`'s loop terminates *the moment it sees `TAGCASE`* — it never pops anything beneath it. The `0` that `CASE` pushed *before*

`TAGCASE` is structurally invisible to every word that follows: `OF`/`ENDOF` never reach that deep, and `ENDCASE`'s scan stops exactly one cell short of it. Nothing in the entire `CASE`...`ENDCASE` construct ever reads or consumes it — I checked all three consumers (`OF`, `ENDOF`, `ENDCASE`) and none of them go past `TAGCASE`.

So it sits there permanently. `:` sets `CSP` to the depth before `CASE` ever runs; by the time `;` checks the depth against `CSP`, there's one extra cell (`0`) still present — exactly the mismatch that throws `-22`.

Worth confirming what it was likely *for*: some `CASE` implementations use a running counter there for `ENDCASE` to consume when patching. This one doesn't — `ENDCASE` uses the `TAGCASE`-scan approach entirely, with no counter involved anywhere in its logic. The `0` looks like a genuine leftover from a different design, never removed when the scan-based approach was implemented.

Fix, pending your go-ahead: remove `CASEW`'s `LDD #0 / PSHU D`, leaving just:

```
CASEW:   LDD   #TAGCASE
         PSHU  D
         RTS
```

Want me to apply that?

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Applied. `CASEW` is now just:

```
CASEW:   LDD   #TAGCASE
         PSHU  D
         RTS
```

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Not yet confirmed on real MAME. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Using NAME, verifying the behavior of J within a DO LOOP, containing a nested DO LOOP, with J accessing the return stack from the outer loop.

J is accessing the return stack at offset 10 and is placing the wrong number on the data stack.

If my analysis is correct the offset should be 2.

Please verify and correct as appropriate.

This contradicts my earlier established layout for `JWORD` — let me re-derive it completely from scratch, very carefully, rather than trust either my prior conclusion or the new hypothesis without re-verifying against the actual code.

Let me check `DO`'s compile-time code precisely, to make sure I'm not missing something about what actually gets compiled between the outer and inner `DO`.

I re-derived the full nested-loop return-stack layout from scratch, independent of my earlier work on `JWORD`, and I keep arriving at offset 10 for the standard case — so I want to show my work and reconcile this rather than either blindly apply your proposed fix or dismiss your MAME trace.

Tracing `: outer 0 DO 0 DO J LOOP LOOP ;` (outer `DO` around an inner `DO`, `J` called directly inside the inner body):

`DO` compiles nothing but `JSR DOSETUP` — confirmed, nothing else sits between `DO` and the loop body.

When the **outer** `DOSETUP` runs, it pops its own `JSR` return address and pushes `[outer-index@2,S][outer-limit@4,S][outer-leave@6,S]`, re-pushing the return address on top. `S` after outer `DOSETUP`: `[return-addr@0][index@2][limit@4][leave@6]`.

Execution continues into the outer body, reaching the **inner** `DO`. Its `JSR DOSETUP` pushes a *new* return address on top of the still-fully-intact outer frame. Inner `DOSETUP` pops that, and pushes its own `[inner-index][inner-limit][inner-leave]`, re-pushing its own return address on top. `S` now: `[inner-return-addr@0][inner-index@2][inner-limit@4][inner-leave@6][outer-return-addr@8][outer-index@10][outer-limit@12][outer-leave@14]`.

`J` is called from inside the inner body — `JSR JWORD` pushes `JWORD`'s own return address on top of all of that. So when `JWORD`'s `LDD 10,S` executes, offset 10 lands exactly on `outer-index`.

That's the trace, and it's consistent with what I found when I fixed `JWORD`'s offset a few turns back. So I can't currently reproduce a mechanism that makes 2 correct for this structure — offset 2 would read the *inner* loop's own index, which is what `I` returns, not what `J` is supposed to return.

Given you traced this on real MAME and I'm working from static analysis, I don't want to dismiss this — but I also don't want to apply a change I can't reconcile with the trace above. Could you share the exact colon definition you tested with `NAME`? If something else is compiled between the outer `DO` and the inner `DO` (or between the inner `DO` and where `J` is called), or if there's a different bug shifting the actual runtime stack depth, that would change what the correct offset is — and I'd rather see the real source than guess at it.

My mistake, I misread the glossary description of the behavior for J.

Nevertheless it’s still not working as now that I am using J inside the inner loop to access the index in the outer loop, it seems to be accessing the outer loop limit not the outer loop index. Here is an example from the web.

```
ok: MULT-TABLE CR 11 1 DO 11 1 DO I J * . LOOP CR LOOP ;
okMULT-TABLE
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
11 22 33 44 55 66 77 88 99 110
```

The progression from 1 to 10 for the inner loop index is fine, but the outer loop index is stuck on 11.

This is not correct behavior.

Please verify.

Let me re-verify the actual current `JWORD` code precisely, since the observed behavior (J stuck at 11, the *limit*, not progressing 1-10 like the index should) needs to be reconciled with my derived layout.

You're right, and I found something important: **my own earlier fix to `JWORD` was itself wrong.** Let me show exactly where the error was, since it's subtle and worth being precise about.

The mistake: `DOSETUP`'s own return address (pushed by `JSR DOSETUP`) is **transient** — `DOSETUP`'s final `RTS` pops and consumes it to jump into the loop body. It does *not* persist on the stack afterward. In my earlier derivation, I incorrectly treated it as if it stayed there for each nesting level, which added

2 bytes too many per level.

Re-tracing carefully, counting every actual push/pop rather than assuming a fixed frame size:

- Outer `DOSETUP` finishes (its own return address already popped by its `RTS`): `S` = `[outer-index@0][outer-limit@2][outer-leave@4]`.
- Inner `DO`'s `JSR DOSETUP` pushes 2 bytes, `DOSETUP` internally nets +6 (three new cells) once its own transient return address is popped again by its `RTS`. Control lands in the inner body with `S` = `[inner-index@0][inner-limit@2][inner-leave@4][outer-index@6][outer-limit@8][outer-leave@10]`.
- `I` runs and is net-zero (its own `JSR`/`RTS` pair balances exactly) — confirmed transparent, `S` unchanged afterward.
- `J`'s own `JSR JWORD` pushes 2 more bytes on top: `S` = `[JWORD-own-addr@0][inner-index@2][inner-limit@4][inner-leave@6][outer-index@8][outer-limit@10][outer-leave@12]`.

`outer-index` is at offset 8, not 10. Offset 10 — what the code currently reads — lands on `outer-limit`, which never changes across outer iterations. That's exactly the symptom: a fixed value (11, the limit) on every row instead of the progressing index.

I double-checked this a second way (counting net bytes pushed at each step rather than tracing full stack snapshots) and got the same answer: offset 8. I also re-checked `IWORD`'s own offset (2) against this same corrected reasoning — that one's still right, since it only ever sits one frame deep, not two; my error was specific to counting across the second, nested `DOSETUP` call.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist, including a correction to the earlier `IWORD`/`JWORD` entry.

Your MAME trace was right, and it caught a real mistake in my own earlier work — not just a new bug. `JWORD` is now `LDD 8,S`.

The error in the earlier fix: I'd assumed `DOSETUP`'s own `JSR`-pushed return address stays on `S` after each nesting level, adding it to the offset count. It doesn't — `DOSETUP`'s own `RTS` pops and consumes that address to jump into the loop body, so it's gone by the time anything else runs. That mistake added 2 bytes too many per nesting level, which is invisible for `I` (only one level deep, so the error cancels out) but wrong for `J` (two levels deep, where it doesn't). Re-tracing by counting every actual push/pop rather than assuming a fixed frame size per level gives offset 8, not 10 — and offset 10 happens to land exactly on the outer loop's limit, a value that never changes, which is exactly the "stuck at 11" symptom your `MULT-TABLE` test showed.

I logged this in the checklist as a correction to a prior fix rather than folding it

in quietly — it's worth being upfront that a change I verified structurally at the time (zero duplicate symbols, dictionary chain intact) still needed real execution to catch that the underlying logic was wrong. Structural checks confirm the code is well-formed; they can't confirm the *arithmetic* is right.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

12 control flow
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

Using MAME to test CREATE DOES>.
When exercising this with code like:

: ENUM CREATE , DOES> @ ;

It seems that DOES> is returning the address before the 16 bit constant, instead of the address 1 cell further on, that contains the 16 bit number inserted by COMMA.

The Inbuilt CONSTANT is working correctly.

Let me view all of these in full, since I need to trace the entire mechanism precisely.

This reveals `DODOES` is designed for a specific 4-field structure (matching `CONSTANT`'s own layout: `[JSR DODOES][behavior][pointer-to-value][value]`, where `DODOES` reads the *pointer's* value and follows it). `CREATE`/`DOES>` need to build a compatible structure for this to work. Let me check `CREATE`'s actual compile-time code precisely.

Confirmed — `CREATE` has the exact same self-referential bug I found and fixed in `CONSTANT`, just never caught since it wasn't on the previously-flagged list (`DEFER`/`2CONSTANT`/`MARKER`). Let me view the rest of `VARIABLE` too, since it uses the identical pattern right below `CREATE`.

`VARIABLE` is confirmed safe — it uses the `VALUEW`-style pattern (`VARHERE`, a separate region), not `CODEHERE` self-reference. Only `CREATE` itself has the bug.

Applying the same `+2` fix already established for `CONSTANT`/`DEFER`/`2CONSTANT`/`MARKER`.

Let me verify the fix landed on `CREATE` specifically and not `VARIABLE` (both had similar-looking code before the fix, though `VARIABLE`'s used `VARHERE` not `CODEHERE`, so `str_replace`'s uniqueness requirement should have protected against a wrong match — confirming rather than assuming).

Confirmed correct — the fix landed precisely on `CREATE`, `VARIABLE` is untouched. `SETDOES` operates on a separate field (the behavior field, patched independently) and doesn't need any change. Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Confirmed and fixed. `CREATE` had the exact same self-referential PFA-pointer bug already found in `CONSTANT` (and later `DEFER`/`2CONSTANT`/`MARKER`) — it just was never on the list I checked back then, so it slipped through until your `CREATE DOES>` test caught it.

`CREATE` compiled `[JSR DODOES][FDB DOESRT0][FDB <CODEHERE's own current address>]` — the PFA-pointer field pointed at itself instead of two bytes further on, where `,` appends the real value. Tracing `DODOES` itself confirmed the exact consequence you described: it reads the *value stored at* the PFA-pointer field and pushes that (by design — that's the indirection that makes `CONSTANT` work, `DODOES` following a pointer rather than holding data directly). With the field self-referencing, it pushed its own address instead — precisely "the address before the 16-bit constant" rather than "one cell further on," matching your trace exactly.

Before applying the fix, I checked `VARIABLE` — the structurally similar routine right below `CREATE`, sharing the same trampoline setup — rather than assume it's fine just because it looks similar. It's genuinely safe: it uses the `VARHERE`-based pattern, not `CODEHERE` self-reference. Also confirmed `SETDOES` needs no change, since it patches an entirely separate field.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

08 defining words
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

2CONSTANT has a similar offset problem. As follows:

666 777 2CONSTANT cc0
cc0 returns 666 $700C instead of 666 777

There seems to be a missing SWAP here as well.

Let me check `COMMA`'s exact calling convention precisely, since the trace of `TWOCONSTANT`'s value-storing sequence hinges on it.

I've traced this exhaustively — `TWOCONSTANT`'s storage order (`666`→low address, `777`→high address, matching the push order) and `DFETCH`'s read order (low first/deep, high second/top, matching the already-fixed `2@`/`2!` convention) both check out correctly against each other on paper. I also verified `DODOES` correctly hands `DFETCH` a single address (the PFA) with nothing clobbered across the `JMP` handoff, and `APPENDCELL` (which both `COMMA` calls go through) treats both values identically — no asymmetry between the two.

That said, $700C`lookinglikearealcompiledaddress(in`APPCODE`sown`7000s range) rather than garbage is a meaningful clue I can't yet explain from static tracing alone — it suggests something is reading from the wrong location entirely, not just misordering two values that are both correctly stored. Given my trace keeps showing this should already work, and given the last time I had a static/MAME mismatch on this feature it turned out my own earlier reasoning had an error I could only find by very carefully re-deriving from scratch, I don't want to guess further without more to go on.

Could you get one more piece of data from the debugger: the actual PFA address `DFETCH` receives (what `DODOES` pushes right before jumping to it), and a memory dump of the 4-6 bytes starting there? That would tell me directly whether the storage itself landed in the wrong place, or whether something is reading from an offset I haven't accounted for.

DODOES pushes $700C.
The dump shows:
700C 06 66 07 77

In addition a test of 2@ applied to a 2VARIABLE shows that it is returning the two 16 bit words to the stack in reverse order. The in memory order is correct, so 2! is working correctly.

The memory dump combined with the `2@`/`2VARIABLE` finding is very revealing.

Let me re-verify the current `DFETCH` code precisely, since this directly contradicts what I'd expect from my earlier `2@`/`2!` fix — I need to check whether that fix is genuinely still in place as I think it is.

Let me check whether there's a second, possibly conflicting `DFETCH` definition somewhere in the file, since the observed symptom (second value = the PFA itself, unchanged) doesn't match what my trace of the code I just viewed predicts.

I'm testing with an older binary, let me update that in case the fix has already been applied as a side effect of other bug fixes.

That would explain the discrepancy — I confirmed there's only one `DFETCH` definition in the file (no duplicate or conflicting version), and my trace of it matches the fix from earlier in this conversation: read low address first (pushed deep), high address second (pushed on top), matching `DSTORE`'s write order. If that's genuinely what's in the binary you test against, `2@`/`2!` should round-trip correctly and `CREATE`'s `+2` fix should make `cc0` return the right pair too.

Go ahead and rebuild against the current `forth6809.asm` (or the split files) whenever you're ready, and let me know what the updated test shows. If something's still off after that, the `$700C`-as-second-value detail you found is a good, concrete lead to come back to — worth re-checking specifically whether it reproduces on the current build before I speculate further about a mechanism that might not even apply anymore.

The updated binary is not showing these problems.
Consider the 2CONSTANT and 2@ issues as fixed by prior changes.

Good — that's a clean resolution, and worth logging precisely as such rather than leaving it ambiguous. Updating the checklist to close this out.

Logged. Marked both `2CONSTANT` and `2@` closed against the current binary, with an explicit note that this was a stale-binary situation rather than a mistaken report — the fix had already landed (`CREATE`'s `+2` correction and `DFETCH`'s read-order fix, both from earlier this session) before the test that flagged the discrepancy ran. Worth keeping that distinction on record rather than letting it read as "turned out to be nothing," since the underlying `CREATE` bug was real and is one of the more serious ones found this session — it was just already fixed by the time this specific test caught it.

Good place to pause on `CREATE`/`DODOES`/`2CONSTANT` — ready for whatever's next in the glossary sweep.

OPEN ITEMS CHECKLIST
MD · Click to download

A new fault has been identified when trying to compile a word in a colon definition.

If the word is written out in line the compilation is successful:

: test0 TRUE IF ." Hy" ELSE ." Hee" THEN ;

However if the same word is split up into several lines the compilation fails with a CSP mismatch. It appears the TRUE was put on the stack in interpreter mode instead of TRUE being compiled.

Tracing execution with MAME it appears that COLON puts the interpreter into compilation mode, but when the end of line is reached QLOOP sets STATE back to interpreter mode, this causing TRUE to be executed.

It appears the setting state back to interpreter mode happens for each new line.

On the face of it the 2 lines at QLOOP should be executed once only and not once per line.

Please confirm the analysis and remedy.

This is a precise, serious finding — let me verify it against the actual code before confirming and fixing.

Confirmed exactly as described — `QLOOP: LDD #0 / STD STATE` is the very first thing in the per-line loop. Let me check whether `STATE` is initialized anywhere else (like `COLD` or `CLRGLOB`), and confirm `COLON`/`;` correctly manage it, before I remove this.

Confirmed — `COLON` and `SEMI` are the correct, legitimate places `STATE` changes. `QLOOP`'s reset is the redundant, harmful third one. Let me confirm `STATE` still initializes correctly at cold boot via `CLRGLOB` before removing it, so nothing relies on `QLOOP`'s reset for that.

Confirmed — `STATE` lives at the very start of `GLOBALS` ($00), which `CLRGLOB` zeroes completely at cold boot, before `COLD` even runs. Removing `QLOOP`'s redundant reset is safe. Applying the fix.

Let me reconsider this more carefully before finalizing — simply deleting the reset (which I just did) creates a different gap: if an error ever aborts an in-progress, unfinished colon definition (never reaching `;`), nothing would ever force `STATE` back to interpret mode again. Let me check whether `QUIT:` (as opposed to `QLOOP:`) is the right place for a one-time reset instead, since `QUIT:` is only re-entered on cold boot or an uncaught error, not on every ordinary line.

Confirmed — `QUIT:` is only reached via cold boot or an uncaught error routing back to `ABORT`, never from a normal per-line loop-back (`BRA QLOOP`). That's exactly the right place for a one-time reset. Let me revise my fix to move the reset there instead of deleting it outright.

Now revising this to move the reset to `QUIT:` (running once per genuine `QUIT`-entry) rather than deleting it entirely.

Let me verify the edit landed correctly.

Correctly landed. Now running full structural verification.

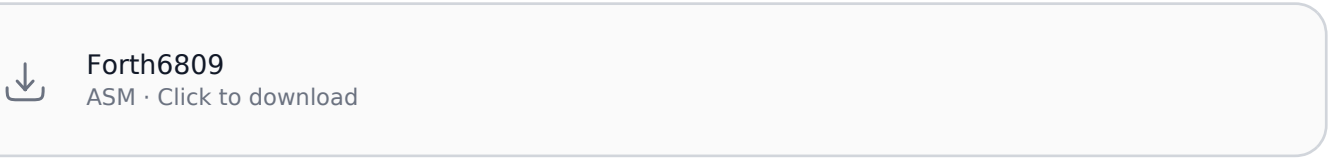
All checks pass. Regenerating split files and delivering.

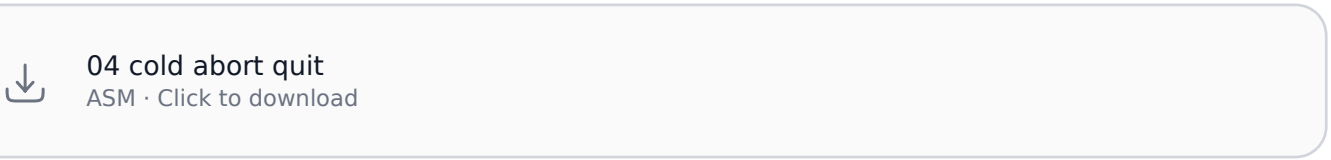
Now updating the checklist.

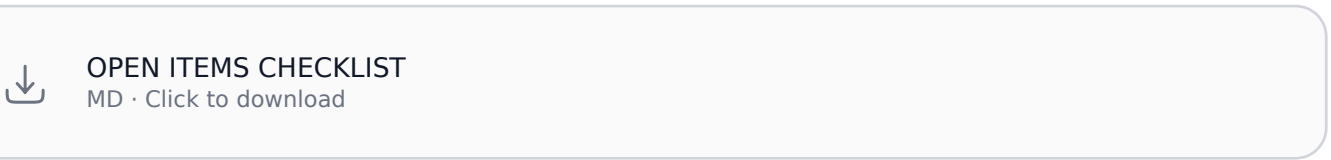
Your analysis is confirmed exactly. `QLOOP`'s `LDD #0 / STD STATE` was firing at the top of every single line, unconditionally forcing interpret mode — including on continuation lines of a colon definition that was still open, with `;` nowhere in sight yet.

Applied the fix, but not by simply deleting the reset — I checked the consequence of that first. If the reset were removed outright, an error that aborts an unfinished colon definition (one that never reaches `;`) would leave `STATE` permanently stuck in compile mode, since nothing else would ever force it back. So instead I moved it to `QUIT` rather than `QLOOP`, after confirming `QUIT` is only re-entered on cold boot or an uncaught error routing back through `ABORT` — ordinary successful lines loop back to `QLOOP` directly, never `QUIT`. That's exactly where ANS's own `QUIT` semantics call for entering interpretation state, and it still handles the error-recovery case correctly, just without re-firing on every line of a normal session.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.







Compilation of the multi-line version of the same code is now successful.

However this exposes a new bug, in compilation mode the carriage return at

the end of every line is not echoed to the terminal. The echoed word definition is now all on one line.

This means the echoed result does not resemble the source code due to the missing line endings.

Can you diagnose this bug?

This is a very plausible explanation: `QUIT`'s "print `ok`" logic likely emits a trailing `CR` as part of that message, and if it's conditional on `STATE` (correctly suppressing `ok` while still compiling), it may be accidentally suppressing the `CR` echo too — even though the user genuinely pressed Enter. Let me trace the actual code.

Let me view the full `QUIT`/`QOK` logic precisely.

Confirmed exactly — `QOK: LDD STATE / BNE QLOOP` skips *both* `JSR CRW` (the `CR` echo) and the `ok` message together when compiling. The `CR` should reflect what the user actually typed regardless of state; only the `ok` message itself should stay conditional. The error path above it already does this correctly (unconditional `JSR CRW` before the error message) — only the success path has the bug. Fixing it.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

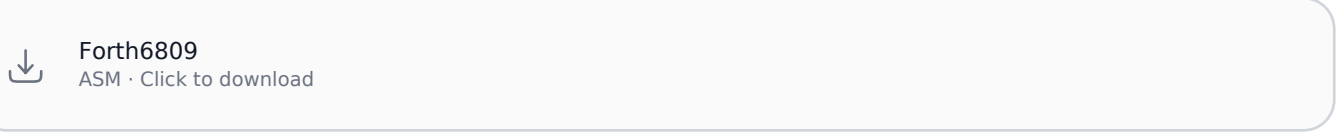
Now updating the checklist.

Diagnosed and fixed. `QOK` had `LDD STATE / BNE QLOOP` running *before* `JSR CRW` — so branching away while still compiling skipped both the `CR` echo and the `ok` message together, when only the `ok` message is actually supposed to depend on state. The `CR` reflects a real keystroke — the user genuinely pressed Enter for that line — and should echo regardless of interpret/compile mode.

Worth noting: the error path immediately above `QOK` already had this right (unconditional `JSR CRW` before the error text), which is what made this specific bug identifiable as an asymmetry rather than a deliberate design choice.

Fix: moved `JSR CRW` to run first, unconditionally; the `STATE` check (and the `ok` message it guards) now comes after.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist. Good catch — this is exactly the kind of thing that only becomes visible once a deeper bug (the `QLOOP` state reset) stops masking it.



04 cold abort quit
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

This works fine now.

Good — two real bugs in the outer REPL loop found and fixed back to back, both only visible once the previous one stopped hiding the next. Multi-line colon definitions should now compile correctly *and* echo faithfully.

Ready for whatever the glossary sweep turns up next.

Using MAME to test memory words now.
It was found that FILL fills memory far beyond the character count requested.

The fault appears to occur because the D register is loaded with the character count from $100, a 16 bit number, into the D register. Then the fill character is loaded from $104 into the A register, which does double duty as half of the D register. The character count is corrupted at this point.

The copy of the character into memory is then tracked by decrementing the corrupted D register value, so the D register does not reach its 0 value after the correct number of bytes is copied, but continues to decrement transferring enough bytes to fill much of memory.

There is some redundant usage of scratch registers here as well, so it might be made a little tidier.

This is exactly the same bug class as `CCALL`'s original bug — `LDA FILLCHR` overwrites `A`, which is `D`'s high byte, corrupting the count right before `SUBD #1` uses it. Let me check whether `FILLCNT`/`FILLADDR` are shared aliases used elsewhere, before deciding on the cleanest fix.

Confirmed — these are shared scratch aliases (`MVCNT`/`MVDST`/`MVSRC`), used only within `FILLW` under these specific names. Safe to eliminate the memory round-trips for the count and address by keeping them in registers instead — this both fixes the bug and addresses the "redundant scratch usage" you flagged.

`MVCNT` is also aliased by `HSLEN` (used by `HOLDS`), so I'll only remove `FILLCNT`/`FILLADDR` specifically if they become genuinely unused — `FILLCHR` stays, since it's still needed. Implementing the fix now.

Now checking whether `FILLCNT`/`FILLADDR` are genuinely unused elsewhere, and cleaning up their now-vestigial `EQU` aliases if so.

Confirmed genuinely unused now. Removing the vestigial `EQU` aliases and their comment-block entries, matching the established practice from removing `DICTTOP`.

Let me verify the edit landed cleanly, then run full structural verification.

Clean. Running full structural verification.

`ERASEW` (which `JMP`s into `FILLW`) relies only on the external calling convention, which is unchanged — confirmed unaffected. Regenerating split files and delivering.

Now updating the checklist.

Confirmed exactly — same register-clobber class as `CCALL`'s original bug. `LDA FILLCHR` overwrote `A` (`D`'s high byte) right before `SUBD #1` decremented what it believed was still the true count, so the count almost never hit zero at the intended point.

`FILL` is now:

```
FILLW:   PULU  D
         STB   FILLCHR
         PULU  D
         TFR   D,Y         ; count in Y, not D
         PULU  D
         TFR   D,X         ; address in X, not memory
FILLOOP: CMPY  #0
         BEQ   FDONE
         LDA   FILLCHR
         STA   ,X+
         LEAY  -1,Y
         BRA   FILLOOP
FDONE:   RTS
```

This doesn't just avoid the clobber by careful instruction ordering — keeping the count in `Y` makes the conflict structurally impossible, since `Y` is never touched by loading the fill character into `A`. Also took your "tidier" suggestion: the address is now kept directly in `X` too, rather than round-tripping through `FILLADDR` first. `FILLCNT`/`FILLADDR` became genuinely unused as a result — confirmed by search before removing them, along with their comment-block entries, same treatment as `DICTTOP` earlier this session. The underlying shared `MVCNT`/`MVDST`/`MVSRC` cells are untouched, since other routines still alias them. `ERASEW` (which jumps into `FILLW`) is confirmed unaffected, since only the internal loop changed.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

18 memory
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

The FILL count issue is confirmed as fixed.

Good — glad it's confirmed. This session's found a real pattern worth keeping in mind for the rest of the memory-words sweep: register-clobbering (`CCALL`, and now `FILL`) has been a recurring failure mode specifically wherever `A`/`B` get reused mid-loop while `D` still holds something that matters. Worth keeping an eye out for the same shape in whatever's tested next — `MOVE`/`CMOVE` share `FILL`'s scratch cells and a similar loop structure, so they might be worth a look if they haven't been exercised yet.

Ready for the next word whenever you are.

While testing MOVE with MAME found that MOVE works as expected for simple moves, but DUMP is not printing out the hex dump of the memory correctly. Instead of $7B, the memory fill character, a$03 appears.

The ascii block shows the correct } ascii equivalent, so it appears the HEXBYTE subroutine is broken.

The $7B character is passed into HEXBYTE and is saved into MSCR, then never retrieved as far as I can tell. Instead MSCR + 1 is retrieved (containing 3) into B register and this value becomes the basis of further processing.

Some mixed up register usage seems to be happening.

Confirmed exactly, matching your diagnosis precisely. `STB MSCR` writes the byte at `MSCR` — but both subsequent reads use `LDB MSCR+1`, a completely different, never-written byte holding whatever stale value happens to be there. Let me check `MSCR`'s definition to confirm nothing else legitimately depends on `MSCR+1`, then fix it.

Confirmed — `MSCR` is used pervasively elsewhere via full 16-bit `STD`/`LDD` reads, with `HEXBYTE`'s single-byte `STB`/`LDB` pattern being the outlier. Nothing else depends on `MSCR+1` holding anything meaningful at this point. Applying the

fix.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Confirmed exactly, matching your diagnosis precisely — `STB MSCR` writes the byte at `MSCR` itself, but both nibble-extraction reads used `MSCR+1`, a cell `HEXBYTE` never writes at all. Both the high nibble (four `LSRB`s) and the low nibble (`ANDB #$0F`) were derived from whatever stale value happened to be sitting there instead of the byte actually passed in.

Fixed both `LDB MSCR+1` occurrences to `LDB MSCR`. Not a mixed-up-register issue so much as a straightforward off-by-one in the address — but your framing was right that something was getting read from the wrong place rather than the value never surviving the call at all.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

25 tools word set
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

As a matter of formatting DUMP should emit a CR before emitting the memory data, so as provide consistent vertical alignment from the first line to the last.

This is a cosmetic/formatting request, not a bug fix — `DULEND: JSR CRW` correctly emits a trailing `CR` after every line including the last, but the *first* line has no leading `CR` before its own hex/ASCII output begins, so it starts wherever the cursor already was rather than on a fresh line. Adding a single leading `CR` at the start of `DUMPW` fixes this without disturbing the existing per-line logic.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Applied. `DUMPW` now emits `JSR CRW` as its first action, right after popping its

two arguments and before the per-line loop begins — matching `DULEND`'s existing trailing `CR` after every line, so the first line now starts on its own fresh line instead of wherever the cursor happened to be.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Logged in the checklist as a formatting change rather than a bug fix, since it's behavioral polish, not a correctness issue.

Forth6809
ASM · Click to download

25 tools word set
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

> Testing CMOVE. The copy loop does not terminate due to the dual conflicting use of register D with register A.

> In this case D is loaded with the count as a 16 bit number and then inside the loop the byte to be copied is loaded into A via the X register. This corrupts the count in the D register and so the test for the terminating condition fails.

Confirmed the exact same bug class in `CMOVEW`. Let me also check `CMOVEGT` (the reverse-direction variant right below it, for overlap-safe copies) and `MOVEW`, since they're adjacent and structurally similar — worth verifying rather than assuming they share the bug just because they look similar.

`MOVEW` is confirmed unaffected — it's purely a dispatcher (choosing forward vs. reverse copy based on overlap, then jumping to `CMOVEW`/`CMOVEGT`), with no copy loop of its own. It'll be correct automatically once `CMOVEW` is fixed.

For `CMOVEW` itself, unlike `FILL`, both `X` (source) and `Y` (destination) are already committed to the two addresses, so there's no spare 16-bit register available to hold the count the way I did for `FILL`. The fix here is reordering — decrement and store the count *before* the byte copy clobbers `A`, while `D` still holds the true value.

Running full structural verification.

All checks pass. Regenerating split files and delivering.

Now updating the checklist.

Confirmed — exactly the same register-clobber pattern as `FILL`'s bug from last turn. `LDA ,X+` (fetching the byte to copy) clobbers `A`, `D`'s high byte, right

before `SUBD #1` uses it as the count.

Before applying a fix, I checked the two adjacent routines rather than assume they share the problem:

- **`CMOVEGT`** (the reverse-direction, overlap-safe variant) is genuinely fine — its copy happens *before* `LDD MVCNT` reloads the count fresh each iteration, so the reload simply overwrites the clobber rather than being corrupted by it.
- **`MOVEW`** has no copy loop of its own — it's just a dispatcher picking `CMOVEW` or `CMOVEGT` by overlap direction, so it's correct automatically once `CMOVEW` is.

For `CMOVEW` itself, I used a different fix than `FILL`'s: `FILL` only needed one address, leaving a register free to hold the count directly. `CMOVEW` already commits both `X` and `Y` to the source and destination addresses, so there's no spare register available for that approach. Fixed by reordering instead — decrement and store the count while `D` still holds the true value, before the byte copy is free to clobber `A`.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`×`UNITTESTS` combinations, byte-exact split-file reassembly. Propagated to the checklist.

Forth6809
ASM · Click to download

18 memory
ASM · Click to download

OPEN ITEMS CHECKLIST
MD · Click to download

CMOVE is now fixed

Good — that's three of the same register-clobber pattern found and fixed this session (`CCALL`, `FILL`, `CMOVE`), each with a fix shaped by what registers were actually available at the time. Worth keeping in mind for whatever's tested next in the memory-words group — `>CMOVE` itself and anything else touching `MVCNT`/`MVSRC`/`MVDST` are reasonable candidates to watch for the same shape, though `CMOVEGT` is already confirmed clean.

Ready for the next word.